DEVOPS ENGINEERING MASTERY

OBSERVABILITY ENGINEERING

Prometheus, Grafana, Logging, Distributed Tracing, Alerting & Production Monitoring

VOLUME 10 • FIRST EDITION

```
nahid@technova:~$ curl -s localhost:9090/-/healthy
✓ Prometheus is Healthy
nahid@technova:~$ promtool check config prometheus.yml
✓ SUCCESS — 12 rules loaded
nahid@technova:~$ rate(http_requests_total{status=~"5.."}[5m])
```

Prometheus • Grafana • Loki • OpenTelemetry • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Observability Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 10 (Companion Workbook). First Edition, 9 November 2026. Prometheus, Grafana, Loki, Grafana Alloy and OpenTelemetry with a MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your monitoring lab without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab step on a production system. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

এই ওয়ার্কবুকটি **DEVOPS ENGINEERING MASTERY — Volume 10: Observability Engineering** বইয়ের সহচর। প্রতিটি ল্যাব হাতে-কলমে করার জন্য লেখা হয়েছে। TechNova একটি কাল্পনিক কোম্পানি; `technova.example` ডোমেইন এবং `203.0.113.x` IP ঠিকানা শুধু উদাহরণের জন্য। সব ল্যাব স্থানীয় (local) পরিবেশে, Docker Compose বা kind/Minikube দিয়ে করা যায় — কোনো ক্লাউড খরচের প্রয়োজন নেই। প্রতিটি ল্যাবে কমপক্ষে 4 GB RAM ও 2 CPU কোর প্রয়োজন; Kubernetes ল্যাবের জন্য 8 GB RAM সুপারিশ করা হয়।

> **WARNING:** এই ফাইলের কোনো উদাহরণে আসল পাসওয়ার্ড, API key বা secret লিখবেন না। `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন। নমুনা টার্মিনাল আউটপুট চিত্রণমূলক — আপনার মেশিনে সংখ্যা কিছুটা ভিন্ন হতে পারে।

> **CONCEPT:** Observability মানে শুধু monitoring নয়। Metrics (সংখ্যাগত পরিমাপ), Logs (ঘটনার বিবরণ) এবং Traces (একটি অনুরোধের যাত্রাপথ) — এই তিনটি স্তম্ভ (three pillars) একসাথে থাকলে আপনি প্রশ্ন করতে পারেন: কী হয়েছে, কেন হয়েছে, কোথায় হয়েছে।

সার্ভিস ও পোর্টের ধারাবাহিক নিয়ম (সব ল্যাবে একই): `api:3000`, `web:3000/5173`, `mongo:27017`, `prometheus:9090`, `grafana:3000`, `loki:3100`, `tempo:3200`, `jaeger:16686`, `otel-collector:4317/4318`। Grafana ও API দুটোই 3000 ব্যবহার করে বলে স্থানীয়ভাবে Grafana-কে 3000 এবং API-কে 3000 আলাদা Compose প্রজেক্টে বা ভিন্ন হোস্ট পোর্টে চালান — ল্যাবে যেখানে সংঘাত হতে পারে সেখানে স্পষ্ট নির্দেশ দেওয়া আছে।

ল্যাব 1 • Observability Lab Workbook

Lab 1 — Build a local monitoring environment

উদ্দেশ্য

এই ল্যাবের শেষে আপনি Docker Compose দিয়ে একটি সম্পূর্ণ স্থানীয় monitoring পরিবেশ তৈরি করতে পারবেন — Prometheus, Grafana, Node Exporter এবং একটি নমুনা অ্যাপ — এবং প্রতিটি সার্ভিসের health যাচাই করতে পারবেন। এটি পুরো বইয়ের ভিত্তি; পরের সব ল্যাব এই পরিবেশের উপর দাঁড়িয়ে আছে।

প্রয়োজনীয় টুল

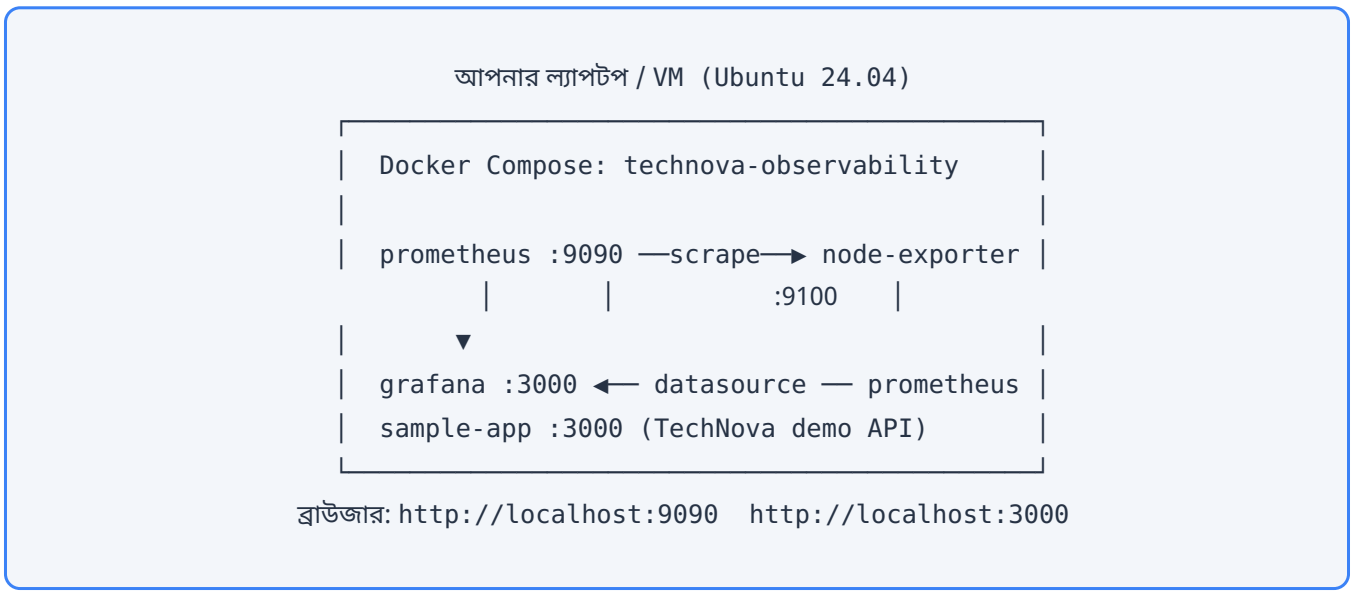
- Ubuntu Server 24.04 LTS (বা Ubuntu Desktop / WSL2)
- Docker Engine 26+ এবং Docker Compose v2

- curl, git, একটি টেক্সট এডিটর
- কমপক্ষে 4 GB RAM, 2 CPU, 10 GB ফাঁকা ডিস্ক

পূর্বশর্ত

Docker ইনস্টল থাকতে হবে এবং আপনার ব্যবহারকারী `docker` গ্রুপে থাকতে হবে। ইন্টারনেট সংযোগ প্রয়োজন — প্রথমবার image pull করতে হবে। কোনো ক্লাউড অ্যাকাউন্ট বা খরচের প্রয়োজন নেই।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — Docker যাচাই করুন

```
docker --version
docker compose version
docker ps
```

দুটো কমান্ডই সংস্করণ নম্বর দেখাবে। `docker ps` খালি তালিকা দিলেও চলবে, তবে permission error দিলে `sudo usermod -aG docker $USER` চালিয়ে লগ আউট করে আবার লগ ইন করুন।

ধাপ ২ — প্রজেক্ট কাঠামো তৈরি করুন

```
mkdir -p ~/technova-observability/prometheus ~/technova-observability/grafana/provisioning/
datasources
cd ~/technova-observability
ls -R
```

ধাপ ৩ — Prometheus কনফিগারেশন লিখুন

`prometheus/prometheus.yml` ফাইলে নিচের কনফিগারেশন লিখুন (সম্পূর্ণ ফাইল পরের অংশে দেওয়া আছে)।

ধাপ ৪ — Compose ফাইল লিখুন ও চালু করুন

```
docker compose up -d
docker compose ps
docker compose logs -f prometheus
```

সব কনটেইনার `Up` অবস্থায় আসা পর্যন্ত অপেক্ষা করুন। Grafana প্রথম চালুতে 20–30 সেকেন্ড নিতে পারে।

ধাপ ৫ — UI যাচাই করুন

```
curl -s http://localhost:9090/-/healthy
curl -s http://localhost:3000/api/health
curl -s http://localhost:9100/metrics | head -20
```

ব্রাউজারে `http://localhost:9090` খুলুন — Prometheus UI দেখা যাবে। `http://localhost:3000` খুলুন — Grafana লগইন পেজ আসবে (ডিফল্ট `admin / admin`, প্রথম লগইনে পাসওয়ার্ড বদলাতে বলবে)।

সম্পূর্ণ কনফিগারেশন

docker-compose.yml

```
services:
  prometheus:
    image: prom/prometheus:v3.0.0
    container_name: prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus-data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--storage.tsdb.retention.time=15d'
    ports:
      - "9090:9090"
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:v1.8.2
    container_name: node-exporter
    ports:
      - "9100:9100"
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--path.rootfs=/rootfs'
      - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($$|/)'
    restart: unless-stopped

  grafana:
    image: grafana/grafana:11.4.0
```

```
    container_name: grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=CHANGE-ME-grafana-admin
      - GF_USERS_ALLOW_SIGN_UP=false
    restart: unless-stopped

volumes:
  prometheus-data:
  grafana-data:
```

prometheus/prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

grafana/provisioning/datasources/prometheus.yml

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: true
```

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `docker compose ps` — তিনটি সার্ভিসই `Up`।
- Prometheus UI → Status → Targets — `prometheus` ও `node-exporter` দুটোই `UP` (সবুজ)।
- Prometheus UI-তে `up` কুয়েরি চালালে দুটি সিরিজের মান `1`।
- Grafana → Connections → Data sources — Prometheus `working` দেখায়।
- `curl http://localhost:9100/metrics` — `node_cpu_seconds_total`, `node_memory_MemTotal_bytes` লাইন দেখা যায়। (নমুনা আউটপুট — চিত্রণমূলক)

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Port 9090 already in use	আগের Prometheus চলছে	`docker ps` দিয়ে খুঁজে `docker stop` করুন, বা Compose-এ পোর্ট বদলে `9091:9090` করুন
Target DOWN, connection refused	কনটেইনার নাম ভুল বা সার্ভিস চালু হয়নি	`docker compose logs node-exporter` দেখুন; Compose নেটওয়ার্কে সার্ভিস নাম দিয়ে যোগাযোগ হয়, `localhost` দিয়ে নয়
Grafana লগইন ব্যর্থ	পাসওয়ার্ড ভুল	`docker compose down` করে ভলিউম মুছে আবার চালু করুন, অথবা `grafana-cli admin reset-admin-password`
permission denied (docker)	ব্যবহারকারী docker গ্রুপে নেই	`sudo usermod -aG docker $USER` তম লগ আউট/লগ ইন

পরিষ্কারকরণ (Cleanup)

```
docker compose stop
docker compose down
docker compose down -v   # শুধু তখনই, যখন ডেটা মুছতে চান
```

> **WARNING:** `docker compose down -v` Prometheus ও Grafana-র সব সংরক্ষিত ডেটা (metrics, dashboard) মুছে দেয়। পরের ল্যাবে ডেটা দরকার হলে শুধু `docker compose down` (v ছাড়া) ব্যবহার করুন।

পর্যালোচনা প্রশ্ন

1. Prometheus কীভাবে metrics সংগ্রহ করে — push না pull? এর সুবিধা কী?
2. `scrape_interval: 15s` মানে কী? 5s করলে কী ট্রেড-অফ হবে?
3. Grafana কেন Prometheus-কে `http://prometheus:9090` দিয়ে ডাকে, `localhost` দিয়ে নয়?
4. Node Exporter হোস্টের কোন কোন metrics দেয়? তিনটি উদাহরণ দিন।
5. `retention.time=15d` না দিলে ডিফল্ট কী হয়?

ল্যাব 2 • Observability Lab Workbook

Lab 2 — Collect Linux server metrics

উদ্দেশ্য

CPU, মেমরি, ডিস্ক ও নেটওয়ার্ক — একটি Linux সার্ভারের চারটি মূল সম্পদ (resource) কীভাবে পরিমাপ করা হয় তা শিখবেন। `/proc` ও `/sys` থেকে আসা কাঁচা সংখ্যাকে Node Exporter কীভাবে Prometheus metric-এ রূপান্তর করে, তা বুঝবেন এবং হোস্ট-লেভেল metric দিয়ে সার্ভারের স্বাস্থ্য নির্ণয় করতে পারবেন।

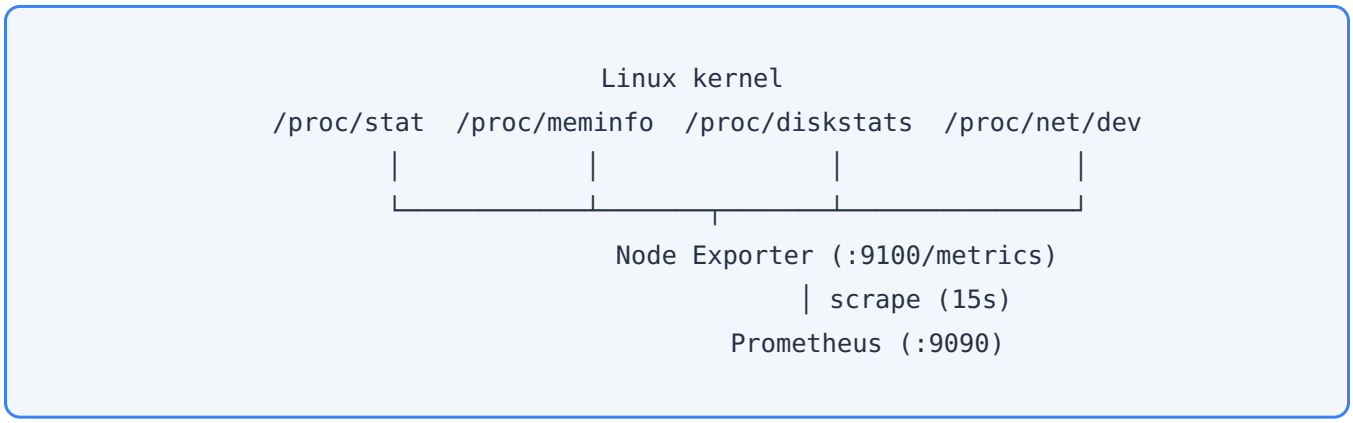
প্রয়োজনীয় টুল

- Lab 1-এর পরিবেশ (চালু অবস্থায়)
- Node Exporter 1.8.x, Prometheus 3.x
- `htop`, `df`, `free`, `ss` (ঐচ্ছিক, তুলনার জন্য)

পূর্বশর্ত

Lab 1 সম্পূর্ণ এবং `node-exporter` target `UP`। হোস্টে `stress-ng` থাকলে লোড তৈরি করে পরীক্ষা করা যাবে (ঐচ্ছিক)।

আর্কিটেকচার



Node Exporter নিজে কোনো হিসাব করে না — সে kernel-এর কাউন্টার পড়ে HTTP endpoint-এ প্রকাশ করে। হার (rate) বা শতাংশের হিসাব PromQL-এ করতে হয়।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — কাঁচা metrics দেখুন

```
curl -s http://localhost:9100/metrics | grep -E 'node_cpu_seconds_total|node_memory_MemTotal|
node_filesystem_size_bytes' | head -20
```

ধাপ ২ — হোস্টের আসল অবস্থার সাথে মেলান

```
free -h
df -h /
uptime
ss -tuln | head
```

`free`-এর মেমরি সংখ্যার সাথে `node_memory_MemTotal_bytes` ও `node_memory_MemAvailable_bytes` মিলিয়ে দেখুন।

ধাপ ৩ — Prometheus-এ CPU ব্যবহারের কুয়েরি চালান

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

এটি গত 5 মিনিটের গড় CPU ব্যবহার শতাংশে দেয়। Prometheus UI-র Graph ট্যাবে চালিয়ে দেখুন।

ধাপ ৪ — মেমরি ও ডিস্ক কুয়েরি

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
(node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"}) *
100
```

ধাপ ৫ — লোড তৈরি করে পর্যবেক্ষণ করুন (ঐচ্ছিক)

```
sudo apt-get install -y stress-ng
stress-ng --cpu 2 --timeout 60s &
# Prometheus-এ CPU গ্রাফ দেখুন — 60 সেকেন্ডে ব্যবহার বাড়বে, তারপর নামবে
```

সম্পূর্ণ কনফিগারেশন

এই ল্যাবে নতুন কোনো কনফিগারেশন ফাইলের প্রয়োজন নেই — Lab 1-এর Node Exporter-ই যথেষ্ট। তবে হোস্টে সরাসরি (Docker ছাড়া) চালাতে চাইলে systemd unit:

```
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=nobody
ExecStart=/usr/local/bin/node_exporter \
  --collector.filesystem.mount-points-exclude='^/(sys|proc|dev|host|etc)($$|/)'
Restart=always

[Install]
WantedBy=multi-user.target
```

প্রধান metric-এর রেফারেন্স:

Metric	অর্থ	টাইপ

`node_cpu_seconds_total`	প্রতি CPU মোডে ব্যয়িত মোট সেকেন্ড	Counter
`node_memory_MemTotal_bytes`	মোট RAM	Gauge
`node_memory_MemAvailable_bytes`	ব্যবহারযোগ্য RAM	Gauge
`node_filesystem_size_bytes`	ফাইলসিস্টেমের মোট আকার	Gauge
`node_filesystem_avail_bytes`	ফাঁকা জায়গা	Gauge
`node_disk_io_time_seconds_total`	ডিস্ক ব্যস্ত থাকার মোট সময়	Counter
`node_network_receive_bytes_total`	নেটওয়ার্কে প্রাপ্ত মোট বাইট	Counter
`node_load1` / `node_load5`	1 / 5 মিনিটের load average	Gauge

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Prometheus-এ `node_cpu_seconds_total` কুয়েরিতে প্রতি CPU কোরে `idle`, `user`, `system`, `iowait` মোডের সিরিজ দেখা যায়।
- CPU শতাংশের কুয়েরি 0–100-এর মধ্যে মান দেয়; `stress-ng` চলাকালে মান স্পষ্ট বাড়ে।
- মেমরি শতাংশ `free -h`-এর সাথে সামঞ্জস্যপূর্ণ।
- ডিস্ক ফাঁকা শতাংশ `df -h /`-এর সাথে মেলে।

> **CONCEPT:** Counter কখনো কমে না (রিসেট ছাড়া) — তাই Counter দিয়ে সরাসরি শতাংশ বের করা যায় না; `rate()` দিয়ে প্রতি সেকেন্ডের হার বের করতে হয়। Gauge যেকোনো দিকে যেতে পারে — মেমরি, ডিস্ক, তাপমাত্রা Gauge।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
CPU কুয়েরি 100-র বেশি দেখায়	`avg` ছাড়া প্রতি কোরের হার যোগ হয়েছে	`avg(rate(...))` ব্যবহার করুন, অথবা `sum` করে কোর সংখ্যা দিয়ে ভাগ করুন
`node_filesystem_*` খালি	mount-point exclude নিয়মে `/` বাদ পড়েছে	Compose-এর `--collector.filesystem.mount-points-exclude` যাচাই করুন
Docker-এর ভেতরে হোস্টের নয়, কনটেইনারের metrics	`/proc` মাউন্ট করা হয়নি	Lab 1-এর Compose-এ `/proc:/host/proc:ro` মাউন্ট নিশ্চিত করুন
Load average বেশি কিন্তু CPU কম	I/O wait বা D-state প্রসেস	`node_cpu_seconds_total{mode="iowait"}` ও `node_disk_io_time_seconds_total` দেখুন

পরিষ্কারকরণ (Cleanup)

```
pkill -f stress-ng || true
# পরিবেশ পরের ল্যাবের জন্য চালু রাখুন; বন্ধ করতে চাইলে:
docker compose down
```

পর্যালোচনা প্রশ্ন

1. Counter ও Gauge-এর পার্থক্য কী? প্রতিটির দুটি উদাহরণ দিন।
2. `rate(node_cpu_seconds_total[5m])` কেন সরাসরি metric-এর চেয়ে অর্থবহ?
3. Load average 4.0 মানে কী — 2-কোর মেশিনে কি তা সমস্যা?
4. `iowait` বেশি থাকলে আপনি কোন metric-গুলো পাশাপাশি দেখবেন?
5. Node Exporter কেন হোস্টের root ফাইলসিস্টেমকে `/rootfs` হিসেবে মাউন্ট করে?

ল্যাব 3 • Observability Lab Workbook

Lab 3 — Install and configure Prometheus

উদ্দেশ্য

Prometheus-কে Docker-এর বাইরে, সরাসরি Linux সার্ভারে (binary) ইনস্টল করে এর কনফিগারেশন, storage (TSDB), retention ও service ব্যবস্থাপনা শিখবেন। Docker ভার্সনের সাথে তুলনা করে বুঝবেন কখন কোন ইনস্টলেশন পদ্ধতি উপযুক্ত।

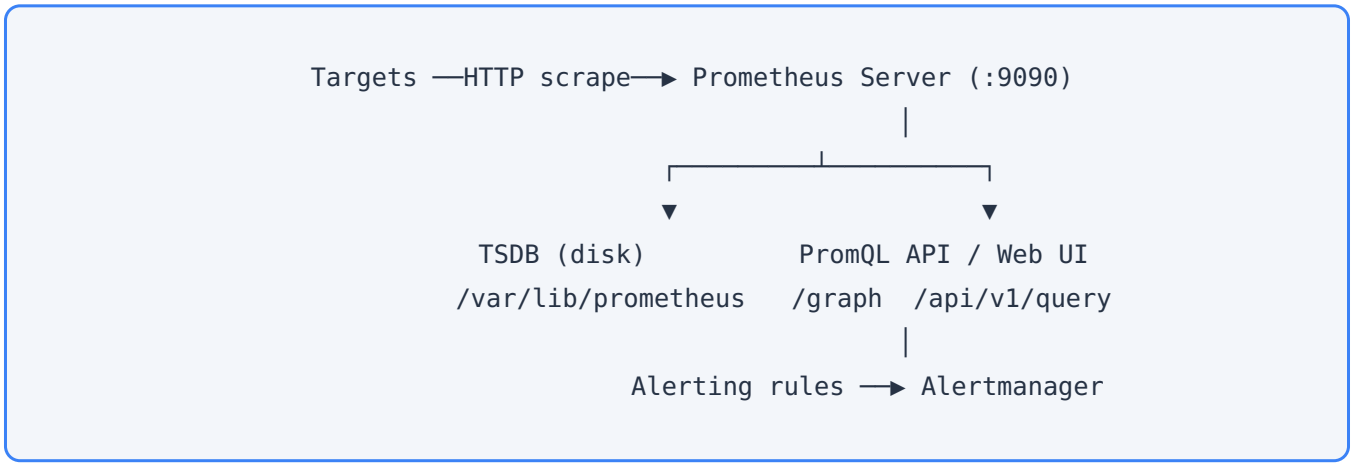
প্রয়োজনীয় টুল

- Ubuntu 24.04, Prometheus 3.x binary (prometheus.io থেকে)
- `systemd`, `curl`, `tar`
- 2 GB RAM, 10 GB ডিস্ক (TSDB-র জন্য)

পূর্বশর্ত

Lab 1–2 সম্পূর্ণ। Prometheus-এর ডাউনলোড পেজ থেকে আপনার আর্কিটেকচারের (amd64/arm64) সঠিক সংস্করণ বেছে নিন — সংস্করণ নম্বর ডাউনলোডের সময় যাচাই করুন।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — ব্যবহারকারী ও ডিরেক্টরি তৈরি

```
sudo useradd --no-create-home --shell /bin/false prometheus
sudo mkdir -p /etc/prometheus /var/lib/prometheus
sudo chown prometheus:prometheus /etc/prometheus /var/lib/prometheus
```

ধাপ ২ — Binary ডাউনলোড ও ইনস্টল

```
cd /tmp
curl -LO https://github.com/prometheus/prometheus/releases/download/v3.0.0/
prometheus-3.0.0.linux-amd64.tar.gz
tar xvf prometheus-3.0.0.linux-amd64.tar.gz
cd prometheus-3.0.0.linux-amd64
sudo cp prometheus promtool /usr/local/bin/
sudo cp -r consoles console_libraries /etc/prometheus/
prometheus --version
```

> **WARNING:** ডাউনলোড URL-এর সংস্করণ নম্বর উদাহরণ মাত্র — prometheus.io/download থেকে বর্তমান স্থিতিশীল সংস্করণ যাচাই করে নিন। Checksum যাচাই করতে প্রকাশিত SHA256 ফাইল ব্যবহার করুন।

ধাপ ৩ — কনফিগারেশন লিখুন ও যাচাই করুন

```
sudo cp prometheus.yml /etc/prometheus/prometheus.yml
sudo chown -R prometheus:prometheus /etc/prometheus
promtool check config /etc/prometheus/prometheus.yml
promtool check rules /etc/prometheus/rules/*.yml || echo "no rules yet - ok"
```

ধাপ ৪ — systemd service তৈরি ও চালু

```
sudo systemctl daemon-reload
sudo systemctl enable --now prometheus
sudo systemctl status prometheus --no-pager
curl -s http://localhost:9090/-/healthy
```

ধাপ ৫ — TSDB ও retention পরীক্ষা

```
curl -s http://localhost:9090/api/v1/status/tsdb | head -c 500; echo
du -sh /var/lib/prometheus
```

সম্পূর্ণ কনফিগারেশন

/etc/systemd/system/prometheus.service

```
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus \
  --storage.tsdb.retention.time=15d \
```

```
  --storage.tsdb.retention.size=10GB \
  --web.console.templates=/etc/prometheus/consoles \
  --web.console.libraries=/etc/prometheus/console_libraries \
  --web.enable-lifecycle
Restart=always

[Install]
WantedBy=multi-user.target
```

/etc/prometheus/prometheus.yml (Lab 1-এর ফাইলের বর্ধিত রূপ)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: 'technova-lab'
    replica: 'prom-01'

rule_files:
  - 'rules/*.yml'

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['localhost:9093']

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['localhost:9100']
```

> **CONCEPT:** `retention.time` ও `retention.size` — যেটি আগে পূর্ণ হবে সেটিই কার্যকর হবে। `--web.enable-lifecycle` থাকলে `POST /-/reload` দিয়ে কনফিগারেশন রিলোড করা যায়, সার্ভিস রিস্টার্ট ছাড়াই।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `systemctl status prometheus` — `active (running)`।
- `promtool check config` — `SUCCESS`।
- UI-তে Status → Runtime & Build Information — সংস্করণ ও retention দেখা যায়।
- `/api/v1/status/tsdb` — head statistics ও series সংখ্যা দেখায়।
- কনফিগারেশন বদলে `curl -X POST http://localhost:9090/-/reload` দিলে রিস্টার্ট ছাড়াই নতুন কনফিগারেশন কার্যকর হয়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান

Service failed to start	ডিরেক্টরির মালিকানা ভুল	`journalctl -u prometheus -n 50` দেখুন; `chown -R prometheus:prometheus /var/lib/prometheus /etc/prometheus`
promtool: config error	YAML indentation ভুল	`promtool check config`-এর লাইন নম্বর দেখে ঠিক করুন; ট্যাব ব্যবহার করবেন না
ডিস্ক দ্রুত ভরে যাচ্ছে	Retention বেশি / cardinality বেশি	`retention.size` কমান; Status → TSDB-তে series সংখ্যা দেখুন
Port 9090 সংঘাত	Docker Prometheus-ও চলছে	একটি বন্ধ করুন: `docker compose stop prometheus` অথবা `--web.listen-address=:9091`

পরিষ্কারকরণ (Cleanup)

```
sudo systemctl stop prometheus
sudo systemctl disable prometheus
# সম্পূর্ণ মুছতে চাইলে (ডেটা সহ):
sudo rm -rf /var/lib/prometheus /etc/prometheus /usr/local/bin/prometheus /usr/local/bin/promtool
sudo userdel prometheus
```

> **WARNING:** `rm -rf /var/lib/prometheus` সব সংরক্ষিত metrics চিরতরে মুছে দেয়। Lab 4 থেকে আবার Docker Compose পরিবেশ ব্যবহার করবেন — binary ইনস্টলেশন শুধু এই ল্যাবের শেখার জন্য।

পর্যালোচনা প্রশ্ন

1. TSDB কী? Prometheus কেন রিলেশনাল ডেটাবেস ব্যবহার করে না?
2. `external_labels` কখন কাজে লাগে?
3. Retention time বনাম size — কোনটি আগে কার্যকর হয় এবং কেন দুটোই দেওয়া ভালো?
4. High availability-র জন্য একাধিক Prometheus কীভাবে চালানো হয়? (Thanos/Cortex-এর নাম শুনে থাকলে লিখুন)
5. `promtool check config` কেন প্রতিবার কনফিগারেশন বদলের পর চালানো উচিত?

ল্যাব 4 • Observability Lab Workbook

Lab 4 — Configure scrape targets

উদ্দেশ্য

Static config, file-based service discovery ও DNS-based discovery — তিনভাবে scrape target কনফিগার করতে শিখবেন। Target-এর label (যেমন `env`, `team`) দিয়ে metrics-কে সংগঠিত করা এবং scrape ব্যর্থতা নির্ণয় করা এই ল্যাবের মূল কাজ।

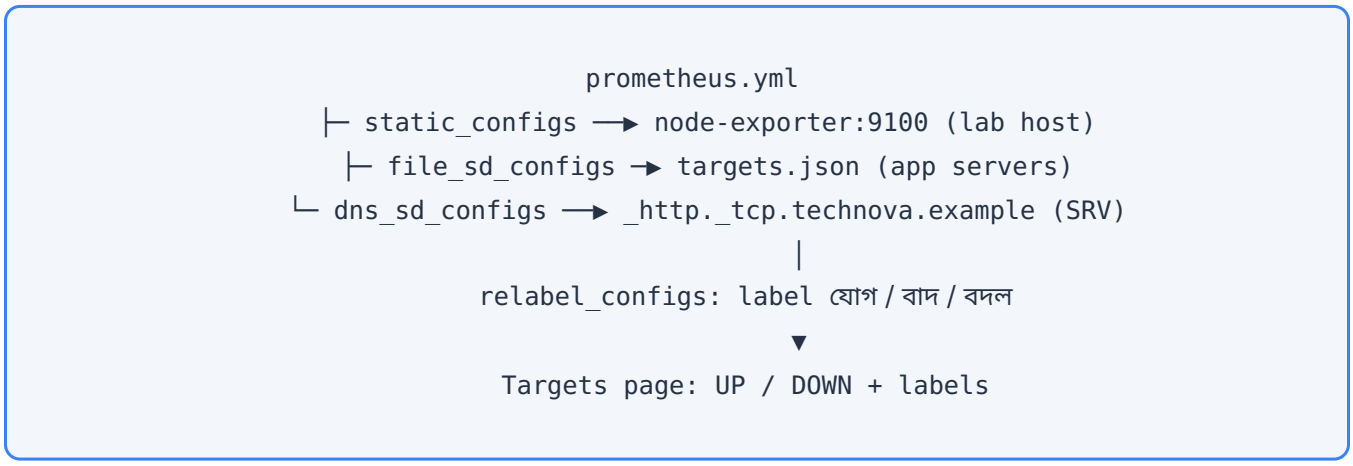
প্রয়োজনীয় টুল

- Lab 1-এর Docker Compose পরিবেশ
- দ্বিতীয় একটি Node Exporter (ভিন্ন পোর্টে) ও একটি নমুনা HTTP সার্ভিস

পূর্বশর্ত

Prometheus চলছে এবং Targets পেজ দেখতে জানেন। YAML-এর indentation সম্পর্কে ধারণা আছে।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — Static target যোগ করুন

`prometheus.yml` -এ একটি নতুন job যোগ করুন:

```
- job_name: 'technova-api'
  scrape_interval: 10s
  static_configs:
    - targets: ['api:3000']
      labels:
        env: 'lab'
        team: 'platform'
```

ধাপ ২ — File-based discovery ব্যবহার করুন

`prometheus/targets.json` তৈরি করুন:

```
[
  {
    "targets": ["node-exporter:9100"],
    "labels": {"env": "lab", "role": "host", "dc": "dhaka-lab"}
  },
  {
    "targets": ["web:5173"],
    "labels": {"env": "lab", "role": "frontend"}
  }
]
```

আর `prometheus.yml`-এ job যোগ করুন:

```
  - job_name: 'file-sd-hosts'
    file_sd_configs:
      - files:
          - /etc/prometheus/targets.json
        refresh_interval: 30s
```

Compose-এ `targets.json`-কে volume হিসেবে মাউন্ট করুন।

ধাপ ৩ — Relabeling দিয়ে label পরিষ্কার করুন

```
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
    relabel_configs:
      - source_labels: [__address__]
        target_label: instance
        replacement: 'technova-lab-host'
      - regex: 'node_(.+)'
        action: labeldrop
        # সতর্কতা: এটি উদাহরণ — আসলে metric label drop এভাবে হয় না;
        # metric_relabel_configs ব্যবহার করুন (নিচে দেখুন)
```

> **TROUBLESHOOTING:** `relabel_configs` target-এর label নিয়ে কাজ করে; metric-এর label বদলাতে `metric_relabel_configs` লাগে। দুটো গুলিয়ে ফেললে কুয়েরিতে label খুঁজে পাবেন না।

ধাপ ৪ — Reload ও যাচাই

```
docker compose exec prometheus promtool check config /etc/prometheus/prometheus.yml
curl -X POST http://localhost:9090/-/reload
curl -s http://localhost:9090/api/v1/targets | python3 -m json.tool | head -60
```

সম্পূর্ণ কনফিগারেশন

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
        labels: {env: 'lab'}

  - job_name: 'technova-api'
    scrape_interval: 10s
    metrics_path: /metrics
    static_configs:
      - targets: ['api:3000']
        labels: {env: 'lab', team: 'platform'}

  - job_name: 'file-sd-hosts'
    file_sd_configs:
      - files: ['/etc/prometheus/targets.json']
        refresh_interval: 30s
    metric_relabel_configs:
      # উচ্চ-cardinality label বাদ দেওয়ার উদাহরণ
      - regex: 'container_id'
        action: labeldrop
```

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Targets পেজে প্রতিটি job-এর অধীনে target `UP`, Last Scrape সময় 15 সেকেন্ডের মধ্যে।
- `up{job="technova-api"}` কুয়েরিতে `env="lab"` label দেখা যায়।
- `targets.json` বদলে 30 সেকেন্ডের মধ্যে নতুন target Targets পেজে আসে — Prometheus রিস্টার্ট লাগে না।
- ভুল target (যেমন বন্ধ পোর্ট) `DOWN` দেখায় এবং Error কলামে `connection refused` লেখা থাকে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
File SD target আসছে না	JSON ফরম্যাট ভুল বা মাউন্ট হয়নি	`docker compose exec prometheus cat /etc/prometheus/targets.json` ও `python3 -m json.tool` দিয়ে যাচাই করুন
Target UP কিন্তু metric নেই	`metrics_path` ভুল	সার্ভিসের `/metrics` endpoint ব্রাউজারে খুলে দেখুন
Scrape timeout	Target ধীর বা নেটওয়ার্ক ল্যাগ	Job-এ `scrape_timeout: 10s` বাড়ান; target-এর লগ দেখুন

Label দেখা যাচ্ছে না	relabel নিয়মে ভুল regex	Targets পেজের Discovered Labels দেখুন — সেখানে মূল label থাকে

পরিষ্কারকরণ (Cleanup)

```
docker compose down
# পরীক্ষামূলক targets.json রাখতে চাইলে মুছবেন না; পরের ল্যাবে আবার লাগবে
```

পর্যালোচনা প্রশ্ন

1. Static config কখন যথেষ্ট, আর কখন service discovery দরকার?
2. File SD-তে নতুন সার্ভার যোগ করতে Prometheus রিস্টার্ট লাগে কেন না?
3. `relabel_configs` বনাম `metric_relabel_configs`-এর পার্থক্য কী?
4. একটি target DOWN হলে প্রথমে কোন তিনটি জিনিস পরীক্ষা করবেন?
5. Kubernetes-এ `kubernetes_sd_configs` কী সুবিধা দেয়?

ল্যাব 5 • Observability Lab Workbook

Lab 5 — Write PromQL queries

উদ্দেশ্য

PromQL — Prometheus Query Language — দিয়ে instant vector, range vector, aggregation, `rate()` / `increase()`, `histogram_quantile()` এবং `label_replace` লিখতে শিখবেন। এই ল্যাবের কুয়েরিগুলোই পরের ল্যাবে dashboard, recording rule ও alert-এর ভিত্তি হবে।

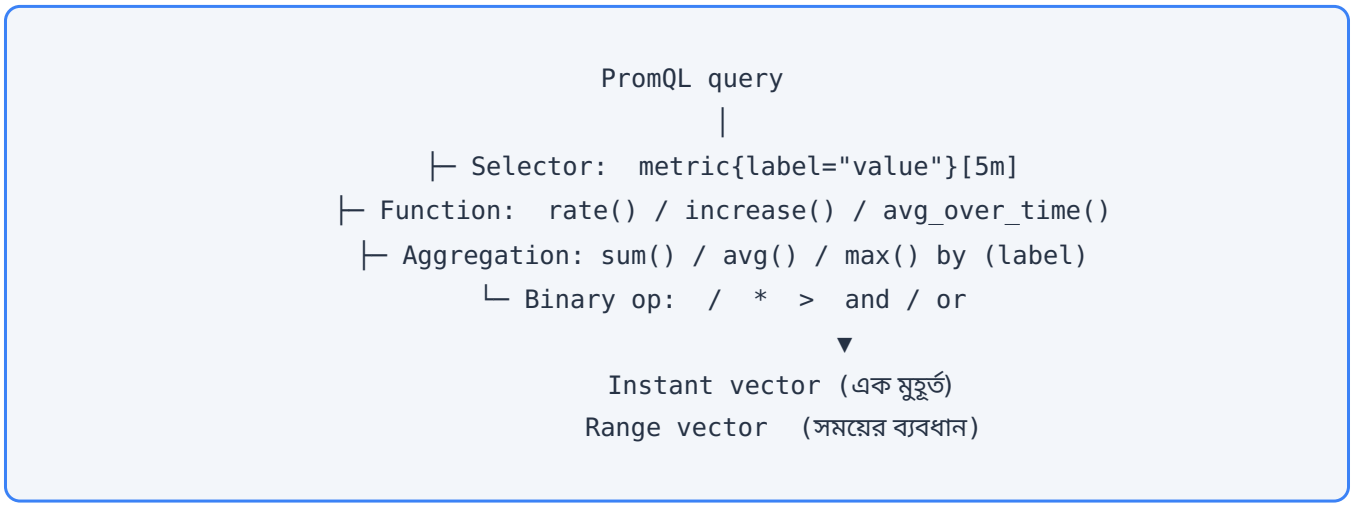
প্রয়োজনীয় টুল

- চালু Prometheus + Node Exporter (Lab 1–4)
- কিছুক্ষণ ধরে চলা metrics (কমপক্ষে 10 মিনিটের ডেটা ভালো)

পূর্বশর্ত

Counter বনাম Gauge-এর পার্থক্য জানেন (Lab 2)। Prometheus UI-র Graph/Console ট্যাব খুলতে পারেন।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — বেসিক সিলেক্টর

```
up
up{job="node-exporter"}
node_cpu_seconds_total{mode="idle"}
node_cpu_seconds_total{mode="idle"}[5m]
```

প্রথমটি instant vector, শেষটি range vector — Graph ট্যাবে শেষটি চালালে সময়ের সাথে রেখা দেখা যায়।

ধাপ ২ — Rate ও Increase

```
rate(node_cpu_seconds_total{mode="user"}[5m])
increase(node_network_receive_bytes_total[1h])
rate(http_requests_total[5m])
```

> **CONCEPT:** `rate()` প্রতি সেকেন্ডের গড় হার দেয়; `irate()` শেষ দুটি পয়েন্ট দিয়ে হিসাব করে — দ্রুত বদল দেখতে ভালো কিন্তু noisy। `increase()` পুরো ব্যবধানে মোট বৃদ্ধি দেয়। Range `[5m]` scrape interval-এর কমপক্ষে 4 গুণ হওয়া উচিত, নইলে ফলাফল অস্থির হয়।

ধাপ ৩ — Aggregation

```
sum(rate(node_cpu_seconds_total{mode!="idle"}[5m])) by (instance)
avg(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) by (instance)
topk(3, sum by (instance) (rate(node_network_transmit_bytes_total[5m])))
```

ধাপ ৪ — শতাংশ ও অনুপাত

```
# CPU ব্যবহার %
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
# মেমরি ব্যবহার %
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
# ডিস্ক ফাঁকা %
(node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"}) *
100
# HTTP error rate %
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) * 100
```

ধাপ ৫ — Histogram ও Quantile

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
```

ধাপ ৬ — ফিল্টার ও লেবেল অপারেশন

```
up == 0
node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} < 0.2
label_replace(up, "short", "$1", "instance", "(.+):.+")
```

সম্পূর্ণ কনফিগারেশন

এই ল্যাবে কোনো নতুন ফাইল নেই — নিচে একটি চিট-শিট টেবিল দেওয়া হলো, যা পরের ল্যাবগুলোতেও ব্যবহার করবেন:

প্রয়োজন	PromQL
CPU %	`100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`

মেমরি %	`(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100`
ডিস্ক ব্যবহার %	`100 - ((node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"}) * 100)`
নেটওয়ার্ক MB/s	`rate(node_network_receive_bytes_total[5m]) / 1024 / 1024`
Load per CPU	`node_load5 / count(count(node_cpu_seconds_total{mode="idle"}) by (cpu))`
Up/Down	`up == 0`
p95 latency	`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))`
Error rate %	`sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) * 100`

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- প্রতিটি কুয়েরি Console ট্যাবে সংখ্যা এবং Graph ট্যাবে রেখা দেখায়; খালি ফলাফল মানে selector-এ label ভুল।
- `up == 0` কোনো DOWN target না থাকলে খালি ফলাফল দেয় — এটাই প্রত্যাশিত।
- p95 কুয়েরি histogram metric থাকলে মান দেয়; না থাকলে Lab 7-এর পর আবার চেষ্টা করুন।
- `rate()` কুয়েরি প্রথম 5 মিনিটে অস্থির হতে পারে — ডেটা জমলে স্থির হয়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
No data / empty result	Metric নাম বা label ভুল	UI-তে metric নাম টাইপ করে autocomplete দেখুন; `{__name__=~".*cpu.*"}` দিয়ে খুঁজুন
rate() = 0 সবসময়	Range খুব ছোট বা Counter রিসেট	`[5m]` ব্যবহার করুন; scrape interval যাচাই করুন
histogram_quantile NaN	Bucket-এ `+Inf` নেই বা লেবেল মিলছে না	`by (le)` ঠিক আছে কি না দেখুন; histogram metric-এর নাম যাচাই করুন
গ্রাফে অনেক লাইন	প্রতি instance/mode আলাদা সিরিজ	`sum by (instance)` বা `avg` দিয়ে aggregate করুন

পরিষ্কারকরণ (Cleanup)

```
# এই ল্যাবে কোনো cleanup নেই — Prometheus চালু রাখুন
# কুয়েরি ইতিহাস মুছতে চাইলে ব্রাউজার history যথেষ্ট
```

পর্যালোচনা প্রশ্ন

1. `rate()` ও `irate()`-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
2. `sum by (job)` ও `sum without (instance)`-এর পার্থক্য কী?
3. `[5m]` range-কে scrape interval-এর 4 গুণ করার পরামর্শ কেন দেওয়া হয়?
4. Histogram-এর `le` label-এর অর্থ কী? `+Inf` bucket কেন দরকার?
5. একটি কুয়েরি লিখুন যা গত 1 ঘণ্টায় প্রতি মিনিটে গড় HTTP request সংখ্যা দেয়।

ল্যাব 6 • Observability Lab Workbook

Lab 6 — Configure Node Exporter

উদ্দেশ্য

Node Exporter-এর collector (যেমন `cpu`, `filesystem`, `systemd`, `textfile`) চালু/বন্ধ করা, custom textfile metric প্রকাশ করা এবং একাধিক হোস্টে Exporter স্থাপন করে কেন্দ্রীয় Prometheus দিয়ে সংগ্রহ করা শিখবেন।

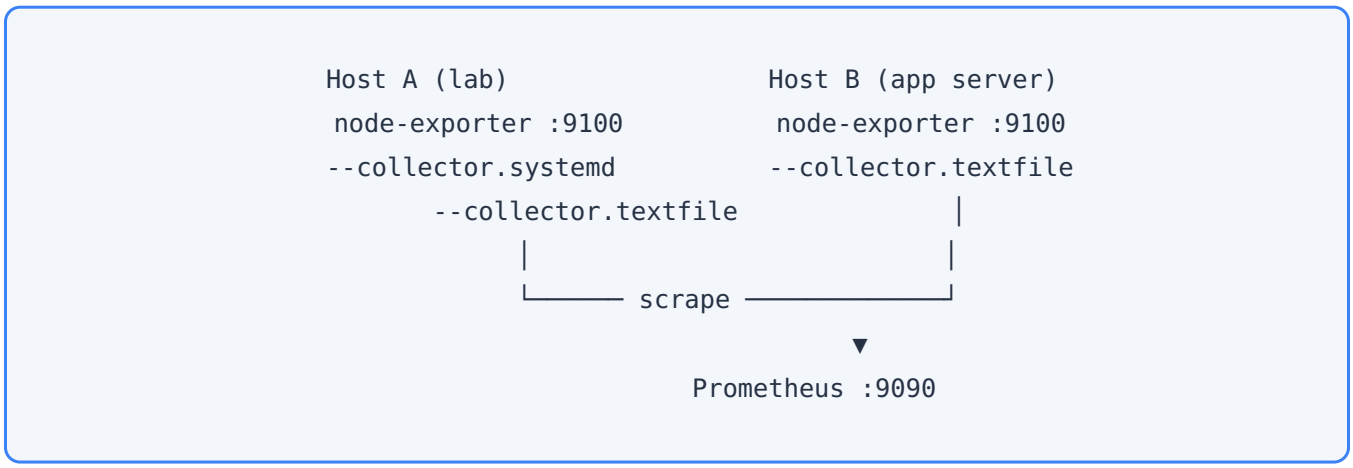
প্রয়োজনীয় টুল

- Lab 1-এর পরিবেশ, Node Exporter 1.8.x
- দ্বিতীয় হোস্ট বা দ্বিতীয় কনটেইনার (ঐচ্ছিক)

পূর্বশর্ত

Lab 2-এর metric টেবিল জানা আছে। Linux-এ systemd service সম্পর্কে প্রাথমিক ধারণা আছে।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — Collector তালিকা দেখুন

```
curl -s http://localhost:9100/metrics | grep -oE 'node_[a-z_]+' | sort -u | head -40
node_exporter --help 2>&1 | grep collector | head -40
```

ধাপ ২ — systemd collector চালু করুন

Compose-এ Node Exporter-এর command-এ যোগ করুন:

```
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
```

```
      - '--collector.systemd'
      - '--collector.systemd.unit-include=(docker|prometheus|grafana).service'
```

তারপর `docker compose up -d node-exporter` দিয়ে রিস্টার্ট করুন।

ধাপ ৩ — textfile collector দিয়ে custom metric প্রকাশ করুন

```
mkdir -p /tmp/textfile
cat > /tmp/textfile/app.prom <<'EOF'
# HELP technova_backup_last_success_timestamp_seconds Last successful backup time
# TYPE technova_backup_last_success_timestamp_seconds gauge
technova_backup_last_success_timestamp_seconds 1730995200
# HELP technova_deploy_version_info Current deployed version
# TYPE technova_deploy_version_info gauge
technova_deploy_version_info{version="1.4.2", service="api"} 1
EOF
cat /tmp/textfile/app.prom
```

Compose-এ মাউন্ট ও ফ্ল্যাগ যোগ করুন:

```
    volumes:
      - /tmp/textfile:/textfile:ro
    command:
      - '--collector.textfile.directory=/textfile'
```

ধাপ ৪ — দ্বিতীয় হোস্ট যোগ করুন

```
# দ্বিতীয় কনটেইনার হিসেবে (উদাহরণ):
docker run -d --name node-exporter-2 -p 9101:9100 \
  -v /proc:/host/proc:ro -v /sys:/host/sys:ro \
  prom/node-exporter:v1.8.2 \
  --path.procfs=/host/proc --path.sysfs=/host/sys
# prometheus.yml-এ target যোগ করুন:
#   - targets: ['host.docker.internal:9101']
```

ধাপ ৫ — যাচাই করুন

```
curl -s http://localhost:9100/metrics | grep technova_
curl -s 'http://localhost:9090/api/v1/query?query=node_systemd_unit_state' | head -c 400; echo
```

সম্পূর্ণ কনফিগারেশন

docker-compose.yml — node-exporter সার্ভিস (সম্পূর্ণ)

```
  node-exporter:
    image: prom/node-exporter:v1.8.2
    container_name: node-exporter
    ports:
      - "9100:9100"
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
```

```
      - /:/rootfs:ro
      - /tmp/textfile:/textfile:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--path.rootfs=/rootfs'
      - '--collector.systemd'
      - '--collector.textfile.directory=/textfile'
      - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($$|/)'
    restart: unless-stopped
```

Collector	কী দেয়	কখন চালু করবেন
`cpu`, `meminfo`, `diskstats`, `netdev`	মূল সম্পদ	সবসময় (ডিফল্ট)
`systemd`	সার্ভিসের অবস্থা	সার্ভিস monitoring দরকার হলে
`textfile`	Custom metric (.prom ফাইল থেকে)	ব্যাচ জব, ব্যাকআপ, ডেপ্লয় তথ্য
`logind`, `ntp`	সেশন, সময়ের বিচ্যুতি	প্রয়োজন অনুযায়ী

PRO TIP: textfile collector cron job-এর ফলাফল প্রকাশের সবচেয়ে সহজ উপায়। ব্যাকআপ স্ক্রিপ্টের শেষে `.prom` ফাইল লিখুন — Prometheus নিজে থেকেই তা তুলে নেবে। ফাইল লেখার সময় atomic write ব্যবহার করুন (অস্থায়ী ফাইলে লিখে rename করুন), নইলে Prometheus অর্ধেক লেখা ফাইল পড়তে পারে।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `node_systemd_unit_state{name="docker.service",state="active"}` মান `1` দেখায়।
- `technova_backup_last_success_timestamp_seconds` Prometheus-এ কুয়েরি করলে মান পাওয়া যায়।
- দ্বিতীয় হোস্টের target `UP` এবং `up{instance="...:9101"}` মান `1`।
- ভুল ফরম্যাটের `.prom` ফাইল থাকলে Node Exporter লগে `textfile` error দেখা যায় — ফাইল ঠিক করলে error যায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
systemd metric নেই	Docker-এ D-Bus নেই	হোস্টে সরাসরি ইনস্টল করা Node Exporter-এ systemd collector ভালো কাজ করে; কনটেইনারে সীমিত
textfile metric আসছে না	ডিরেক্টরি মাউন্ট হয়নি বা ফাইল `.prom` নয়	মাউন্ট ও ফাইলের এক্সটেনশন যাচাই করুন; ফাইলে `# HELP` / `# TYPE` লাইন থাকা ভালো
দ্বিতীয় হোস্টে সংযোগ ব্যর্থ	ফায়ারওয়াল / পোর্ট বন্ধ	`ss -tlnp | grep 9100` ও `curl` দিয়ে সরাসরি পরীক্ষা করুন

পরিষ্কারকরণ (Cleanup)

```
docker stop node-exporter-2 && docker rm node-exporter-2
docker compose restart node-exporter
```

পর্যালোচনা প্রশ্ন

1. কোন collector-গুলো ডিফল্ট চালু থাকে, কোনগুলো বন্ধ? কেন?
2. textfile collector কেন ব্যাচ জবের জন্য উপযোগী, আর Pushgateway কখন দরকার?
3. একটি সার্ভিস DOWN হলে `node_systemd_unit_state` দিয়ে কীভাবে alert লিখবেন?
4. Node Exporter-কে root হিসেবে চালানো কি নিরাপদ? বিকল্প কী?
5. দুটি হোস্টের একই metric-কে PromQL-এ `by (instance)` দিয়ে আলাদা করবেন কীভাবে?

ল্যাব 7 • Observability Lab Workbook

Lab 7 — Instrument a Node.js application

উদ্দেশ্য

TechNova MERN অ্যাপের Node.js (Express) API-তে `prom-client` দিয়ে `/metrics` endpoint যোগ করবেন। HTTP request সংখ্যা, latency histogram ও process metric স্বয়ংক্রিয়ভাবে প্রকাশ করা শিখবেন — এটাই application instrumentation-এর প্রথম ধাপ।

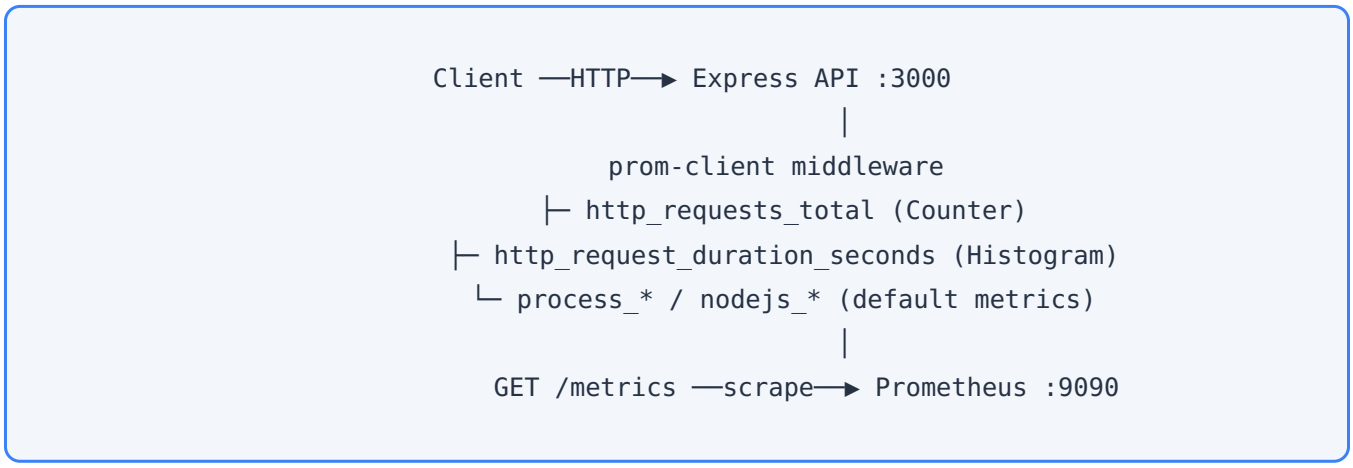
প্রয়োজনীয় টুল

- Node.js 20, npm, Express 4.x, `prom-client`
- Lab 1-এর Prometheus (scrape-এর জন্য)

পূর্বশর্ত

JavaScript ও Express-এর প্রাথমিক ধারণা। `npm init` ও `npm install` চালাতে পারেন।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — প্রজেক্ট তৈরি

```
mkdir -p ~/technova-api && cd ~/technova-api
npm init -y
npm install express prom-client
node --version
```

ধাপ ২ — API + metrics লিখুন

`server.js` — সম্পূর্ণ কোড পরের অংশে।

ধাপ ৩ — চালু করে ট্রাফিক তৈরি করুন

```
node server.js &
curl http://localhost:3000/
curl http://localhost:3000/api/products
curl http://localhost:3000/api/products/123
curl http://localhost:3000/metrics | head -40
```

ধাপ ৪ — Prometheus-এ scrape করুন

`prometheus.yml`-এ job যোগ করে reload করুন (Lab 4-এর পদ্ধতি):

```
  - job_name: 'technova-api'
    static_configs:
      - targets: ['host.docker.internal:3000']
```

Docker-এর ভেতর থেকে হোস্টের API-তে পৌঁছাতে `host.docker.internal` ব্যবহার করুন; Linux-এ প্রয়োজনে `--add-host=host.docker.internal:host-gateway` যোগ করুন।

ধাপ ৫ — কুয়েরি দিয়ে যাচাই

```
rate(http_requests_total[5m])
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
```

সম্পূর্ণ কনফিগারেশন

server.js

```
const express = require('express');
const client = require('prom-client');

const app = express();
const PORT = process.env.PORT || 3000;

// Default metrics: process CPU/memory, Node.js event loop, GC
client.collectDefaultMetrics({ prefix: 'technova_' });

// HTTP request counter
const httpRequestsTotal = new client.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'route', 'status'],
});

// HTTP latency histogram (seconds)
const httpRequestDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request duration in seconds',
  labelNames: ['method', 'route', 'status'],
  buckets: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
});

// Middleware: প্রতিটি request মাপা হয়
```

```
app.use((req, res, next) => {
  const end = httpRequestDuration.startTimer();
  res.on('finish', () => {
    const route = req.route ? req.route.path : req.path;
    httpRequestsTotal.inc({ method: req.method, route, status: res.statusCode });
    end({ method: req.method, route, status: res.statusCode });
  });
  next();
});

app.use(express.json());

const products = [
  { id: '1', name: 'TechNova Laptop Stand', price: 2500 },
  { id: '2', name: 'TechNova USB-C Hub', price: 3200 },
];

app.get('/', (req, res) => res.json({ service: 'technova-api', status: 'ok' }));
app.get('/api/products', (req, res) => res.json(products));
app.get('/api/products/:id', (req, res) => {
  const p = products.find((x) => x.id === req.params.id);
  if (!p) return res.status(404).json({ error: 'not found' });
  res.json(p);
});
app.get('/health', (req, res) => res.json({ status: 'healthy' }));

// Prometheus scrape endpoint — auth ছাড়া প্রকাশ করবেন না production-এ
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});

app.listen(PORT, () => console.log(`technova-api listening on ${PORT}`));
```

Dockerfile (ঐচ্ছিক — Compose-এ API চালাতে)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY server.js ./
EXPOSE 3000
CMD ["node", "server.js"]
```

> **WARNING:** `/metrics` endpoint-এ internal তথ্য থাকে। Production-এ এটি public নেটওয়ার্কে খুলে রাখবেন না — reverse proxy-তে IP allowlist বা basic auth দিন, অথবা আলাদা internal পোর্টে প্রকাশ করুন।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `curl localhost:3000/metrics`-এ `http_requests_total`, `http_request_duration_seconds_bucket`, `technova_process_cpu_user_seconds_total` দেখা যায়।
- কয়েকবার `/api/products` হিট করার পর Counter-এর মান বাড়ে।

- Prometheus Targets-এ `technova-api` `UP`।
- p95 কুয়েরি একটি সংখ্যা দেয় (সাধারণত 0.01 সেকেন্ডের কম, স্থানীয়ভাবে)।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
`/metrics` 404	রুটটি middleware-এর পরে বা ভুল পথে	রুটের ক্রম যাচাই করুন; `app.get('/metrics')` সব রুটের আগে/পরে যেকোনো জায়গায় কাজ করে যদি পথ ঠিক থাকে
route label `/api/products/123` আলাদা আলাদা	`req.path` ব্যবহার হয়েছে	`req.route.path` ব্যবহার করুন যাতে `/api/products/:id` একটিই সিরিজ হয় (high cardinality এড়ান)
Prometheus target DOWN	হোস্টনেম ভুল	হোস্ট থেকে `curl localhost:3000/metrics` দিয়ে API যাচাই করুন; Compose-এ `host.docker.internal` যোগ করুন
Histogram bucket খালি	কোনো request হয়নি	কয়েকটি request পাঠান, তারপর কুয়েরি করুন

পরিষ্কারকরণ (Cleanup)

```
pkill -f "node server.js" || true
# API বন্ধ করলে Prometheus-এ target DOWN দেখাবে — এটাই প্রত্যাশিত
```

পর্যালোচনা প্রশ্ন

1. Histogram কেন latency-র জন্য Counter বা Gauge-এর চেয়ে ভালো?
2. Bucket নির্বাচন কীভাবে করবেন? খুব বেশি bucket-এর ক্ষতি কী?
3. High-cardinality label (যেমন user ID) metric-এ দিলে কী সমস্যা হয়?
4. `collectDefaultMetrics` কোন কোন metric দেয়? দুটি উদাহরণ দিন।
5. `/metrics` endpoint-কে production-এ কীভাবে সুরক্ষিত করবেন?

ল্যাব 8 • Observability Lab Workbook

Lab 8 — Create custom application metrics

উদ্দেশ্য

ব্যবসায়িক (business) metric তৈরি করবেন — যেমন অর্ডার সংখ্যা, পেমেন্ট ব্যর্থতা, ক্যাশে হিট রেট, সক্রিয় ব্যবহারকারী। Counter, Gauge ও Histogram-এর মধ্যে কখন কোনটি বেছে নিতে হয় তা শিখবেন এবং RED/USE পদ্ধতিতে metric ডিজাইন করবেন।

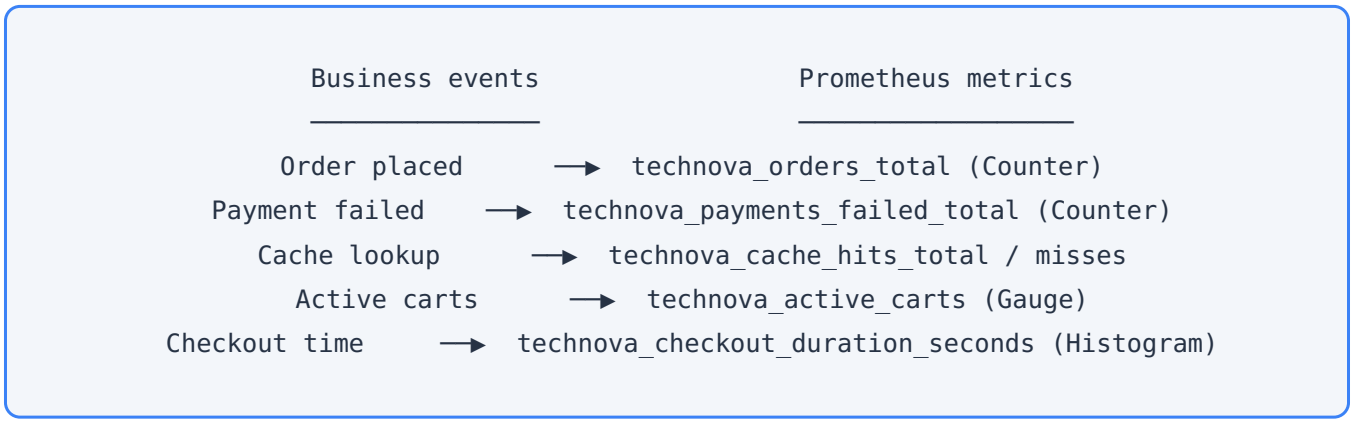
প্রয়োজনীয় টুল

- Lab 7-এর TechNova API (চালু)
- Node.js 20, `prom-client`

পূর্বশর্ত

Lab 7 সম্পূর্ণ। Counter/Gauge/Histogram/Summary-এর পার্থক্য জানেন।

আর্কিটেকচার



RED পদ্ধতি (Request, Error, Duration) — প্রতিটি সার্ভিসের জন্য এই তিনটি মাপুন। **USE পদ্ধতি** (Utilization, Saturation, Errors) — প্রতিটি সম্পদের জন্য এই তিনটি মাপুন।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — Metric ডিজাইন করুন

Metric	টাইপ	Label	কেন
`technova_orders_total`	Counter	`status`, `payment_method`	বিক্রির হার ও সাফল্য
`technova_payment_failures_total`	Counter	`reason`	পেমেন্ট সমস্যা নির্ণয়
`technova_active_carts`	Gauge	—	বর্তমান চাপ

`technova_checkout_duration_seconds`	Histogram	`route`	p95 latency
`technova_cache_requests_total`	Counter	`result` (hit/miss)	ক্যাশে কার্যকারিতা

ধাপ ২ — কোডে যোগ করুন

`server.js`-এ নিচের কোড যোগ করুন (সম্পূর্ণ ফাইল পরের অংশে)।

ধাপ ৩ — ট্রাফিক তৈরি করুন

```
curl -X POST http://localhost:3000/api/orders -H 'Content-Type: application/json' -d
'{"productId":"1","qty":2}'
curl -X POST http://localhost:3000/api/orders -H 'Content-Type: application/json' -d
'{"productId":"2","qty":1,"failPayment":true}'
curl http://localhost:3000/metrics | grep technova_
```

ধাপ ৪ — PromQL দিয়ে ব্যবসায়িক প্রশ্নের উত্তর দিন

```
sum(rate(technova_orders_total[5m])) * 60
sum(rate(technova_payment_failures_total[5m])) / sum(rate(technova_orders_total[5m])) * 100
sum(rate(technova_cache_requests_total{result="hit"}[5m])) /
sum(rate(technova_cache_requests_total[5m])) * 100
```

সম্পূর্ণ কনফিগারেশন

server.js — custom metrics অংশ (Lab 7-এর ফাইলে যোগ করুন)

```
const ordersTotal = new client.Counter({
  name: 'technova_orders_total',
  help: 'Total orders placed',
  labelNames: ['status', 'payment_method'],
});
const paymentFailuresTotal = new client.Counter({
  name: 'technova_payment_failures_total',
  help: 'Total failed payments',
  labelNames: ['reason'],
});
const activeCarts = new client.Gauge({
  name: 'technova_active_carts',
  help: 'Currently active shopping carts',
});
const checkoutDuration = new client.Histogram({
  name: 'technova_checkout_duration_seconds',
  help: 'Checkout duration in seconds',
  buckets: [0.1, 0.25, 0.5, 1, 2, 5],
});
const cacheRequestsTotal = new client.Counter({
  name: 'technova_cache_requests_total',
  help: 'Cache lookups by result',
  labelNames: ['result'],
});

// উদাহরণ endpoint: অর্ডার তৈরি
```

```
let carts = 0;
app.post('/api/orders', (req, res) => {
  const end = checkoutDuration.startTimer();
  carts += 1;
  activeCarts.set(carts);
  // নমুনা: failPayment=true পাঠালে পেমেন্ট ব্যর্থ ধরা হয়
  if (req.body && req.body.failPayment) {
    paymentFailuresTotal.inc({ reason: 'gateway_timeout' });
    ordersTotal.inc({ status: 'failed', payment_method: 'sslcommerz' });
    end();
    carts -= 1;
    activeCarts.set(carts);
    return res.status(402).json({ error: 'payment failed' });
  }
  // নমুনা ক্যাশে: জোড় সংখ্যার productId হিট ধরা হয়
  cacheRequestsTotal.inc({ result: Number(req.body.productId) % 2 === 0 ? 'hit' : 'miss' });
  ordersTotal.inc({ status: 'success', payment_method: 'sslcommerz' });
  end();
  carts -= 1;
  activeCarts.set(carts);
  res.status(201).json({ orderId: `ord-${Date.now()}`, status: 'success' });
});
```

CONCEPT: Gauge-এর মান `set()`, `inc()`, `dec()` দিয়ে বদলানো যায় — বর্তমান অবস্থা বোঝাতে ব্যবহার করুন (সক্রিয় সংযোগ, queue দৈর্ঘ্য)। Counter শুধু বাড়ে — ঘটনা গুনতে ব্যবহার করুন। ভুল টাইপ বেছে নিলে `rate()` ভুল ফল দেবে।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- অর্ডার পোস্ট করার পর `technova_orders_total{status="success"}` বাড়ে।
- `failPayment` পাঠালে `technova_payment_failures_total{reason="gateway_timeout"}` বাড়ে এবং HTTP 402 ফেরত আসে।
- ক্যাশে হিট রেট কুয়েরি 0–100-এর মধ্যে মান দেয়।
- Checkout p95: `histogram_quantile(0.95, sum(rate(technova_checkout_duration_seconds_bucket[5m])) by (le))` মান দেয়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Custom metric `/metrics`-এ নেই	Metric তৈরি হয়েছে কিন্তু কখনো inc হয়নি	Counter প্রথম inc না হওয়া পর্যন্ত প্রকাশ পায় না — একটি request পাঠান
Gauge ঋণাত্মক হচ্ছে	`dec()` বেশি বার ডাকা হয়েছে	কাউন্টার ভেরিয়েবলের সাথে Gauge সবসময় sync রাখুন; সম্ভব হলে `set()` ব্যবহার করুন
Label-এর মান অনেক (cardinality বিস্ফোরণ)	orderId/userId label-এ দেওয়া হয়েছে	High-cardinality মান কখনো label-এ দেবেন না — log/trace-এ রাখুন

পরিষ্কারকরণ (Cleanup)

```
pkill -f "node server.js" || true
# পরের ল্যাবে আবার চালু করবেন: node server.js &
```

পর্যালোচনা প্রশ্ন

1. অর্ডার সংখ্যার জন্য Counter কেন, Gauge কেন নয়?
2. RED বনাম USE — প্রতিটির একটি উদাহরণ দিন।
3. একটি metric-এ সর্বোচ্চ কয়টি label রাখা উচিত? কেন?
4. Summary বনাম Histogram — কখন কোনটি বেছে নেবেন?
5. পেমেন্ট ব্যর্থতার হার 5%-এর বেশি হলে কোন alert লিখবেন? (Lab 10-এর প্রস্তুতি)

ল্যাব 9 • Observability Lab Workbook

Lab 9 — Configure recording rules

উদ্দেশ্য

ঘন ঘন ব্যবহৃত জটিল PromQL কুয়েরিকে recording rule হিসেবে আগে থেকে হিসাব করে নতুন metric হিসেবে সংরক্ষণ করতে শিখবেন। এতে dashboard দ্রুত লোড হয় এবং alert কুয়েরি সহজ হয়।

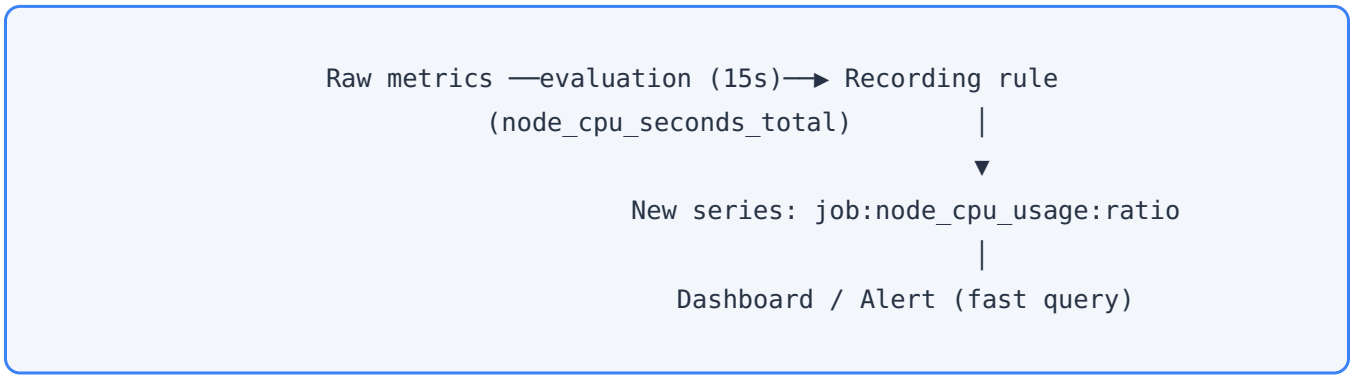
প্রয়োজনীয় টুল

- Prometheus 3.x, `promtool`
- Lab 5-এর কুয়েরিগুলো

পূর্বশর্ত

PromQL aggregation ও `rate()` জানেন। Prometheus কনফিগারেশনে `rule_files` কীভাবে যোগ করতে হয় জানেন (Lab 3)।

আর্কিটেকচার



নামকরণের প্রচলিত নিয়ম: `<level>:<metric>:<operation>` — যেমন `job:node_cpu_usage:ratio`, `instance:http_requests:rate5m`।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — Rules ফাইল তৈরি করুন

`prometheus/rules/recording.yml` তৈরি করুন (সম্পূর্ণ ফাইল নিচে)।

ধাপ ২ — যাচাই করুন

```
promtool check rules prometheus/rules/recording.yml
# Docker-এ:
docker compose exec prometheus promtool check rules /etc/prometheus/rules/recording.yml
```

ধাপ ৩ — Prometheus-এ যোগ করুন

`prometheus.yml`-এ:

```
rule_files:
  - 'rules/*.yml'
```

Compose-এ rules ডিরেক্টরি মাউন্ট করুন এবং reload করুন:

```
docker compose up -d prometheus
curl -X POST http://localhost:9090/-/reload
```

ধাপ ৪ — নতুন metric যাচাই করুন

```
curl -s 'http://localhost:9090/api/v1/query?query=job:node_cpu_usage:ratio' | python3 -m
json.tool | head -30
```

ধাপ ৫ — Dashboard কুয়েরি সহজ করুন

আগে: `100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`

এখন: `job:node_cpu_usage:ratio * 100`

সম্পূর্ণ কনফিগারেশন

prometheus/rules/recording.yml

```
groups:
  - name: node-recording-rules
    interval: 30s
    rules:
      - record: job:node_cpu_usage:ratio
        expr: |
          1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) by (job)
      - record: job:node_memory_usage:ratio
        expr: |
          1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
      - record: job:node_disk_usage:ratio
        expr: |
          1 - (node_filesystem_avail_bytes{mountpoint="/"} /
node_filesystem_size_bytes{mountpoint="/"})
      - record: instance:node_network_receive:rate5m
        expr: |
          rate(node_network_receive_bytes_total[5m])

  - name: http-recording-rules
    interval: 30s
    rules:
      - record: job:http_requests:rate5m
        expr: |
          sum(rate(http_requests_total[5m])) by (job)
      - record: job:http_errors:ratio5m
        expr: |
          sum(rate(http_requests_total{status=~"5.."}[5m])) by (job)
          /
```

```
          sum(rate(http_requests_total[5m])) by (job)
      - record: job:http_latency_p95:5m
        expr: |
          histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by
(job, le))
```

PRO TIP: Recording rule-এর `interval` কুয়েরির range-এর চেয়ে ছোট রাখুন। `[5m]` range-এর জন্য 30s interval ভালো। খুব ঘন ঘন evaluation (যেমন 5s) CPU খরচ বাড়ায়; খুব বিরল (5m) হলে dashboard-এ ডেটা দেরিতে আসে।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Prometheus UI → Status → Rules — দুটি গ্রুপ, সব rule `OK` অবস্থায়, Last Evaluation সময় সাম্প্রতিক।
- `job:node_cpu_usage:ratio` কুয়েরি 0–1-এর মধ্যে মান দেয়।
- `job:http_errors:ratio5m` কোনো error না থাকলে 0 দেয়।
- ভুল expr থাকলে Rules পেজে error দেখা যায় এবং `promtool check rules` ব্যর্থ হয়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Rule দেখা যাচ্ছে না	`rule_files` পথ ভুল বা reload হয়নি	মাউন্ট যাচাই করুন; `/-/reload` বা রিস্টার্ট করুন
Rule evaluation error	Expr-এ label মিলছে না	Expr-টি Console-এ সরাসরি চালিয়ে পরীক্ষা করুন
নতুন metric-এ গ্যাপ	Interval খুব বড় বা Prometheus রিস্টার্ট হয়েছে	Interval 30s-এ নামান; গ্যাপ স্বাভাবিক যদি evaluation মিস হয়
নামে `:` ব্যবহারে বিভ্রান্তি	নামকরণ নিয়ম না জানা	`level:metric:operation` নিয়ম মেনে চলুন — `:` Prometheus-এ বৈধ

পরিষ্কারকরণ (Cleanup)

```
# Rules ফাইল রেখে দিন — Lab 10-এ alert rule পাশে যোগ হবে
# মুছতে চাইলে: rm prometheus/rules/recording.yml && curl -X POST http://localhost:9090/-/reload
```

পর্যালোচনা প্রশ্ন

1. Recording rule কখন ব্যবহার করবেন, আর কখন সরাসরি কুয়েরি যথেষ্ট?
2. Recording rule কি ডিস্ক ব্যবহার বাড়ায়? কীভাবে?
3. `job:http_errors:ratio5m`-এ `by (job)` দুই পাশে কেন দরকার?
4. একটি recording rule লিখুন যা প্রতি instance-এর ডিস্ক ব্যবহারের হার দেয়।

5. Alert কুয়েরিতে recording rule ব্যবহারের সুবিধা কী?

ল্যাব 10 • Observability Lab Workbook

Lab 10 — Create alert rules

উদ্দেশ্য

Prometheus alerting rule লিখবেন — `for` সময়সীমা, severity label, annotation (summary/description/runbook) সহ। Alert-এর জীবনচক্র (inactive → pending → firing) বুঝবেন এবং ভালো alert লেখার নীতি (actionable, সঠিক threshold) শিখবেন।

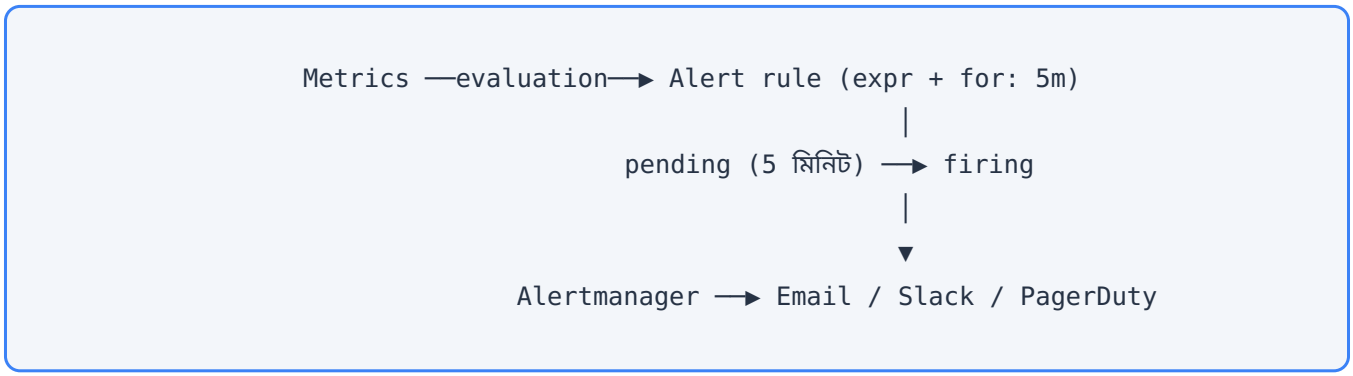
প্রয়োজনীয় টুল

- Prometheus 3.x, Lab 9-এর recording rules
- Alertmanager (পরের ল্যাবে, তবে rule যাচাই এই ল্যাবেই)

পূর্বশর্ত

PromQL ও recording rule জানেন। Lab 5-এর CPU/মেমরি/ডিস্ক কুয়েরি মনে আছে।

আর্কিটেকচার



ভালো alert-এর নীতি: প্রতিটি alert-এর জন্য কেউ কিছু করতে পারে (actionable), threshold এমন যে false positive কম, এবং annotation-এ কী করতে হবে লেখা থাকে।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — Alerts ফাইল তৈরি করুন

`prometheus/rules/alerts.yml` তৈরি করুন (সম্পূর্ণ ফাইল নিচে)।

ধাপ ২ — যাচাই ও লোড করুন

```
promtool check rules prometheus/rules/alerts.yml
docker compose up -d prometheus
curl -X POST http://localhost:9090/-/reload
```

ধাপ ৩ — Alert ট্রিগার করে দেখুন

```
# CPU alert পরীক্ষা করতে (ঐচ্ছিক):
stress-ng --cpu 2 --timeout 300s &
# Prometheus UI → Alerts পেজে HighCPUUsage pending → firing হতে দেখুন
# তারপর: pkill -f stress-ng
```

ধাপ ৪ — API বন্ধ করে Down alert দেখুন

```
pkill -f "node server.js" || true
# 1 মিনিট পর Alerts পেজে InstanceDown firing দেখা যাবে
node ~/technova-api/server.js &
```

সম্পূর্ণ কনফিগারেশন

prometheus/rules/alerts.yml

```
groups:
  - name: technova-host-alerts
    rules:
      - alert: InstanceDown
        expr: up == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Instance {{ $labels.instance }} down"
          description: "Job {{ $labels.job }} / instance {{ $labels.instance }} 1 মিনিট ধরে
scrape করা যাচ্ছে না।"
          runbook_url: "https://technova.example/runbooks/instance-down"

      - alert: HighCPUUsage
        expr: job:node_cpu_usage:ratio > 0.85
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High CPU on {{ $labels.job }} ({{ $value | printf \"%.1f\" }}%)"
          description: "CPU ব্যবহার 85%-এর বেশি, 5 মিনিট ধরে। মান: {{ $value }}"

      - alert: HighMemoryUsage
        expr: job:node_memory_usage:ratio > 0.9
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High memory on {{ $labels.job }}"
          description: "মেমরি ব্যবহার 90%-এর বেশি, 5 মিনিট ধরে।"

      - alert: DiskSpaceLow
        expr: job:node_disk_usage:ratio > 0.85
        for: 5m
        labels:
          severity: warning
```

```
        annotations:
          summary: "Disk space low on {{ $labels.job }}"
          description: "ডিস্ক 85%-এর বেশি পূর্ণ। পুরনো লগ/ডেটা পরিষ্কার করুন।"

      - alert: DiskSpaceCritical
        expr: job:node_disk_usage:ratio > 0.95
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "Disk critically full on {{ $labels.job }}"
          description: "ডিস্ক 95%-এর বেশি পূর্ণ — এখনই ব্যবস্থা নিন।"

  - name: technova-api-alerts
    rules:
      - alert: HighErrorRate
        expr: job:http_errors:ratio5m > 0.05
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "High HTTP error rate on {{ $labels.job }}"
          description: "5xx error হার 5%-এর বেশি, 5 মিনিট ধরে। মান: {{ $value }}"

      - alert: HighLatency
        expr: job:http_latency_p95:5m > 1
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High p95 latency on {{ $labels.job }}"
          description: "p95 latency 1 সেকেন্ডের বেশি, 5 মিনিট ধরে।"

      - alert: PaymentFailureRateHigh
        expr: |
          sum(rate(technova_payment_failures_total[5m])) by (job)
          /
          sum(rate(technova_orders_total[5m])) by (job) > 0.05
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Payment failure rate high"
          description: "পেমেন্ট ব্যর্থতার হার 5%-এর বেশি — SSLCommerz গেটওয়ে ও লগ পরীক্ষা করুন।"
```

CONCEPT: `for: 5m` মানে শর্তটি 5 মিনিট ধরে সত্য থাকলে তবেই firing হবে। এতে ক্ষণিকের স্পাইকে alert যায় না। কিন্তু `for` খুব বড় হলে (যেমন 30m) আসল সমস্যায় দেরি হয়। Critical alert-এ `for` ছোট (1–2m), warning-এ বড় (5–10m) রাখা সাধারণ অভ্যাস।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Prometheus UI → Alerts পেজে সব alert `inactive` (সবুজ) অবস্থায় তালিকাভুক্ত।
- API বন্ধ করলে `InstanceDown` 1 মিনিটে `pending` (হলুদ), তারপর `firing` (লাল) হয়।

- `curl http://localhost:9090/api/v1/alerts`-এ alert-গুলোর state দেখা যায়।
- API আবার চালু করলে alert `resolved` হয়ে তালিকা থেকে যায় (Alertmanager-এ গেলে)।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Alert কখনো firing হয় না	Expr ভুল বা threshold খুব বেশি	Expr-টি Console-এ চালিয়ে মান দেখুন; পরীক্ষার জন্য threshold সাময়িক কমান
Alert বারবার firing/ resolved হচ্ছে (flapping)	Threshold সীমার কাছে ওঠানামা করছে	`for` বাড়ান; অথবা recording rule-এর গড় ব্যবহার করুন
`$value` annotation-এ দেখা যাচ্ছে না	টেমপ্লেট সিনট্যাক্স ভুল	`{{ $value }}` ও `{{ $labels.job }}` ঠিকভাবে লিখুন; `printf` ব্যবহার করুন
Alertmanager-এ alert যাচ্ছে না	`alerting.alertmanagers` কনফিগার করা হয়নি	Lab 11-এ Alertmanager যোগ করুন; Status → Alertmanagers পেজে দেখুন

পরিষ্কারকরণ (Cleanup)

```
pkill -f stress-ng || true
# Alerts ফাইল রেখে দিন — Lab 11-এ এগুলোই Alertmanager-এ যাবে
```

পর্যালোচনা প্রশ্ন

1. `for` সময়সীমার কাজ কী? এটি না দিলে কী হয়?
2. Severity label-এ `critical` বনাম `warning` কখন ব্যবহার করবেন?
3. Alert fatigue কী? তা কমানোর তিনটি উপায় লিখুন।
4. `up == 0` alert-এ `for: 1m` কেন, 0m কেন নয়?
5. একটি alert লিখুন যা ডিস্ক 80% পূর্ণ হলে warning দেয়, তবে শুধু রাত ২টার পর নয় — সবসময়।

ল্যাব 11 • Observability Lab Workbook

Lab 11 — Configure Alertmanager

উদ্দেশ্য

Alertmanager দিয়ে alert-কে group করা, নীরব করা (silence), রুট করা (routing) এবং ইমেইল/Slack-এ পাঠানো শিখবেন। Inhibition (যেমন InstanceDown থাকলে HighCPU চাপা দেওয়া) ও notification-এর পুনরাবৃত্তি নিয়ন্ত্রণ করবেন।

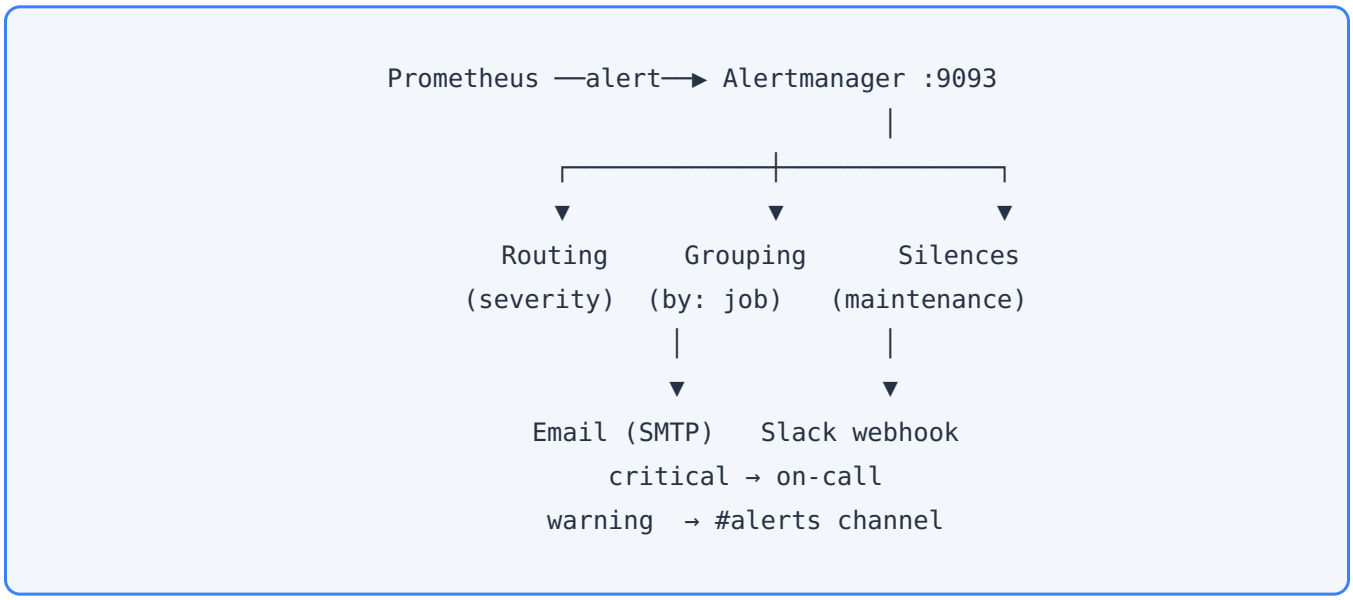
প্রয়োজনীয় টুল

- Alertmanager 0.27.x, Prometheus 3.x
- পরীক্ষার জন্য একটি Gmail/SMTP অ্যাকাউন্ট বা Slack webhook (ঐচ্ছিক — না থাকলে webhook receiver দিয়ে পরীক্ষা করুন)

পূর্বশর্ত

Lab 10-এর alert rules চালু আছে। SMTP বা Slack webhook-এর ধারণা আছে। আসল secret কখনো উদাহরণে লিখবেন না।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — Alertmanager কনফিগারেশন লিখুন

`alertmanager/alertmanager.yml` তৈরি করুন (সম্পূর্ণ ফাইল নিচে)। প্রথমে SMTP ছাড়া, webhook receiver দিয়ে শুরু করুন যাতে secret-এর প্রয়োজন না পড়ে।

ধাপ ২ — Compose-এ যোগ করুন

```
  alertmanager:
    image: prom/alertmanager:v0.27.0
    container_name: alertmanager
    ports:
      - "9093:9093"
    volumes:
      - ./alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro
      - alertmanager-data:/alertmanager
    command:
      - '--config.file=/etc/alertmanager/alertmanager.yml'
      - '--storage.path=/alertmanager'
    restart: unless-stopped
```

ধাপ ৩ — Prometheus-কে Alertmanager-এর সাথে যুক্ত করুন

`prometheus.yml`-এ ইতিমধ্যে আছে (Lab 3):

```
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']
```

Compose-এ Prometheus রিস্টার্ট করে `curl -X POST http://localhost:9090/-/reload` করুন।

ধাপ ৪ — যাচাই করুন

```
curl -s http://localhost:9093/-/healthy
curl -s http://localhost:9093/api/v2/status | python3 -m json.tool | head -40
# Prometheus UI → Status → Alertmanagers — alertmanager UP দেখা যাবে
```

ধাপ ৫ — Silence তৈরি করুন

Alertmanager UI (`http://localhost:9093`) → New Silence → matcher `alertname="HighCPUUsage"` → 1 ঘণ্টা। Maintenance-এর সময় এভাবে alert থামানো হয়।

সম্পূর্ণ কনফিগারেশন

alertmanager/alertmanager.yml

```
global:
  resolve_timeout: 5m
  # SMTP — শুধু তখনই পূরণ করুন যখন আসল SMTP আছে; secret এখানে লিখবেন না
  # smtp_smarthost: 'smtp.example.com:587'
  # smtp_from: 'alerts@technova.example'
  # smtp_auth_username: 'CHANGE-ME-smtp-user'
  # smtp_auth_password: 'CHANGE-ME-smtp-password'

route:
  receiver: 'team-platform'
  group_by: ['alertname', 'job']
  group_wait: 30s
```

```
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers:
        - severity = "critical"
      receiver: 'oncall-critical'
      continue: true
    - matchers:
        - severity = "warning"
      receiver: 'team-platform'

receivers:
  - name: 'team-platform'
    webhook_configs:
      - url: 'http://webhook-receiver:5001/alerts'
        send_resolved: true
  - name: 'oncall-critical'
    webhook_configs:
      - url: 'http://webhook-receiver:5001/critical'
        send_resolved: true
    # Email receiver — SMTP সেট থাকলে আনকমেন্ট করুন:
    # email_configs:
    #   - to: 'oncall@technova.example'
    #     send_resolved: true

inhibit_rules:
  - source_matchers:
      - severity = "critical"
    target_matchers:
      - severity = "warning"
    equal: ['job', 'instance']
```

CONCEPT: `group_wait` — প্রথম alert আসার পর 30s অপেক্ষা করে একসাথে পাঠায়। `group_interval` — একই গ্রুপে নতুন alert এলে 5 মিনিট পর পাঠায়। `repeat_interval` — সমস্যা চলতে থাকলে 4 ঘণ্টা পর আবার জানায়। Inhibition মানে: সার্ভার DOWN (critical) থাকলে তার CPU warning চাপা পড়ে — একই ঘটনায় দুটি alert নয়।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Alertmanager UI (`:9093`) খোলে; Status-এ config `OK`।
- Prometheus UI → Status → Alertmanagers — `alertmanager:9093` `UP`।
- API বন্ধ করে `InstanceDown` firing করালে Alertmanager UI-তে alert দেখা যায়।
- Silence তৈরি করলে alert `silenced` হিসেবে চিহ্নিত হয়।
- Webhook receiver থাকলে সেখানে JSON notification পৌঁছায় (নমুনা আউটপুট — চিত্রণমূলক)।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান

Alertmanager config error	YAML ভুল বা receiver নাম মিলছে না	`amtool check-config alertmanager/alertmanager.yml` চালান
Prometheus Alertmanager-কে দেখতে পাচ্ছে না	সার্ভিস নাম ভুল	Compose নেটওয়ার্কে নাম `alertmanager`; `docker compose exec prometheus wget -qO- http://alertmanager:9093/-/healthy`
Notification আসছে না	Receiver URL ভুল বা group_wait চলছে	30s অপেক্ষা করুন; webhook receiver-এর লগ দেখুন
একই alert বারবার আসছে	`repeat_interval` ছোট	4h-এ বাড়ান; resolved notification চালু আছে কি না দেখুন

পরিষ্কারকরণ (Cleanup)

```
docker compose stop alertmanager
# Silence মুছতে UI থেকে Delete করুন; ডেটা মুছতে:
docker compose down -v   # সতর্কতা: সব ডেটা মুছে যাবে
```

> **WARNING:** `docker compose down -v` Alertmanager-এর silence ও notification ইতিহাস মুছে দেয়। পরের ল্যাবে ডেটা দরকার হলে শুধু `stop` ব্যবহার করুন।

পর্যালোচনা প্রশ্ন

1. Grouping কেন দরকার? 50টি alert একসাথে এলে কী হয়?
2. Inhibition-এর উদাহরণ দিন — কখন একটি alert আরেকটিকে চাপা দেওয়া উচিত?
3. Silence বনাম inhibition — পার্থক্য কী?
4. `repeat_interval` খুব ছোট হলে কী সমস্যা হয়?
5. Critical alert-কে email ও Slack দুটোতেই পাঠাতে routing কীভাবে লিখবেন?

ল্যাব 12 • Observability Lab Workbook

Lab 12 — Connect Grafana to Prometheus

উদ্দেশ্য

Grafana-তে Prometheus-কে data source হিসেবে যুক্ত করবেন — UI দিয়ে এবং provisioning YAML দিয়ে (infrastructure as code)। Explore ভিউ দিয়ে কুয়েরি পরীক্ষা করবেন এবং data source-এর স্বাস্থ্য যাচাই করবেন।

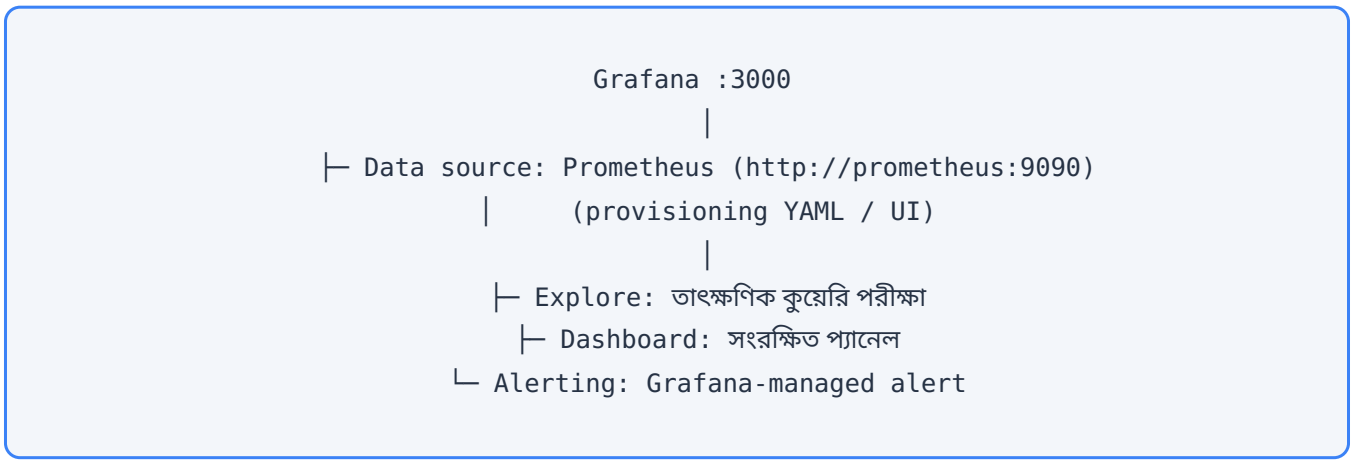
প্রয়োজনীয় টুল

- Grafana 11.x, Prometheus 3.x (Lab 1-এর পরিবেশ)
- ব্রাউজার

পূর্বশর্ত

Prometheus চলছে এবং `http://localhost:9090` থেকে কুয়েরি করা যায়। Grafana-তে admin লগইন জানেন।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — UI দিয়ে data source যোগ করুন

Grafana → Connections → Data sources → Add data source → Prometheus → URL: `http://prometheus:9090` → Save & test। সবুজ `Successfully queried the Prometheus API` বার্তা আসা উচিত।

TROUBLESHOOTING: Grafana কনটেইনারের ভেতর থেকে `localhost:9090` মানে Grafana নিজে — Prometheus নয়। Compose নেটওয়ার্কে সার্ভিস নাম `prometheus` ব্যবহার করুন। হোস্টে সরাসরি Grafana চালালে `localhost:9090` ঠিক আছে।

ধাপ ২ — Explore-এ কুয়েরি চালান

Grafana → Explore → Data source: Prometheus → কুয়েরি: `job:node_cpu_usage:ratio * 100` → Run query। গ্রাফ দেখা যাবে। Time range বদলে (Last 1 hour / Last 6 hours) দেখুন।

ধাপ ৩ — Provisioning YAML দিয়ে একই কাজ করুন

`grafana/provisioning/datasources/prometheus.yml` (Lab 1-এ তৈরি) — Grafana রিস্টার্ট করলে এটি স্বয়ংক্রিয়ভাবে লোড হয়:

```
docker compose restart grafana
docker compose logs grafana | grep -i datasource
```

ধাপ ৪ — একাধিক data source যোগ করুন

একই ফাইলে Loki ও Tempo যোগ করার প্রস্তুতি (Lab 16 ও 21-এ কার্যকর হবে):

```
  - name: Loki
    type: loki
    access: proxy
    url: http://loki:3100
    isDefault: false
  - name: Tempo
    type: tempo
    access: proxy
    url: http://tempo:3200
```

সম্পূর্ণ কনফিগারেশন

grafana/provisioning/datasources/datasources.yml

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: true
    jsonData:
      timeInterval: 15s
      httpMethod: POST
      manageAlerts: true
  - name: Loki
    type: loki
    access: proxy
    url: http://loki:3100
    editable: true
    jsonData:
      maxLines: 1000
  - name: Tempo
    type: tempo
    access: proxy
    url: http://tempo:3200
```

```
    editable: true
    jsonData:
      tracesToLogsV2:
        datasourceUid: 'loki-uid'
        spanStartTimeShift: '1h'
        spanEndTimeShift: '1h'
        filterByTraceID: true
  - name: Jaeger
    type: jaeger
    access: proxy
    url: http://jaeger:16686
    editable: true
```

> **PRO TIP:** Provisioning ফাইল Git-এ রাখুন — Grafana সেটআপ তখন কোড হিসেবে সংরক্ষিত থাকে, নতুন পরিবেশে একই ফাইল দিলেই একই data source তৈরি হয়। UI দিয়ে করা পরিবর্তন provisioning ফাইলে ফিরে আসে না।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Save & test-এ সবুজ সফল বার্তা।
- Explore-এ `up` কুয়েরি সব target-এর মান `1` দেখায়।
- Grafana রিস্টার্টের পরও data source থাকে (provisioning-এর কারণে)।
- Data source-এর UID স্থির থাকে — dashboard JSON-এ সেই UID ব্যবহার করা যায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Data source test failed	URL ভুল / Prometheus বন্ধ	`docker compose exec grafana wget -qO- http://prometheus:9090/-/healthy` দিয়ে নেটওয়ার্ক যাচাই করুন
Explore-এ No data	Time range-এ ডেটা নেই	Time range বাড়ান (Last 6 hours); কুয়েরি Prometheus UI-তে চালিয়ে দেখুন
Provisioning ফাইল লোড হচ্ছে না	মাউন্ট পথ ভুল	Compose-এ `./grafana/provisioning:/etc/grafana/provisioning:ro` যাচাই করুন; লগে `provisioning` খুঁজুন

পরিষ্কারকরণ (Cleanup)

```
# Data source মুছতে: UI → Data sources → Delete
# Provisioning ফাইল থাকলে রিস্টার্টে আবার তৈরি হবে — ফাইলও মুছতে হবে
```

পর্যালোচনা প্রশ্ন

1. Grafana-তে `proxy` বনাম `direct` access-এর পার্থক্য কী?

2. Provisioning ব্যবহারের সুবিধা কী? কখন UI ব্যবহার করবেন?
3. একটি Grafana-তে একাধিক Prometheus data source কখন দরকার হয়?
4. `timeInterval: 15s` কী নির্ধারণ করে?
5. Data source-এর UID কেন dashboard JSON-এ গুরুত্বপূর্ণ?

ল্যাব 13 • Observability Lab Workbook

Lab 13 — Build production dashboards

উদ্দেশ্য

Production-মানের Grafana dashboard তৈরি করবেন — হোস্ট overview, API RED (Rate, Error, Duration) ও ব্যবসায়িক (business) প্যানেল সহ। Variable (template variable), প্যানেলের ধরন (time series, stat, gauge, table) ও dashboard-এর JSON export শিখবেন।

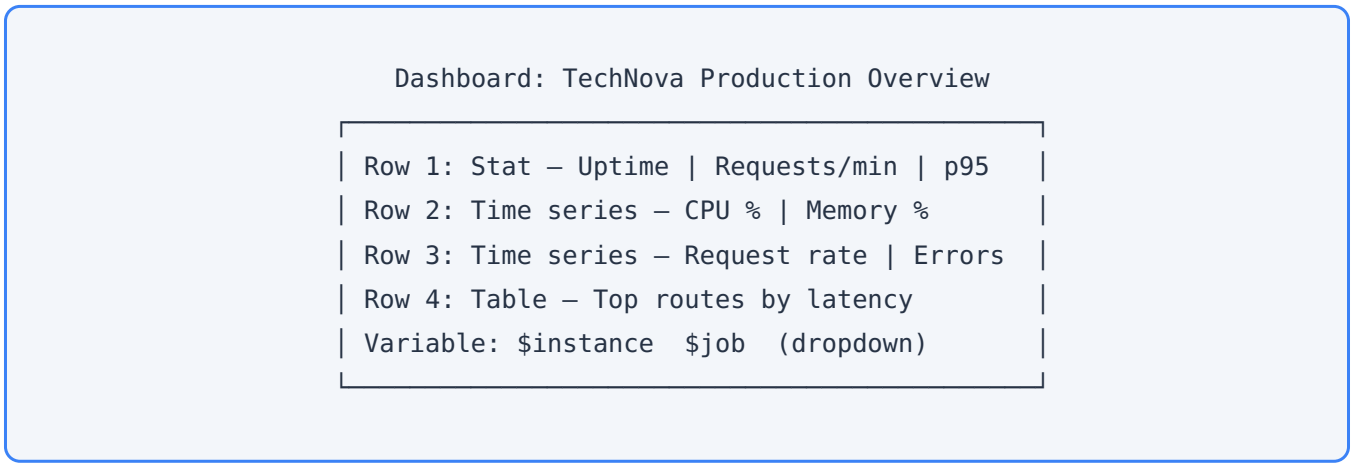
প্রয়োজনীয় টুল

- Grafana 11.x + Prometheus (Lab 12)
- TechNova API চলছে (Lab 7–8)

পূর্বশর্ত

PromQL জানেন (Lab 5)। Recording rule-গুলো চালু আছে (Lab 9)।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — নতুন dashboard তৈরি করুন

Grafana → Dashboards → New → New dashboard → Add visualization → Data source: Prometheus।

ধাপ ২ — Stat প্যানেল: বর্তমান CPU

কুয়েরি: `job:node_cpu_usage:ratio * 100` → Visualization: Stat → Unit: Percent (0-100) → Thresholds: 70 (yellow), 85 (red)।

ধাপ ৩ — Time series: Request rate ও Error rate

```
sum(rate(http_requests_total[5m])) by (route)
sum(rate(http_requests_total{status=~"5.."}[5m])) by (route)
```

ধাপ ৪ — Variable যোগ করুন

Dashboard settings → Variables → New → Name: `instance`, Type: Query, Query: `label_values(up, instance)`। তারপর প্যানেলের কুয়েরিতে `{instance="$instance"}` ব্যবহার করুন।

ধাপ ৫ — Table প্যানেল: Top routes

```
topk(5, sum by (route) (rate(http_requests_total[5m])))
# p95 latency table:
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
```

ধাপ ৬ — Dashboard সংরক্ষণ ও export করুন

Save dashboard → নাম: `TechNova Production Overview`। Dashboard settings → JSON Model → Copy করে `grafana/dashboards/technova-overview.json`-এ সংরক্ষণ করুন (নিচে sketch দেওয়া আছে)।

সম্পূর্ণ কনফিগারেশন

Dashboard JSON sketch (মূল অংশ — সম্পূর্ণ JSON Grafana থেকে export করুন)

```
{
  "title": "TechNova Production Overview",
  "uid": "technova-overview",
  "tags": ["technova", "production"],
  "timezone": "browser",
  "templating": {
    "list": [
      {
        "name": "instance",
        "type": "query",
        "query": "label_values(up, instance)",
        "multi": true,
        "includeAll": true
      }
    ]
  },
  "panels": [
    {
      "title": "CPU Usage %",
      "type": "timeseries",
      "targets": [
        { "expr": "job:node_cpu_usage:ratio * 100", "legendFormat": "CPU %" }
      ],
      "fieldConfig": {
        "defaults": { "unit": "percent", "min": 0, "max": 100,
          "thresholds": { "steps": [
            { "value": 0, "color": "green" },
            { "value": 70, "color": "yellow" },
            { "value": 85, "color": "red" } ] } }
```

```
      }
    },
    {
      "title": "Memory Usage %",
      "type": "timeseries",
      "targets": [
        { "expr": "job:node_memory_usage:ratio * 100", "legendFormat": "Memory %" }
      ]
    },
    {
      "title": "HTTP Requests / sec by route",
      "type": "timeseries",
      "targets": [
        { "expr": "sum(rate(http_requests_total[5m])) by (route)", "legendFormat":
"{{route}}" }
      ]
    },
    {
      "title": "p95 Latency by route",
      "type": "timeseries",
      "targets": [
        { "expr": "histogram_quantile(0.95,
sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))",
          "legendFormat": "{{route}}" }
      ]
    }
  ]
}
```

Provisioning দিয়ে dashboard auto-load — `grafana/provisioning/dashboards/dashboards.yml`:

```
apiVersion: 1
providers:
  - name: 'technova'
    orgId: 1
    folder: 'TechNova'
    type: file
    disableDeletion: false
    updateIntervalSeconds: 30
    options:
      path: /etc/grafana/provisioning/dashboards
```

> **PRO TIP:** ভালো dashboard-এ 8–12টি প্যানেলের বেশি রাখবেন না। প্রতিটি প্যানেলের প্রশ্ন হওয়া উচিত: "এই গ্রাফ দেখে আমি কী সিদ্ধান্ত নেব?" — উত্তর না থাকলে প্যানেলটি বাদ দিন। Time range ডিফল্ট `Last 6 hours` ও refresh `30s` রাখুন।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Dashboard-এ CPU/Memory গ্রাফ বাস্তব মান দেখায়; API-তে ট্রাফিক পাঠালে Request rate গ্রাফ ওঠে।
- Variable dropdown বদলালে প্যানেলের ডেটা বদলায়।
- Threshold পার হলে Stat প্যানেলের রং বদলায়।

- Export করা JSON ফাইল দিয়ে নতুন Grafana-তে একই dashboard আমদানি করা যায় (Import → Upload JSON)।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
প্যানেলে No data	কুয়েরি বা time range ভুল	Explore-এ একই কুয়েরি চালিয়ে দেখুন; `$instance` variable-এর মান যাচাই করুন
গ্রাফে অনেক লাইন, পড়া যায় না	Aggregation হয়নি	`sum by (route)` ব্যবহার করুন; legend-এ `{{route}}` দিন
Dashboard provisioning-এ আসছে না	JSON ফাইল পথে নেই বা UID সংঘাত	মাউন্ট ও লগ যাচাই করুন; UID অনন্য রাখুন
Unit ভুল দেখাচ্ছে	Unit সেট করা হয়নি	প্যানেল → Standard options → Unit সেট করুন (percent, bytes, reqps)

পরিষ্কারকরণ (Cleanup)

```
# Dashboard মুছতে: Dashboard settings → Delete
# JSON ফাইল রেখে দিন — Project 01-এ আবার লাগবে
```

পর্যালোচনা প্রশ্ন

1. RED dashboard-এ কোন তিনটি প্যানেল থাকা উচিত? কেন?
2. Variable ব্যবহারের সুবিধা কী? একটি dashboard কীভাবে অনেক সার্ভিসে ব্যবহার করা যায়?
3. Stat বনাম Gauge বনাম Time series — কখন কোনটি?
4. Dashboard-এ threshold রং কীভাবে সিদ্ধান্ত নেবেন?
5. Golden signals (latency, traffic, errors, saturation) কী? প্রতিটির একটি metric দিন।

ল্যাব 14 • Observability Lab Workbook

Lab 14 — Configure Grafana alerting and provisioning

উদ্দেশ্য

Grafana-managed alerting (Grafana Alerting) দিয়ে alert rule, contact point ও notification policy তৈরি করবেন এবং provisioning YAML দিয়ে alert rule-কে কোড হিসেবে সংরক্ষণ করবেন। Prometheus alert বনাম Grafana alert-এর পার্থক্য বুঝবেন।

প্রয়োজনীয় টুল

- Grafana 11.x, Prometheus data source (Lab 12)
- ঐচ্ছিক: Slack webhook বা SMTP (secret ছাড়া পরীক্ষার জন্য webhook.site ব্যবহার করা যায়)

পূর্বশর্ত

Lab 10-এর Prometheus alert ও Lab 13-এর dashboard জানা আছে। Grafana UI-তে Alerting মেনু খুঁজে পান।

আর্কিটেকচার



Prometheus alert বনাম Grafana alert: Prometheus alert rule Prometheus-এ চলে, Alertmanager-এ যায়। Grafana alert rule Grafana-তে চলে, যেকোনো data source (Prometheus, Loki, CloudWatch) ব্যবহার করতে পারে এবং Grafana-র UI-তেই notification যায়। দুটো একসাথে ব্যবহার করলে ডুপ্লিকেট alert এড়াতে একটিকে প্রাধান্য দিন।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — UI দিয়ে alert rule তৈরি করুন

Grafana → Alerting → Alert rules → New alert rule → নাম: `TechNova High CPU` → Query A: `job:node_cpu_usage:ratio * 100` → Condition: `WHEN last() OF A IS ABOVE 85` → Evaluate every `1m`, for `5m` → Folder: TechNova, Labels: `severity=warning`।

ধাপ ২ — Contact point যোগ করুন

Alerting → Contact points → New contact point → নাম: `platform-webhook` → Integration: Webhook → URL: `http://webhook-receiver:5001/grafana` (পরীক্ষার জন্য) → Test → Save।

ধাপ ৩ — Notification policy

Alerting → Notification policies → Default policy-তে contact point `platform-webhook` সেট করুন। Label matcher `severity=critical`-এর জন্য আলাদা policy তৈরি করুন (ঐচ্ছিক)।

ধাপ ৪ — Mute timing (রক্ষণাবেক্ষণের সময়)

Alerting → Mute timings → New mute timing → নাম: `weekend-maintenance` → শনি 02:00–04:00 → Notification policy-তে যুক্ত করুন।

ধাপ ৫ — Provisioning দিয়ে alert rule সংরক্ষণ করুন

Grafana 11-এ alert rule provisioning ফাইল `grafana/provisioning/alerting/alerts.yml`-এ লেখা যায় (নিচে উদাহরণ)। ফাইলটি রেখে Grafana রিস্টার্ট করুন।

সম্পূর্ণ কনফিগারেশন

grafana/provisioning/alerting/alerts.yml (উদাহরণ — Grafana 11 ফরম্যাট)

```
apiVersion: 1
groups:
  - orgId: 1
    name: technova-grafana-alerts
    folder: TechNova
    interval: 1m
    rules:
      - uid: technova-high-cpu
        title: TechNova High CPU
        condition: C
        data:
          - refId: A
            queryType: ''
            relativeTimeRange: { from: 600, to: 0 }
            datasourceUid: '__expr__'
            model:
              expr: 'job:node_cpu_usage:ratio * 100'
              refId: A
              datasource: { type: 'prometheus', uid: 'prometheus-uid' }
          - refId: C
            datasourceUid: '__expr__'
```

```
        model:
          expression: 'A'
          conditions:
            - evaluator: { params: [85], type: 'gt' }
              operator: { type: 'and' }
              query: { params: ['C'] }
              reducer: { params: [], type: 'last' }
        refId: C
    for: 5m
    labels: { severity: 'warning' }
    annotations:
      summary: 'High CPU on TechNova host'
      description: 'CPU usage above 85% for 5 minutes.'
```

> **WARNING:** Grafana alert provisioning YAML-এর ফরম্যাট সংস্করণভেদে বদলায়। UI দিয়ে তৈরি করা rule-কে export করে (Alert rules → Export) সঠিক ফরম্যাট পাওয়া সবচেয়ে নিরাপদ উপায়। হাতে লেখা YAML সবসময় `grafana-cli` বা রিস্টার্টের পর লগ দেখে যাচাই করুন।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Alert rules তালিকায় rule `Normal` অবস্থায় দেখা যায়।
- CPU লোড তৈরি করলে (`stress-ng`) rule `Pending` → `Firing` হয়।
- Firing হলে contact point-এ webhook notification যায়।
- Alert state history-তে প্রতিটি পরিবর্তনের সময় লেখা থাকে।
- Silence/Mute timing চলাকালে notification থেমে থাকে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Alert rule Error অবস্থায়	কুয়েরি বা data source UID ভুল	Rule-এর Query preview চালান; datasourceUid যাচাই করুন
Notification আসছে না	Contact point বা policy ভুল	Contact point-এ Test পাঠান; Default policy যাচাই করুন
Prometheus ও Grafana দুটোতেই alert আসছে	ডুপ্লিকেট rule	একটি বন্ধ করুন — সাধারণত Prometheus+Alertmanager-কে প্রাধান্য দিন, Grafana alert শুধু Loki/বিশেষ ক্ষেত্রে
Provisioning rule লোড হচ্ছে না	YAML ফরম্যাট ভুল	UI থেকে export করা YAML ব্যবহার করুন; লগ দেখুন

পরিষ্কারকরণ (Cleanup)

```
# Alert rule মুছতে: Alerting → Alert rules → Delete
# Contact point মুছতে: Contact points → Delete (policy থেকে সরিয়ে)
```

পর্যালোচনা প্রশ্ন

1. Prometheus alert বনাম Grafana alert — কখন কোনটি বেছে নেবেন?
2. Grafana alert-এ `for` সময়সীমার কাজ কী?
3. Contact point, notification policy ও mute timing-এর ভূমিকা কী?
4. Alert rule-কে কোড হিসেবে রাখা কেন গুরুত্বপূর্ণ?
5. একটি alert rule লিখুন যা Loki log-এর error সংখ্যা দেখে alert দেয় (Lab 17-এর প্রস্তুতি)।

ল্যাব 15 • Observability Lab Workbook

Lab 15 — Implement structured JSON logging

উদ্দেশ্য

TechNova API-তে unstructured (সাধারণ টেক্সট) লগের বদলে structured JSON লগ চালু করবেন। প্রতিটি লগ লাইনে timestamp, level, service, request ID (trace ID) ও context থাকবে — যাতে Loki/LogQL দিয়ে খোঁজা যায় এবং trace-এর সাথে যুক্ত করা যায়।

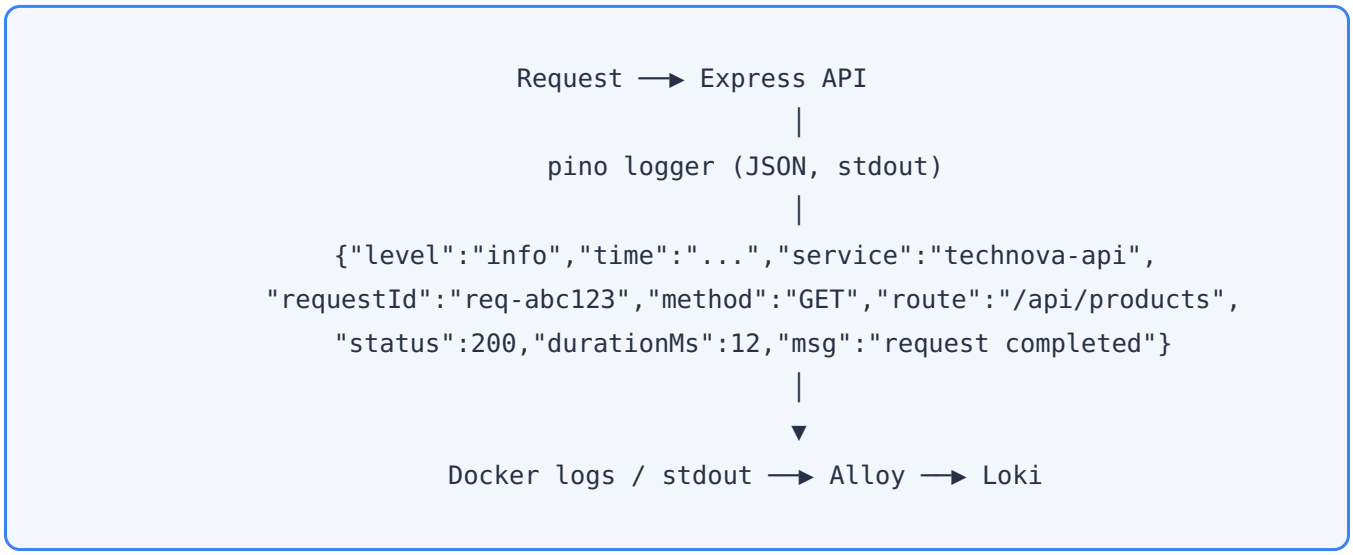
প্রয়োজনীয় টুল

- Node.js 20, `pino` বা `winston` (এই ল্যাবে pino)
- Lab 7-এর TechNova API

পূর্বশর্ত

Express middleware-এর ধারণা আছে। JSON ফরম্যাট পড়তে পারেন।

আর্কিটেকচার



Structured logging-এর নীতি: লগ মানুষের পড়ার জন্য নয়, মেশিনের খোঁজার জন্য। প্রতিটি লাইনে একই field-গুলো থাকবে; বার্তা (msg) ছোট হবে; সংবেদনশীল তথ্য (পাসওয়ার্ড, টোকেন) কখনো লগে আসবে না।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — pino ইনস্টল করুন

```
cd ~/technova-api
npm install pino pino-http
npm install -D pino-pretty
```

ধাপ ২ — Logger তৈরি করুন

`logger.js` তৈরি করুন (সম্পূর্ণ ফাইল নিচে)।

ধাপ ৩ — server.js-এ যুক্ত করুন

`console.log`-এর বদলে logger ব্যবহার করুন এবং request logger middleware যোগ করুন।

ধাপ ৪ — চালু করে লগ দেখুন

```
node server.js
curl http://localhost:3000/api/products
curl -X POST http://localhost:3000/api/orders -H 'Content-Type: application/json' -d
'{"productId":"1","qty":1}'
```

টার্মিনালে JSON লাইন দেখা যাবে। উন্নয়ন (development)-এ সুন্দরভাবে দেখতে: `node server.js | npx pino-pretty`।

ধাপ ৫ — ভুল লগ তৈরি করে দেখুন

```
curl http://localhost:3000/api/products/999
# লগে level=error/warn, status=404, requestId সহ লাইন আসবে
```

সম্পূর্ণ কনফিগারেশন

logger.js

```
const pino = require('pino');

const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  base: { service: 'technova-api', env: process.env.NODE_ENV || 'lab' },
  timestamp: pino.stdTimeFunctions.isoTime,
  // লালিত তথ্য (redaction): এই field-গুলো লগে এলে মুছে যাবে
  redact: {
    paths: ['req.headers.authorization', 'password', 'token', 'creditCard'],
    censor: '[REDACTED]',
  },
});

module.exports = logger;
```

server.js — logging অংশ (Lab 7–8-এর ফাইলে যোগ করুন)

```
const logger = require('./logger');
const pinoHttp = require('pino-http');

// প্রতিটি request-এ একটি requestId দিন (trace-এর সাথে মেলানোর জন্য)
app.use((req, res, next) => {
  req.id = req.headers['x-request-id'] || `req-${Date.now()}-${Math.floor(Math.random() *
10000)}`;
  res.setHeader('x-request-id', req.id);
  next();
```

```
});

app.use(pinoHttp({
  logger,
  customLogLevel: (req, res, err) => {
    if (res.statusCode >= 500 || err) return 'error';
    if (res.statusCode >= 400) return 'warn';
    return 'info';
  },
  customSuccessMessage: (req, res) => `request completed`,
}));

// ব্যবসায়িক ঘটনার লগ
app.post('/api/orders', (req, res) => {
  // ... আগের কোড ...
  logger.info({ orderId: 'ord-123', productId: req.body.productId, qty: req.body.qty,
requestId: req.id }, 'order placed');
  // ব্যর্থতার ক্ষেত্রে:
  // logger.error({ err, orderId, requestId: req.id }, 'payment failed');
});
```

নমুনা লগ লাইন (নমুনা আউটপুট — চিত্রণমূলক)

```
{"level":30,"time":"2026-11-09T06:00:00.000Z","service":"technova-
api","env":"lab","requestId":"req-1730000000-1234","method":"GET","route":"/api/
products","status":200,"durationMs":12,"msg":"request completed"}
{"level":50,"time":"2026-11-09T06:01:00.000Z","service":"technova-
api","env":"lab","requestId":"req-1730000000-5678","method":"GET","route":"/api/products/
999","status":404,"msg":"request completed"}
```

pino-তে level সংখ্যা: 10 trace, 20 debug, 30 info, 40 warn, 50 error, 60 fatal।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- প্রতিটি request-এর জন্য একটি JSON লাইন stdout-এ আসে।
- প্রতিটি লাইনে `service`, `requestId`, `status`, `durationMs` field আছে।
- 404 request-এর লগ level `warn` (40) বা `error` (50)।
- `node server.js | npx pino-pretty` চালালে একই লগ মানুষের পড়ার মতো ফরম্যাটে দেখা যায়।
- Authorization header লগে এলে `[REDACTED]` দেখায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
লগ JSON নয়, টেক্সট আসছে	`console.log` এখনো ব্যবহৃত হচ্ছে	সব `console.log`-কে logger দিয়ে বদলান
requestId সব request-এ একই	Middleware-এর ক্রম ভুল	requestId middleware-কে pinoHttp-এর আগে রাখুন
	Redact পথ ভুল	

লগে সংবেদনশীল তথ্য আসছে		`redact.paths` -এ সঠিক পথ দিন; পুরো request body লগ করবেন না
Docker-এ লগ দেখা যাচ্ছে না	stdout buffer হচ্ছে	Node.js-এ stdout সাধারণত line-buffered; `docker compose logs -f api` দিয়ে দেখুন

পরিষ্কারকরণ (Cleanup)

```
pkill -f "node server.js" || true
# logger.js রেখে দিন — Lab 16-এ Loki-তে এই লগই পাঠানো হবে
```

পর্যালোচনা প্রশ্ন

1. Structured logging কেন unstructured-এর চেয়ে ভালো? তিনটি কারণ দিন।
2. প্রতিটি লগ লাইনে কোন field-গুলো থাকা উচিত? কেন?
3. requestId কী কাজে লাগে? এটি trace ID-র সাথে কীভাবে যুক্ত হয়?
4. লগে কী কী জিনিস কখনো লেখা উচিত নয়? কেন?
5. Log level (debug/info/warn/error) কীভাবে বেছে নেবেন? উদাহরণ দিন।

ল্যাব 16 • Observability Lab Workbook

Lab 16 — Collect logs using Loki and Grafana Alloy

উদ্দেশ্য

Grafana Loki দিয়ে কেন্দ্রীয় log সংগ্রহ (centralized logging) তৈরি করবেন এবং Grafana Alloy দিয়ে Docker কনটেইনারের লগ সংগ্রহ করে Loki-তে পাঠাবেন। Label (যেমন `service`, `level`) দিয়ে লগ সংগঠিত করা শিখবেন।

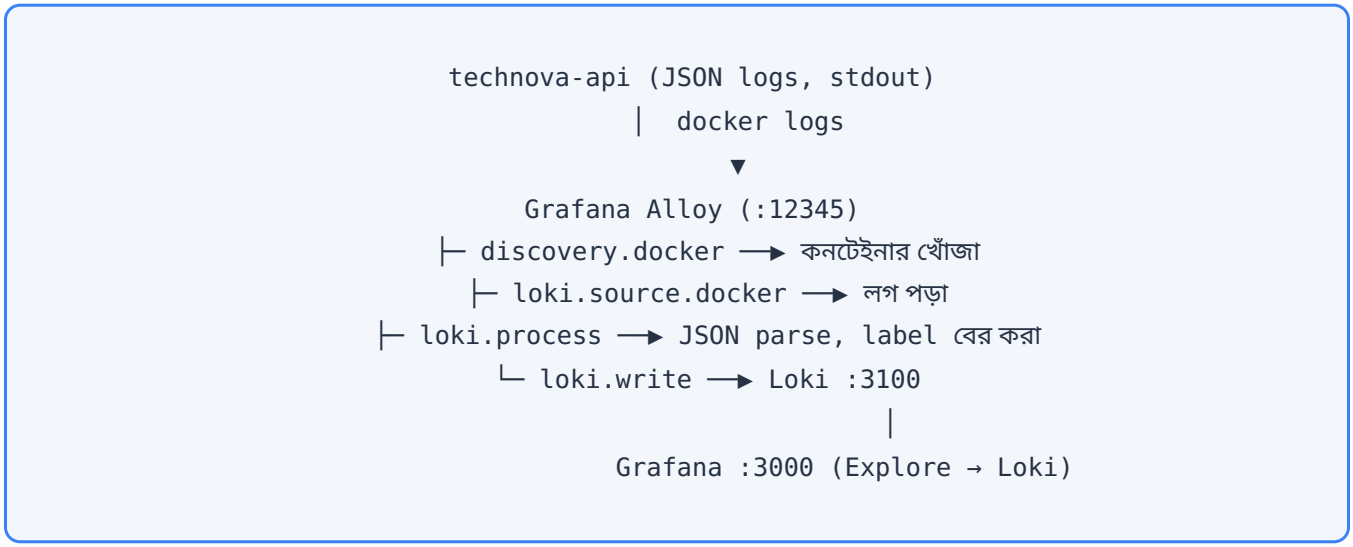
প্রয়োজনীয় টুল

- Loki 3.x, Grafana Alloy (alloy), Grafana 11.x
- Lab 15-এর structured JSON লগসহ TechNova API

পূর্বশর্ত

Lab 15 সম্পূর্ণ — API JSON লগ দেয়। Docker Compose-এ নতুন সার্ভিস যোগ করতে পারেন।

আর্কিটেকচার



Loki বনাম Elasticsearch: Loki শুধু label-কে index করে, লগের পুরো টেক্সটকে নয় — তাই হালকা ও সস্তা। Elasticsearch পুরো টেক্সট index করে — খোঁজা শক্তিশালী কিন্তু সম্পদ বেশি লাগে। ছোট দলের জন্য Loki দিয়ে শুরু করা ভালো।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — Compose-এ Loki ও Alloy যোগ করুন

```
  loki:
    image: grafana/loki:3.2.0
    container_name: loki
    ports:
```

```
      - "3100:3100"
    volumes:
      - ./loki/loki-config.yml:/etc/loki/local-config.yaml:ro
      - loki-data:/loki
    command: -config.file=/etc/loki/local-config.yaml
    restart: unless-stopped

  alloy:
    image: grafana/alloy:v1.3.0
    container_name: alloy
    ports:
      - "12345:12345"
    volumes:
      - ./alloy/config.river:/etc/alloy/config.river:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    command:
      - run
      - /etc/alloy/config.river
      - --server.http.listen-addr=0.0.0.0:12345
    restart: unless-stopped
```

ধাপ ২ — Loki কনফিগারেশন লিখুন

`loki/loki-config.yml` (নিচে সম্পূর্ণ ফাইল)।

ধাপ ৩ — Alloy config.river লিখুন

`alloy/config.river` (নিচে সম্পূর্ণ ফাইল) — Docker কনটেইনারের লগ পড়ে, JSON parse করে `level` ও `service` label বের করে।

ধাপ ৪ — API-কে Compose-এ চালান

Lab 7-এর Dockerfile ব্যবহার করে `api` সার্ভিস যোগ করুন (পোর্ট 3000)। Alloy স্বয়ংক্রিয়ভাবে এর লগ খুঁজে পাবে।

ধাপ ৫ — Grafana-তে Loki যাচাই করুন

Grafana → Explore → Data source: Loki → কুয়েরি: `{service="technova-api"}` → Run। লগ লাইন দেখা যাবে।

সম্পূর্ণ কনফিগারেশন

loki/loki-config.yml

```
auth_enabled: false

server:
  http_listen_port: 3100
  grpc_listen_port: 9096

common:
  instance_addr: 127.0.0.1
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
```

```
  ring:
    kvstore:
      store: inmemory

query_range:
  results_cache:
    cache:
      embedded_cache:
        enabled: true
        max_size_mb: 100

schema_config:
  configs:
    - from: 2024-04-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

ruler:
  alertmanager_url: http://alertmanager:9093

limits_config:
  retention_period: 168h
  max_label_names_per_series: 20
```

alloy/config.river

```
// Docker কনটেইনার খুঁজুন
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"
  refresh_interval = "5s"
}

// শুধু technova লেবেলযুক্ত কনটেইনারের লগ নিন
discovery.relabel "technova_only" {
  targets = discovery.docker.containers.targets
  rule {
    source_labels = ["__meta_docker_container_name"]
    regex = "/(technova.*|api|prometheus|grafana)"
    action = "keep"
  }
  rule {
    source_labels = ["__meta_docker_container_name"]
    target_label = "container"
  }
}

loki.source.docker "logs" {
  host = "unix:///var/run/docker.sock"
  targets = discovery.relabel.technova_only.output
  forward_to = [loki.process.json_parse.receiver]
}

loki.process "json_parse" {
  forward_to = [loki.write.loki.receiver]
```

```
  // JSON লগ parse করে level ও service label বের করুন
  stage.json {
    expressions = {
      level = "level",
      service = "service",
      requestId = "requestId",
    }
  }
  stage.labels {
    values = {
      level = "level",
      service = "service",
    }
  }
  // pino level সংখ্যাকে নামে বদলান (ঐচ্ছিক)
  stage.replace {
    expression = "level=\"30\""
    replace = "level=\"info\""
  }
}

loki.write "loki" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

WARNING: Alloy-কে Docker socket (`/var/run/docker.sock`) দেওয়া মানে Alloy হোস্টের Docker নিয়ন্ত্রণ করতে পারে — এটি শক্তিশালী অনুমতি। শুধু বিশ্বস্ত পরিবেশে (ল্যাব) ব্যবহার করুন; production-এ সীমিত অনুমতির ব্যবস্থা করুন।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `curl http://localhost:3100/ready` — `ready` ফেরত আসে।
- Grafana Explore → Loki → `{service="technova-api"}` — API-র JSON লগ লাইন দেখা যায়।
- `{service="technova-api"} |= "error"` — শুধু error ধারণকারী লাইন দেখায়।
- Alloy UI (`http://localhost:12345`) — component-গুলো healthy দেখায়।
- API-তে নতুন request পাঠালে কয়েক সেকেন্ডের মধ্যে Loki-তে লগ আসে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Loki-তে লগ আসছে না	Alloy কনটেইনার খুঁজে পাচ্ছে না	Alloy লগ দেখুন: `docker compose logs alloy` ; container নামের regex যাচাই করুন
Label খুব বেশি (cardinality)	requestId-কে label করা হয়েছে	requestId-কে label করবেন না — শুধু JSON field হিসেবে রাখুন; label শুধু কম মানের field (service, level)

Loki ready নয়	কনফিগারেশন ভুল	`docker compose logs loki` দেখুন; schema_config-এর `from` তারিখ যাচাই করুন
Grafana-তে Loki data source ব্যর্থ	URL ভুল	URL `http://loki:3100` (Compose নেটওয়ার্কে) ব্যবহার করুন

পরিষ্কারকরণ (Cleanup)

```
docker compose stop loki alloy
# ডেটা মুছতে চাইলে: docker compose down -v  (সতর্কতা: সব metrics/log মুছে যাবে)
```

পর্যালোচনা প্রশ্ন

1. Loki কেন শুধু label index করে? এর সুবিধা ও অসুবিধা কী?
2. কোন field-গুলো label হওয়া উচিত, আর কোনগুলো নয়? কেন?
3. Alloy (এজেন্ট) বনাম Promtail — পার্থক্য কী?
4. Docker socket দেওয়ার নিরাপত্তা ঝুঁকি কী?
5. Retention 168h মানে কী? Production-এ retention কীভাবে ঠিক করবেন?

ল্যাব 17 • Observability Lab Workbook

Lab 17 — Analyze logs with LogQL

উদ্দেশ্য

LogQL — Loki-র কুয়েরি ভাষা — দিয়ে লগ খোঁজা, ফিল্টার করা, JSON parse করা ও metric-এ রূপান্তর করা শিখবেন। Error হার, প্রতি মিনিটে request সংখ্যা ও ধীর request খুঁজে বের করবেন।

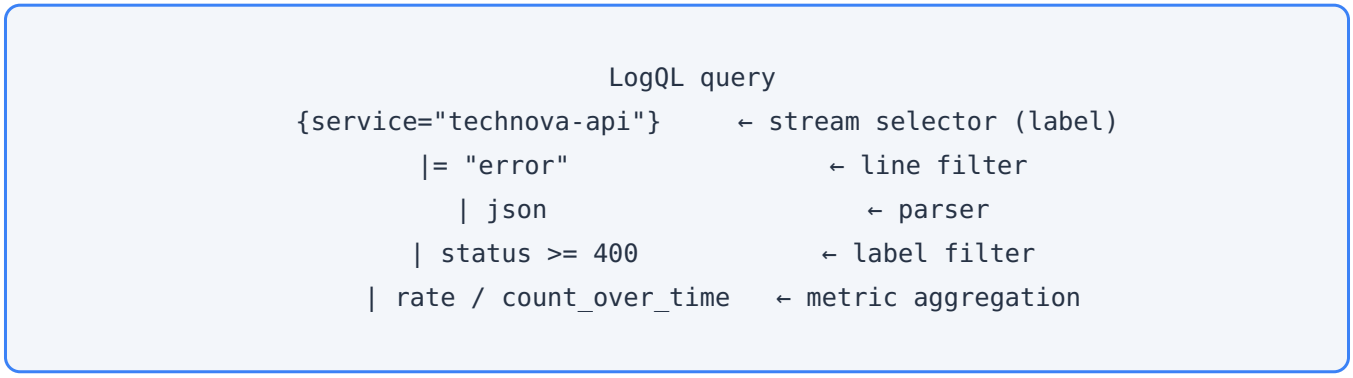
প্রয়োজনীয় টুল

- Loki 3.x + Alloy (Lab 16), Grafana Explore
- TechNova API-তে কিছু ট্রাফিক (সফল ও ব্যর্থ request)

পূর্বশর্ত

Lab 16 সম্পূর্ণ — Loki-তে লগ আসছে। PromQL-এর ধারণা আছে (LogQL-এর অনেক ধারণা একই)।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — Stream selector

```
{service="technova-api"}
{service="technova-api", level="error"}
{container=~"api|web"}
```

ধাপ ২ — Line filter

```
{service="technova-api"} |= "error"
{service="technova-api"} != "health"
{service="technova-api"} |~ "status\":(4|5)[0-9][0-9]"
{service="technova-api"} |= "order" |= "failed"
```

> **CONCEPT:** `|=` মানে "ধারণ করে", `!=` মানে "ধারণ করে না", `|~` মানে regex মেলে, `!~` মানে regex মেলে না। একাধিক ফিল্টার পরপর লেখা যায় — এগুলো AND হিসেবে কাজ করে।

ধাপ ৩ — Parser: JSON field বের করা

```
{service="technova-api"} | json
{service="technova-api"} | json | status >= 400
{service="technova-api"} | json | durationMs > 500
```

ধাপ ৪ — Metric aggregation

```
rate({service="technova-api"}[5m])
sum(rate({service="technova-api"} |= "error" [5m]))
sum by (level) (count_over_time({service="technova-api"}[5m]))
sum(rate({service="technova-api"} | json | status >= 500 [5m]))
```

ধাপ ৫ — Grafana-তে লগ প্যানেল তৈরি করুন

Lab 13-এর dashboard-এ একটি Logs প্যানেল যোগ করুন — কুয়েরি `{service="technova-api"} | json | status >= 400`।

সম্পূর্ণ কনফিগারেশন

এই ল্যাবে নতুন ফাইল নেই — নিচে চিট-শিট টেবিল:

প্রয়োজন	LogQL
সব error লগ	`{service="technova-api"} |= "error"`
4xx/5xx লগ	`{service="technova-api"} | json | status >= 400`
ধীর request (>500ms)	`{service="technova-api"} | json | durationMs > 500`
প্রতি মিনিটে error সংখ্যা	`sum(count_over_time({service="technova-api"} |= "error" [1m]))`
Error হার %	`sum(rate({service="technova-api"} | json | status >= 500 [5m])) / sum(rate({service="technova-api"}[5m])) * 100`
একটি requestId খোঁজা	`{service="technova-api"} |= "req-1730000000-1234"`
Top error বার্তা	`topk(5, sum by (msg) (count_over_time({service="technova-api"} |= "error" [1h])))`

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Grafana Explore-এ প্রতিটি কুয়েরি লগ লাইন বা সংখ্যা দেখায়।
- API-তে 404 request পাঠালে `status >= 400` কুয়েরিতে তা দেখা যায়।
- `count_over_time` কুয়েরি সময়ের সাথে গ্রাফ আঁকে।
- একটি নির্দিষ্ট requestId দিয়ে খুঁজলে শুধু সেই request-এর লগ আসে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
কুয়েরিতে No data	Label নাম ভুল বা time range-এ লগ নেই	`{service="technova-api"}` দিয়ে শুরু করুন; time range বাড়ান
`| json` কাজ করছে না	লগ লাইন JSON নয়	লগ লাইনটি দেখুন — pino JSON দিচ্ছে কি না যাচাই করুন (Lab 15)
`status >= 400` ভুল ফল দিচ্ছে	status স্ট্রিং হিসেবে parse হয়েছে	JSON সংখ্যা হলে ঠিক থাকে; না হলে `| json | status="500"` (স্ট্রিং তুলনা) ব্যবহার করুন
কুয়েরি ধীর	বড় time range, অনেক stream	Label দিয়ে stream কমান; time range ছোট করুন

পরিষ্কারকরণ (Cleanup)

```
# এই ল্যাবে কোনো cleanup নেই — Loki চালু রাখুন, Lab 22-এ আবার লাগবে
```

পর্যালোচনা প্রশ্ন

1. LogQL-এর stream selector বনাম line filter — পার্থক্য কী? কোনটি আগে চলে?
2. `| json` parser কী করে? এটি কখন ব্যর্থ হয়?
3. Log থেকে metric তৈরি করা (যেমন error হার) কখন দরকার হয়?
4. একটি requestId দিয়ে লগ খোঁজা কেন trace-এর চেয়ে কম শক্তিশালী?
5. Loki-তে alert লেখা যায় কীভাবে? (Ruler-এর নাম লিখুন)

ল্যাব 18 • Observability Lab Workbook

Lab 18 — Explore Elasticsearch or OpenSearch logging

উদ্দেশ্য

Elasticsearch/OpenSearch-ভিত্তিক logging-এর ধারণা ও বাস্তব ব্যবহার শিখবেন — কখন Loki-র বদলে এটি দরকার, কীভাবে index, mapping ও Kibana/OpenSearch Dashboards দিয়ে খোঁজা যায়। এই ল্যাবে Docker-এ single-node OpenSearch চালিয়ে TechNova API-র লগ পাঠিয়ে পরীক্ষা করবেন।

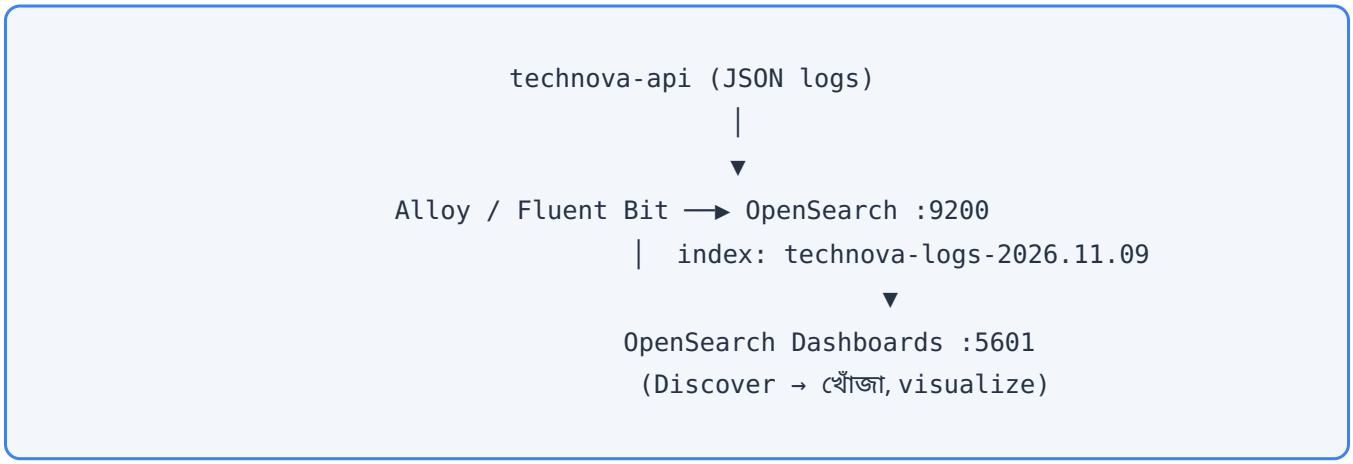
প্রয়োজনীয় টুল

- Docker Compose, OpenSearch 2.x (single-node, ল্যাবের জন্য)
- 8 GB RAM সুপারিশ করা হয় (OpenSearch ভারী); কম RAM-এ শুধু ধারণা পড়ে যাচাই করুন

পূর্বশর্ত

Lab 15–17 সম্পূর্ণ — structured JSON লগ ও LogQL জানেন। REST API ও JSON-এর ধারণা আছে।

আর্কিটেকচার



Loki বনাম OpenSearch: Loki হালকা, সস্তা, label-ভিত্তিক — ছোট দল ও সাধারণ খোঁজার জন্য যথেষ্ট। OpenSearch/Elasticsearch পুরো লগ টেক্সট index করে — full-text search, জটিল aggregation ও দীর্ঘ retention-এর জন্য শক্তিশালী, তবে RAM/CPU ও পরিচালনা খরচ বেশি। Production-এ সিদ্ধান্ত নেওয়ার আগে দুটোর খরচ ও প্রয়োজন মাপুন।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — OpenSearch চালু করুন (single-node, ল্যাবের জন্য)

```
opensearch:
  image: opensearchproject/opensearch:2.15.0
```

```
    container_name: opensearch
    environment:
      - cluster.name=technova-lab
      - node.name=opensearch-node1
      - discovery.type=single-node
      - bootstrap.memory_lock=true
      - "OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m"
      - OPENSEARCH_INITIAL_ADMIN_PASSWORD=CHANGE-ME-opensearch-admin
    ulimits:
      memlock: { soft: -1, hard: -1 }
      nofile: { soft: 65536, hard: 65536 }
    volumes:
      - opensearch-data:/usr/share/opensearch/data
    ports:
      - "9200:9200"
      - "9600:9600"
    restart: unless-stopped

  opensearch-dashboards:
    image: opensearchproject/opensearch-dashboards:2.15.0
    container_name: opensearch-dashboards
    ports:
      - "5601:5601"
    environment:
      - 'OPENSEARCH_HOSTS=["http://opensearch:9200"]'
    depends_on:
      - opensearch
    restart: unless-stopped
```

> **WARNING:** `OPENSEARCH_INITIAL_ADMIN_PASSWORD`-এ আসল পাসওয়ার্ডের বদলে `CHANGE-ME-...` ব্যবহার করুন এবং ল্যাব শেষে পাল্টান। Single-node ও 512 MB heap শুধু ল্যাবের জন্য — production-এ কমপক্ষে 3 node ও সঠিক heap সাইজিং দরকার।

ধাপ ২ — Health যাচাই করুন

```
curl -k -u admin:CHANGE-ME-opensearch-admin https://localhost:9200/
curl -k -u admin:CHANGE-ME-opensearch-admin https://localhost:9200/_cluster/health
```

OpenSearch 2.x ডিফল্ট HTTPS ব্যবহার করে — `curl -k` ল্যাবের জন্য।

ধাপ ৩ — একটি লগ document পাঠান

```
curl -k -u admin:CHANGE-ME-opensearch-admin -X POST https://localhost:9200/technova-
logs-2026.11.09/_doc \
  -H 'Content-Type: application/json' -d '{
    "@timestamp": "2026-11-09T06:00:00.000Z",
    "service": "technova-api",
    "level": "info",
    "message": "request completed",
    "method": "GET",
    "route": "/api/products",
    "status": 200,
    "durationMs": 12,
```

```
    "requestId": "req-1730000000-1234"
  }'
```

ধাপ ৪ — খুঁজুন

```
curl -k -u admin:CHANGE-ME-opensearch-admin -X GET https://localhost:9200/technova-
logs-2026.11.09/_search \
  -H 'Content-Type: application/json' -d '{
    "query": { "match": { "level": "error" } },
    "sort": [{ "@timestamp": "desc" }]
  }'
# status 500-র সংখ্যা:
curl -k -u admin:CHANGE-ME-opensearch-admin -X GET https://localhost:9200/technova-
logs-2026.11.09/_search \
  -H 'Content-Type: application/json' -d '{
    "size": 0,
    "aggs": { "by_status": { "terms": { "field": "status" } } }
  }'
```

ধাপ ৫ — Dashboards-এ দেখুন

ব্রাউজারে `http://localhost:5601` খুলুন → Stack Management → Index Patterns → `technova-logs-*` → Discover-এ লগ দেখুন।

সম্পূর্ণ কনফিগারেশন

Index template (mapping) — উদাহরণ

```
PUT /_index_template/technova-logs
{
  "index_patterns": ["technova-logs-*"],
  "template": {
    "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
    "mappings": {
      "properties": {
        "@timestamp": { "type": "date" },
        "service": { "type": "keyword" },
        "level": { "type": "keyword" },
        "message": { "type": "text" },
        "status": { "type": "integer" },
        "durationMs": { "type": "float" },
        "requestId": { "type": "keyword" },
        "route": { "type": "keyword" }
      }
    }
  }
}
```

Fluent Bit দিয়ে API লগ পাঠানো (উদাহরণ — Alloy-র বিকল্প)

```
[INPUT]
    Name tail
    Path /var/log/containers/*technova-api*.log
    Parser docker
```

```
    Tag technova.api

[OUTPUT]
    Name opensearch
    Match technova.*
    Host opensearch
    Port 9200
    Index technova-logs
    HTTP_User admin
    HTTP_Passwd CHANGE-ME-opensearch-admin
    tls On
    tls.verify Off
```

CONCEPT: `keyword` টাইপ exact match ও aggregation-এর জন্য; `text` টাইপ full-text search-এর জন্য। ভুল mapping (যেমন status-কে text করা) হলে aggregation কাজ করে না — তাই index template আগে থেকে ঠিক করা গুরুত্বপূর্ণ।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `_cluster/health` — `status: green` (single-node) বা `yellow` (replica নেই বলে)।
- Document পাঠানোর পর `_search`-এ তা ফেরত আসে।
- `level: error` খুঁজলে শুধু error লগ আসে।
- Aggregation-এ status অনুযায়ী সংখ্যা দেখা যায়।
- Dashboards-এ Discover ভিউতে লগ লাইন দেখা যায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
OpenSearch চালু হচ্ছে না	RAM কম বা memlock সীমা	Heap 512m-এ রাখুন; `docker compose logs opensearch` দেখুন; `vm.max_map_count` বাড়াতে হতে পারে (`sysctl -w vm.max_map_count=262144`)
401 Unauthorized	পাসওয়ার্ড ভুল	প্রথম চালুতে দেওয়া পাসওয়ার্ড ব্যবহার করুন; ভলিউম মুছে আবার চালু করলে নতুন পাসওয়ার্ড সেট হয়
Index pattern তৈরি হচ্ছে না	কোনো document নেই	আগে একটি document পাঠান, তারপর pattern তৈরি করুন
খরচ/সম্পদ বেশি	OpenSearch ভারী	ল্যাবের জন্য Loki-ই যথেষ্ট — OpenSearch শুধু ধারণা ও তুলনার জন্য

পরিষ্কারকরণ (Cleanup)

```
docker compose stop opensearch opensearch-dashboards
docker compose down -v   # শুধু তখনই, যখন OpenSearch ডেটা মুছতে চান (সতর্কতা: সব ডেটা মুছে যাবে)
```

> **WARNING:** OpenSearch single-node ল্যাবের জন্য। Production-এ single-node ব্যবহার করবেন না — ডেটা হারানোর ঝুঁকি থাকে।

পর্যালোচনা প্রশ্ন

1. Loki বনাম OpenSearch — কখন কোনটি বেছে নেবেন? তিনটি পার্থক্য লিখুন।
2. Index template কেন দরকার? Mapping ভুল হলে কী হয়?
3. `keyword` বনাম `text` field-এর পার্থক্য কী?
4. Single-node OpenSearch production-এ কেন ঝুঁকিপূর্ণ?
5. আপনার দলের জন্য লগ retention 30 দিন হলে খরচ কীভাবে অনুমান করবেন?

ল্যাব 19 • Observability Lab Workbook

Lab 19 — Instrument an application using OpenTelemetry

উদ্দেশ্য

OpenTelemetry (OTel) দিয়ে TechNova API-তে distributed tracing চালু করবেন — প্রতিটি HTTP request-এর জন্য একটি trace ও span তৈরি হবে, যাতে request-এর যাত্রাপথ (API → MongoDB → বাইরের সার্ভিস) দেখা যায়। Auto-instrumentation ও manual span দুটোই শিখবেন।

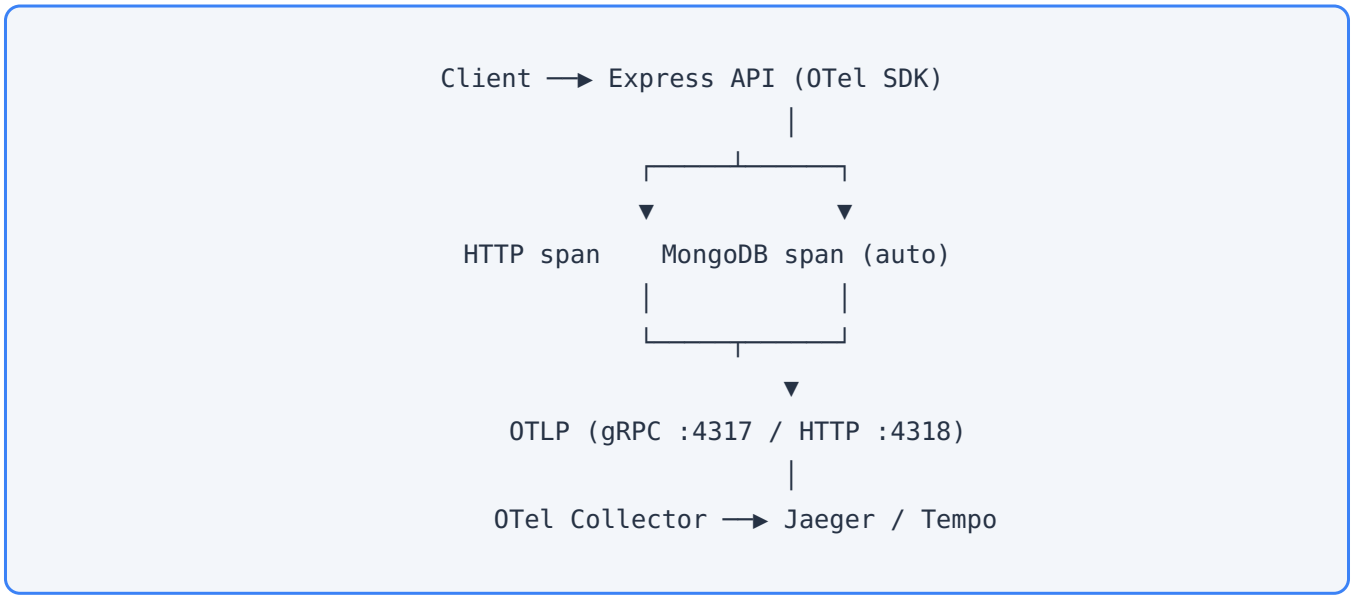
প্রয়োজনীয় টুল

- Node.js 20, `@opentelemetry/sdk-node`, `@opentelemetry/auto-instrumentations-node`
- OTel Collector (Lab 20) বা সরাসরি Jaeger/Tempo (Lab 21)

পূর্বশর্ত

Lab 7-এর API চালু আছে। Trace, span ও context propagation-এর ধারণা এই ল্যাবে ব্যাখ্যা করা আছে।

আর্কিটেকচার



Trace, Span, Context: একটি Trace হলো একটি request-এর সম্পূর্ণ যাত্রা; Span হলো যাত্রার একটি ধাপ (যেমন একটি DB query); Context (trace ID + span ID) প্রতিটি সার্ভিসে header (`traceparent`) দিয়ে পাঠানো হয়, যাতে আলাদা সার্ভিসের span একই trace-এ জোড়া লাগে।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — OTel প্যাকেজ ইনস্টল করুন

```
cd ~/technova-api
npm install @opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node @opentelemetry/
exporter-trace-otlp-grpc @opentelemetry/api
```

ধাপ ২ — tracing.js লিখুন

`tracing.js` — সম্পূর্ণ ফাইল নিচে। এটি `server.js`-এর আগে লোড হতে হবে।

ধাপ ৩ — server.js-এর শুরুতে লোড করুন

```
require('./tracing');   // সবার আগে, অন্য require-এর আগে
const express = require('express');
// ... বাকি কোড
```

ধাপ ৪ — Manual span যোগ করুন (অর্ডার তৈরি)

`/api/orders` endpoint-এ manual span যোগ করুন (নিচে উদাহরণ)।

ধাপ ৫ — চালু করে ট্রাফিক পাঠান

```
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317 node server.js &
curl http://localhost:3000/api/products
curl -X POST http://localhost:3000/api/orders -H 'Content-Type: application/json' -d
'{"productId":"1","qty":1}'
```

OTel Collector না থাকলে প্রথমে Lab 20 সম্পূর্ণ করুন, তারপর এই ধাপটি করুন।

সম্পূর্ণ কনফিগারেশন

tracing.js

```
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-grpc');
const { Resource } = require('@opentelemetry/resources');
const { SemanticResourceAttributes } = require('@opentelemetry/semantic-conventions');

const sdk = new NodeSDK({
  resource: new Resource({
    [SemanticResourceAttributes.SERVICE_NAME]: 'technova-api',
    [SemanticResourceAttributes.SERVICE_VERSION]: '1.4.2',
    [SemanticResourceAttributes.DEPLOYMENT_ENVIRONMENT]: process.env.NODE_ENV || 'lab',
  }),
  traceExporter: new OTLPTraceExporter({
    url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT || 'http://localhost:4317',
  }),
  instrumentations: [getNodeAutoInstrumentations({
    // অপ্রয়োজনীয় instrumentation বন্ধ করা যায়:
```

```
    '@opentelemetry/instrumentation-fs': { enabled: false },
  })],
});

sdk.start();
console.log('OpenTelemetry tracing started');

// Graceful shutdown
process.on('SIGTERM', () => {
  sdk.shutdown().then(() => process.exit(0));
});
```

Manual span — /api/orders endpoint-এ

```
const { trace } = require('@opentelemetry/api');
const tracer = trace.getTracer('technova-api');

app.post('/api/orders', (req, res) => {
  const span = tracer.startSpan('checkout.process');
  span.setAttribute('order.productId', req.body.productId);
  span.setAttribute('order.qty', req.body.qty);
  try {
    // ... অর্ডার প্রক্রিয়াকরণ ...
    span.setAttribute('order.status', 'success');
    span.addEvent('payment.completed');
    res.status(201).json({ orderId: `ord-${Date.now()}`, status: 'success' });
  } catch (err) {
    span.recordException(err);
    span.setStatus({ code: 2, message: err.message });
    res.status(500).json({ error: 'checkout failed' });
  } finally {
    span.end();
  }
});
```

PRO TIP: Auto-instrumentation দিয়ে শুরু করুন — HTTP, Express, MongoDB, Redis-এর span স্বয়ংক্রিয়ভাবে তৈরি হয়। Manual span শুধু ব্যবসায়িক ধাপে (checkout, payment) যোগ করুন যেখানে auto-instrumentation পৌঁছায় না। প্রতিটি span-এ attribute (যেমন productId) দিন, তবে সংবেদনশীল তথ্য দেবেন না।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- API চালু হলে কনসোলে `OpenTelemetry tracing started` দেখা যায়।
- Request পাঠালে OTel Collector-এর লগে span গ্রহণের বার্তা আসে (Lab 20)।
- Jaeger/Tempo UI-তে `technova-api` সার্ভিস দেখা যায় এবং একটি trace-এ HTTP span + MongoDB span দেখা যায় (Lab 21)।
- Manual span `checkout.process` trace-এ আলাদা span হিসেবে দেখা যায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
কোনো span তৈরি হচ্ছে না	`tracing.js` সবার আগে লোড হয়নি	`require('./tracing')`-কে `server.js`-এর প্রথম লাইনে রাখুন
Collector-এ সংযোগ ব্যর্থ	Endpoint ভুল বা Collector বন্ধ	`OTEL_EXPORTER_OTLP_ENDPOINT` যাচাই করুন; Collector চালু আছে কি না দেখুন
Trace ভেঙে গেছে (আলাদা trace)	Context propagation হয়নি	HTTP client-এ auto-instrumentation চালু আছে কি না দেখুন; `traceparent` header যাচাই করুন
অনেক বেশি span (খরচ বেশি)	সব instrumentation চালু	অপ্রয়োজনীয় instrumentation বন্ধ করুন; sampling সেট করুন (Lab 20)

পরিষ্কারকরণ (Cleanup)

```
pkill -f "node server.js" || true
# tracing.js রেখে দিন — Lab 20–22-এ আবার লাগবে
```

পর্যালোচনা প্রশ্ন

1. Trace, span ও context-এর সংজ্ঞা দিন।
2. Auto-instrumentation বনাম manual instrumentation — কখন কোনটি?
3. Context propagation না হলে কী হয়? `traceparent` header-এর কাজ কী?
4. Span-এ attribute দেওয়ার নিয়ম কী? কী দেওয়া উচিত নয়?
5. Sampling কেন দরকার? Tail sampling বনাম head sampling-এর পার্থক্য কী?

ল্যাব 20 • Observability Lab Workbook

Lab 20 — Deploy OpenTelemetry Collector

উদ্দেশ্য

OpenTelemetry Collector স্থাপন করবেন — receiver, processor (batch, sampling, attribute) ও exporter দিয়ে একটি pipeline তৈরি করবেন। Collector কেন অ্যাপ ও backend-এর মাঝখানে রাখা হয় তা বুঝবেন।

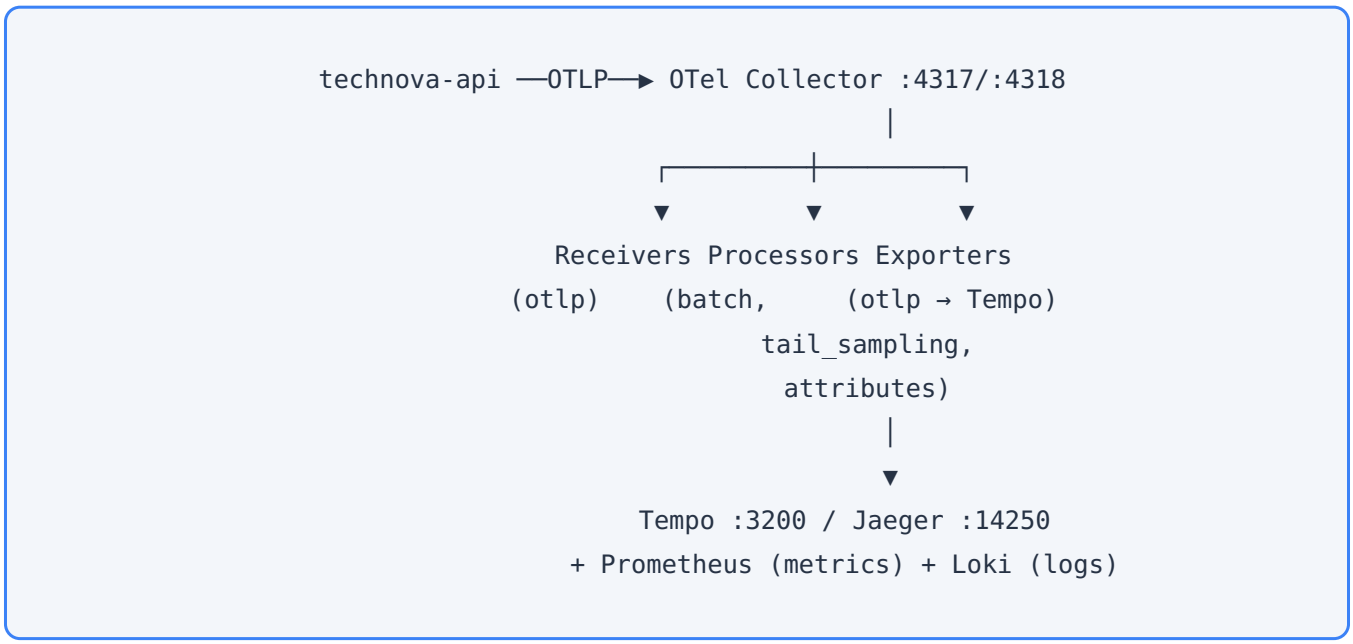
প্রয়োজনীয় টুল

- OTel Collector Contrib 0.110.x, Docker Compose
- Lab 19-এর instrumented API, Jaeger/Tempo (Lab 21)

পূর্বশর্ত

Lab 19 সম্পূর্ণ — API OTLP exporter দিয়ে span পাঠাতে পারে। YAML-এর ধারণা আছে।

আর্কিটেকচার



Collector কেন দরকার: অ্যাপ সরাসরি backend-এ পাঠালে প্রতিটি অ্যাপকে backend-এর ঠিকানা ও credential জানতে হয়। Collector মাঝখানে থাকলে অ্যাপ শুধু Collector-কে চেনে; sampling, filtering, enrichment এক জায়গায় করা যায়; backend বদলালেও অ্যাপ বদলাতে হয় না।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — Collector কনফিগারেশন লিখুন

`otel-collector/otel-collector.yml` তৈরি করুন (সম্পূর্ণ ফাইল নিচে)।

ধাপ ২ — Compose-এ যোগ করুন

```
  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.110.0
    container_name: otel-collector
    command: ["--config=/etc/otel-collector.yml"]
    volumes:
      - ./otel-collector/otel-collector.yml:/etc/otel-collector.yml:ro
    ports:
      - "4317:4317"   # OTLP gRPC
      - "4318:4318"   # OTLP HTTP
      - "8888:8888"   # Collector নিজের metrics
      - "13133:13133" # Health check
    restart: unless-stopped
```

ধাপ ৩ — চালু করে যাচাই করুন

```
docker compose up -d otel-collector
curl -s http://localhost:13133/
curl -s http://localhost:8888/metrics | grep otelcol_receiver_accepted_spans | head
```

ধাপ ৪ — API থেকে span পাঠান

```
cd ~/technova-api
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317 node server.js &
curl http://localhost:3000/api/products
docker compose logs otel-collector | tail -20
```

ধাপ ৫ — Sampling পরীক্ষা করুন

Tail sampling নীতিতে error trace সবসময় রাখা হয়, সফল trace-এর 10% রাখা হয় (নিচের কনফিগারেশনে)।

সম্পূর্ণ কনফিগারেশন

otel-collector/otel-collector.yml

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 1s
    send_batch_size: 1024
  memory_limiter:
    check_interval: 1s
    limit_mib: 512
  attributes:
    actions:
```

```
      - key: deployment.environment
        value: lab
        action: upsert
  tail_sampling:
    decision_wait: 10s
    num_traces: 10000
    policies:
      - name: keep-errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: keep-slow-traces
        type: latency
        latency: { threshold_ms: 1000 }
      - name: sample-success
        type: probabilistic
        probabilistic: { sampling_percentage: 10 }

exporters:
  otlp/tempo:
    endpoint: tempo:3200
    tls:
      insecure: true
  otlp/jaeger:
    endpoint: jaeger:4317
    tls:
      insecure: true
  logging:
    loglevel: info

extensions:
  health_check:
    endpoint: 0.0.0.0:13133

service:
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, attributes, tail_sampling, batch]
      exporters: [otlp/tempo, logging]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [logging]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [logging]
```

CONCEPT: Pipeline-এর ক্রম গুরুত্বপূর্ণ: `memory_limiter` প্রথমে (Collector-কে OOM থেকে বাঁচায়), তারপর `attributes` (enrichment), তারপর `tail_sampling` (সিদ্ধান্ত), সবশেষে `batch` (দক্ষতা)। ভুল ক্রমে sampling-এর আগে batch হলে সিদ্ধান্ত নিতে দেরি হয়।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `curl http://localhost:13133/` — `{"status":"Server available"}` (নমুনা আউটপুট — চিত্রণমূলক)।
- Collector metrics-এ `otelcol_receiver_accepted_spans` বাড়ে।
- Collector লগে `Exporting spans` বা logging exporter-এ span দেখা যায়।
- Tempo/Jaeger UI-তে trace আসে (Lab 21)।
- Error request পাঠালে সেই trace sampling-এ সবসময় থাকে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Collector চালু হচ্ছে না	YAML ভুল	`docker compose logs otel-collector` দেখুন; `otelcol validate --config` চালান
Span আসছে না	Exporter endpoint ভুল	Tempo/Jaeger সার্ভিস নাম ও পোর্ট যাচাই করুন; `logging` exporter দিয়ে span দেখুন
Collector OOM হচ্ছে	memory_limiter নেই বা সীমা বেশি	`limit_mib` কমান; batch size কমান
সব trace হারিয়ে যাচ্ছে	Tail sampling-এ decision_wait ছোট	`decision_wait: 10s` রাখুন; probabilistic হার যাচাই করুন

পরিষ্কারকরণ (Cleanup)

```
docker compose stop otel-collector
# কনফিগারেশন ফাইল রেখে দিন — Lab 21–22-এ আবার লাগবে
```

পর্যালোচনা প্রশ্ন

1. Collector ছাড়া সরাসরি backend-এ পাঠানোর অসুবিধা কী?
2. Head sampling বনাম tail sampling — কখন কোনটি?
3. Pipeline-এ processor-এর ক্রম কেন গুরুত্বপূর্ণ?
4. `memory_limiter` না থাকলে কী ঝুঁকি থাকে?
5. একটি Collector দিয়ে metrics, logs ও traces — তিনটি signal কীভাবে আলাদা pipeline-এ যায়?

ল্যাব 21 • Observability Lab Workbook

Lab 21 — Trace requests using Jaeger or Tempo

উদ্দেশ্য

Jaeger বা Tempo দিয়ে distributed trace সংরক্ষণ ও দেখা শিখবেন। একটি request-এর trace খুলে কোন span-এ সময় বেশি লেগেছে (latency bottleneck) তা নির্ণয় করবেন এবং trace ID দিয়ে লগের সাথে যুক্ত করবেন।

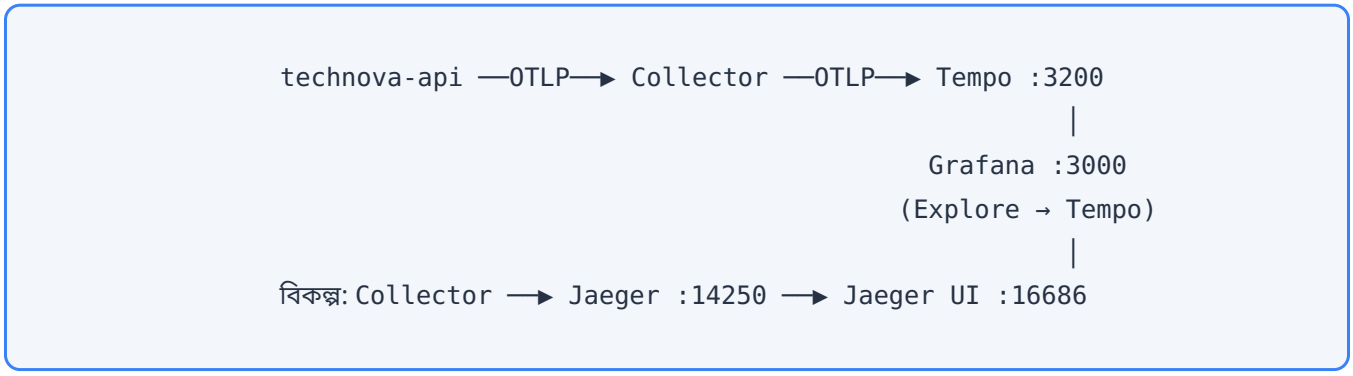
প্রয়োজনীয় টুল

- Tempo 2.x বা Jaeger 1.6x (এই ল্যাবে Tempo প্রধান, Jaeger বিকল্প)
- OTel Collector (Lab 20), instrumented API (Lab 19)

পূর্বশর্ত

Lab 19–20 সম্পূর্ণ — Collector-এ span আসছে। Grafana-তে Tempo data source যোগ করা আছে (Lab 12)।

আর্কিটেকচার



Tempo বনাম Jaeger: Tempo শুধু trace ID দিয়ে trace খোঁজে — Grafana-র সাথে গভীরভাবে যুক্ত, Loki/Prometheus-এর সাথে correlation ভালো। Jaeger নিজস্ব UI-সহ সম্পূর্ণ trace backend — service dependency graph ও trace তুলনা ভালো। ছোট দলের জন্য Tempo + Grafana দিয়ে শুরু করা সহজ।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — Tempo চালু করুন

```
tempo:
  image: grafana/tempo:2.6.0
  container_name: tempo
  command: ["-config.file=/etc/tempo.yaml"]
  volumes:
    - ./tempo/tempo.yaml:/etc/tempo.yaml:ro
    - tempo-data:/tmp/tempo
  ports:
    - "3200:3200"
```

```
      - "4317:4317"   # OTLP gRPC (সরাসরি, Collector ছাড়াও)
      - "4318:4318"   # OTLP HTTP
    restart: unless-stopped
```

tempo/tempo.yaml

```
server:
  http_listen_port: 3200
  grpc_listen_port: 9095

distributor:
  receivers:
    otlp:
      protocols:
        grpc:
          endpoint: 0.0.0.0:4317
        http:
          endpoint: 0.0.0.0:4318

ingester:
  max_block_duration: 5m

compactor:
  compaction:
    block_retention: 48h

storage:
  trace:
    backend: local
    local:
      path: /tmp/tempo/traces
    wal:
      path: /tmp/tempo/wal

metrics_generator:
  registry:
    external_labels:
      source: tempo
      cluster: technova-lab
  storage:
    path: /tmp/tempo/generator/wal
    remote_write:
      - url: http://prometheus:9090/api/v1/write
        send_exemplars: true
  traces:
    span_metrics:
      dimensions: [http.method, http.status_code, service.name]
```

WARNING: Tempo ও OTel Collector দুটোই 4317 পোর্ট ব্যবহার করতে চায়। একই Compose প্রজেক্টে দুটো চালালে পোর্ট সংঘাত হবে — Collector-কে 4317 রাখুন, Tempo-র OTLP পোর্ট হোস্টে প্রকাশ করবেন না (শুধু internal নেটওয়ার্কে), অথবা Tempo-র পোর্ট বদলান। এই ল্যাবে Collector → Tempo পথটি ব্যবহার করুন।

ধাপ ২ — Jaeger বিকল্প (ঐচ্ছিক)

```
  jaeger:
    image: jaegertracing/all-in-one:1.62.0
    container_name: jaeger
    ports:
      - "16686:16686"
      - "14250:14250"
    environment:
      - COLLECTOR_OTLP_ENABLED=true
    restart: unless-stopped
```

ধাপ ৩ — ট্রাফিক তৈরি করুন

```
docker compose up -d tempo
curl http://localhost:3000/api/products
curl -X POST http://localhost:3000/api/orders -H 'Content-Type: application/json' -d
'{"productId":"1","qty":2}'
# কয়েক সেকেন্ড অপেক্ষা করুন — Tempo-তে trace আসতে সময় লাগে
```

ধাপ ৪ — Grafana-তে trace দেখুন

Grafana → Explore → Data source: Tempo → Search → Service: `technova-api` → Run। একটি trace-এ ক্লিক করুন — span-গুলোর সময়রেখা (waterfall) দেখা যাবে।

ধাপ ৫ — Trace ID দিয়ে খুঁজুন

API-র লগ থেকে একটি `requestId`/trace ID নিন (Lab 15-এর লগে `traceId` field থাকলে) এবং Tempo-তে Trace ID দিয়ে সরাসরি খুঁজুন।

সম্পূর্ণ কনফিগারেশন

উপরের Tempo ও Jaeger Compose অংশই সম্পূর্ণ কনফিগারেশন। Grafana-তে Tempo data source (Lab 12-এ দেওয়া):

```
  - name: Tempo
    type: tempo
    access: proxy
    url: http://tempo:3200
    jsonData:
      tracesToLogsV2:
        datasourceUid: 'loki-uid'
        spanStartTimeShift: '1h'
        spanEndTimeShift: '1h'
        filterByTraceID: true
```

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- Grafana Explore → Tempo → Service `technova-api` — trace-এর তালিকা দেখা যায়।
- একটি trace খুললে HTTP span, Express middleware span ও manual `checkout.process` span দেখা যায়।

- সবচেয়ে দীর্ঘ span-টি চিহ্নিত করা যায় (সাধারণত DB বা বাইরের কল)।
- Jaeger UI (`http://localhost:16686`) — Service dropdown-এ `technova-api` দেখা যায় (Jaeger চালু থাকলে)।
- Trace-এর span-এ ক্লিক করলে `http.method`, `http.status_code` attribute দেখা যায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Tempo-তে trace আসছে না	Collector exporter endpoint ভুল	Collector কনফিগারেশনে `otlp/tempo` endpoint `tempo:3200` কি না দেখুন; Collector লগে error খুঁজুন
Port 4317 সংঘাত	Collector ও Tempo দুটোই 4317 প্রকাশ করছে	Tempo-র 4317 হোস্ট পোর্ট প্রকাশ করবেন না; শুধু Collector প্রকাশ করুন
Trace খুঁজে পাওয়া যাচ্ছে না	Sampling-এ বাদ পড়েছে	Tail sampling-এ error trace রাখা হয়; সফল trace-এর 10% রাখা হয় — কয়েকবার request পাঠান
Grafana-তে Tempo data source ব্যর্থ	URL ভুল	`http://tempo:3200` ব্যবহার করুন; `docker compose exec grafana wget -qO- http://tempo:3200/ready`

পরিষ্কারকরণ (Cleanup)

```
docker compose stop tempo jaeger
# Trace ডেটা মুছতে: docker compose down -v (সতর্কতা: সব ডেটা মুছে যাবে)
```

পর্যালোচনা প্রশ্ন

1. Tempo বনাম Jaeger — কখন কোনটি বেছে নেবেন?
2. Trace-এর waterfall ভিউ দেখে bottleneck কীভাবে চিনবেন?
3. Span Metrics কী? এটি কীভাবে Prometheus-এ যায়?
4. একটি trace-এ শুধু একটি span থাকলে কী বোঝায়?
5. Trace retention 48h — production-এ কত রাখা উচিত? কীভাবে ঠিক করবেন?

ল্যাব 22 • Observability Lab Workbook

Lab 22 — Correlate metrics, logs, and traces

উদ্দেশ্য

Metrics, logs ও traces — তিনটি signal-কে একসাথে যুক্ত করে একটি সমস্যার মূল কারণ খুঁজে বের করবেন। Grafana-তে exemplar, trace-to-logs ও log-to-trace লিংক ব্যবহার করবেন এবং একটি বাস্তব ঘটনার তদন্তের ধাপ অনুশীলন করবেন।

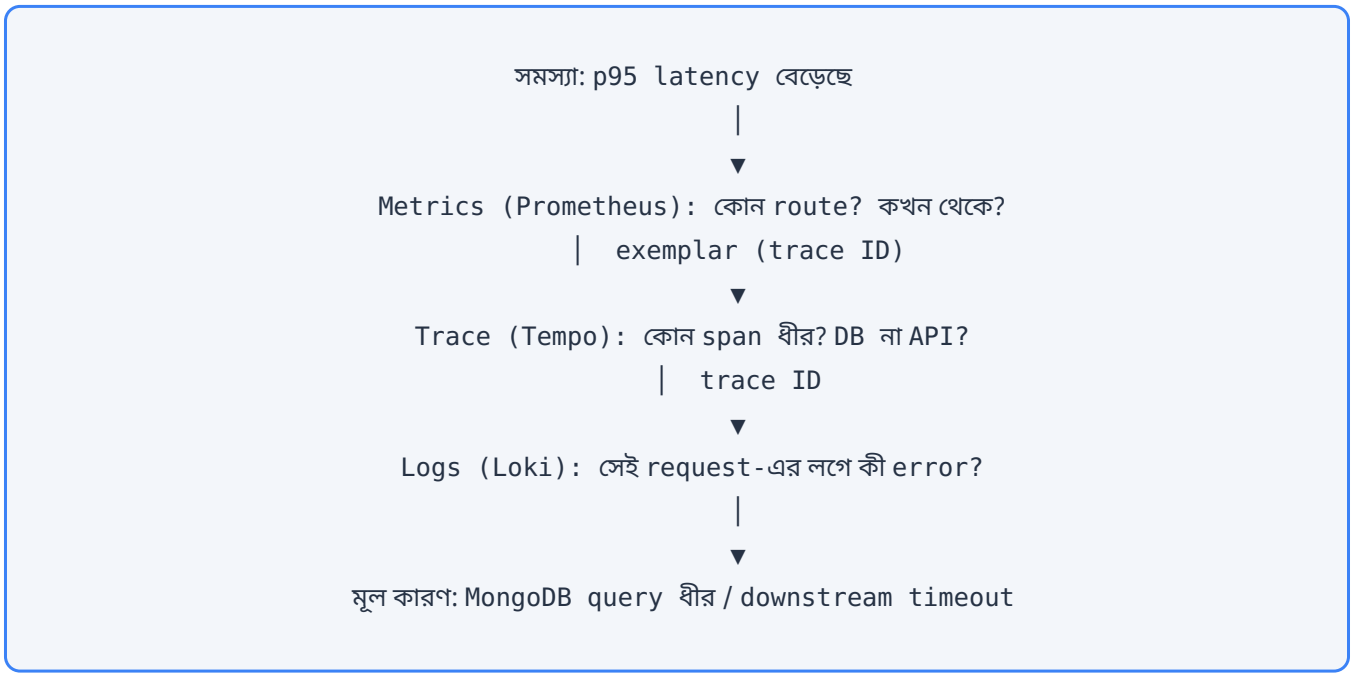
প্রয়োজনীয় টুল

- Grafana 11.x + Prometheus + Loki + Tempo (Lab 12, 16, 21)
- TechNova API (Lab 7–8, 15, 19)

পূর্বশর্ত

Lab 5 (PromQL), 17 (LogQL) ও 21 (Tempo) সম্পূর্ণ। তিনটি signal-ই চালু আছে।

আর্কিটেকচার



Exemplar: একটি metric sample-এর সাথে যুক্ত একটি trace ID। Histogram-এ exemplar থাকলে Grafana-র গ্রাফে একটি বিন্দুতে ক্লিক করলেই সেই request-এর trace খুলে যায়।

ধাপে ধাপে নির্দেশনা

ধাপ ১ — সমস্যা তৈরি করুন

API-তে একটি ধীর endpoint যোগ করুন (উদাহরণ):

```
app.get('/api/slow', async (req, res) => {
  const delay = Number(req.query.ms || 2000);
  await new Promise((r) => setTimeout(r, delay));
  res.json({ ok: true, delayMs: delay });
});
curl "http://localhost:3000/api/slow?ms=2000"
```

ধাপ ২ — Metrics দিয়ে শুরু করুন

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
# দেখুন: /api/slow route-এর p95 বেড়েছে, অন্য route স্বাভাবিক
```

ধাপ ৩ — Exemplar দিয়ে trace-এ যান

Grafana-তে p95 গ্রাফে exemplar বিন্দুতে ক্লিক করুন (exemplar চালু থাকলে) অথবা Tempo-তে Service `technova-api` খুঁজে ধীর trace-টি খুলুন।

ধাপ ৪ — Trace থেকে লগে যান

Trace ভিউতে span-এ ক্লিক → Logs লিংক (Tempo data source-এ `tracesToLogsV2` সেট থাকলে) → Loki-তে সেই trace ID-র লগ দেখা যায়। অথবা Loki-তে সরাসরি:

```
{service="technova-api"} |= "<trace-id>"
```

ধাপ ৫ — লগ থেকে trace-এ ফিরে যান

Loki লগ লাইনে trace ID থাকলে Grafana-তে তা ক্লিকযোগ্য লিংক হয় (derived field সেট থাকলে)। Loki data source-এ derived field যোগ করুন:

```
jsonData:
  derivedFields:
    - datasourceUid: 'tempo-uid'
      matcherRegex: 'traceId=([a-f0-9]{32})'
      name: 'TraceID'
      url: '$${__value.raw}'
```

ধাপ ৬ — তদন্তের সারাংশ লিখুন

একটি ছোট নোট লিখুন: কী দেখলেন (metric), কোথায় ধীর (trace), কী error (log), মূল কারণ কী।

সম্পূর্ণ কনফিগারেশন

এই ল্যাবে নতুন ফাইল নেই — Grafana data source-এর correlation সেটিংসই মূল কনফিগারেশন (উপরে দেওয়া)। API-তে trace ID লগে যোগ করতে `logger.js`-এর সাথে OTel context যুক্ত করুন:

```
const { trace } = require('@opentelemetry/api');
// request logger middleware-এ:
app.use((req, res, next) => {
  const span = trace.getActiveSpan();
  if (span) {
    const ctx = span.spanContext();
```

```
    req.log = logger.child({ traceId: ctx.traceId, spanId: ctx.spanId });
  }
  next();
});
```

> **PRO TIP:** তদন্তের ক্রম মনে রাখুন: **Metrics দিয়ে কী ও কখন → Trace দিয়ে কোথায় → Logs দিয়ে কেন**। উল্টো ক্রমে (লগ দিয়ে শুরু) অনেক সময় নষ্ট হয় — লগের পরিমাণ অনেক বেশি।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- p95 কুয়েরিতে `/api/slow` route-এর মান অন্য route-এর চেয়ে স্পষ্ট বেশি।
- Tempo-তে ধীর trace-এ `/api/slow` span-এর সময় বেশি দেখা যায়।
- সেই trace ID দিয়ে Loki-তে খুঁজলে সংশ্লিষ্ট লগ পাওয়া যায়।
- Grafana-তে trace → logs লিংক কাজ করে।
- তদন্তের নোটে তিনটি signal-এর প্রমাণ (সংখ্যা, trace, লগ লাইন) আছে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Exemplar দেখা যাচ্ছে না	Exemplar চালু নয়	Tempo metrics_generator-এ `send_exemplars: true` যাচাই করুন; Prometheus-এ exemplar storage চালু আছে কি না দেখুন
Trace → Logs লিংক কাজ করছে না	datasourceUid ভুল	Grafana-তে Loki ও Tempo data source-এর UID যাচাই করুন; `tracesToLogsV2`-এ সঠিক UID দিন
লগে traceId নেই	Logger-এ OTel context যোগ হয়নি	উপরের middleware যোগ করুন; pino-তে traceId field যাচাই করুন
Trace ID দিয়ে খুঁজে পাওয়া যাচ্ছে না	Trace sampling-এ বাদ পড়েছে বা retention পার হয়েছে	ধীর/error trace সবসময় রাখা হয় — error request পাঠিয়ে পরীক্ষা করুন

পরিষ্কারকরণ (Cleanup)

```
# /api/slow endpoint রেখে দিন — Lab 24-এর incident simulation-এ লাগবে
pkill -f "node server.js" || true
```

পর্যালোচনা প্রশ্ন

1. তদন্তে Metrics → Trace → Logs ক্রম কেন কার্যকর? উল্টো ক্রমে কী সমস্যা?
2. Exemplar কী? এটি ছাড়া trace খুঁজতে কী করতে হয়?
3. Trace ID লগে রাখা কেন গুরুত্বপূর্ণ?
4. একটি ঘটনায় তিনটি signal-এর প্রমাণ কীভাবে সাজাবেন?

5. Correlation ছাড়া শুধু লগ দিয়ে তদন্ত করলে কী কী মিস হতে পারে?

ল্যাব 23 • Observability Lab Workbook

Lab 23 — Build Kubernetes observability

উদ্দেশ্য

kind বা Minikube-এ একটি Kubernetes ক্লাস্টারে kube-prometheus-stack (Prometheus Operator, Grafana, Alertmanager) স্থাপন করবেন এবং TechNova API-কে Kubernetes-এ চালিয়ে Pod/Service monitoring, ServiceMonitor ও log সংগ্রহ করবেন।

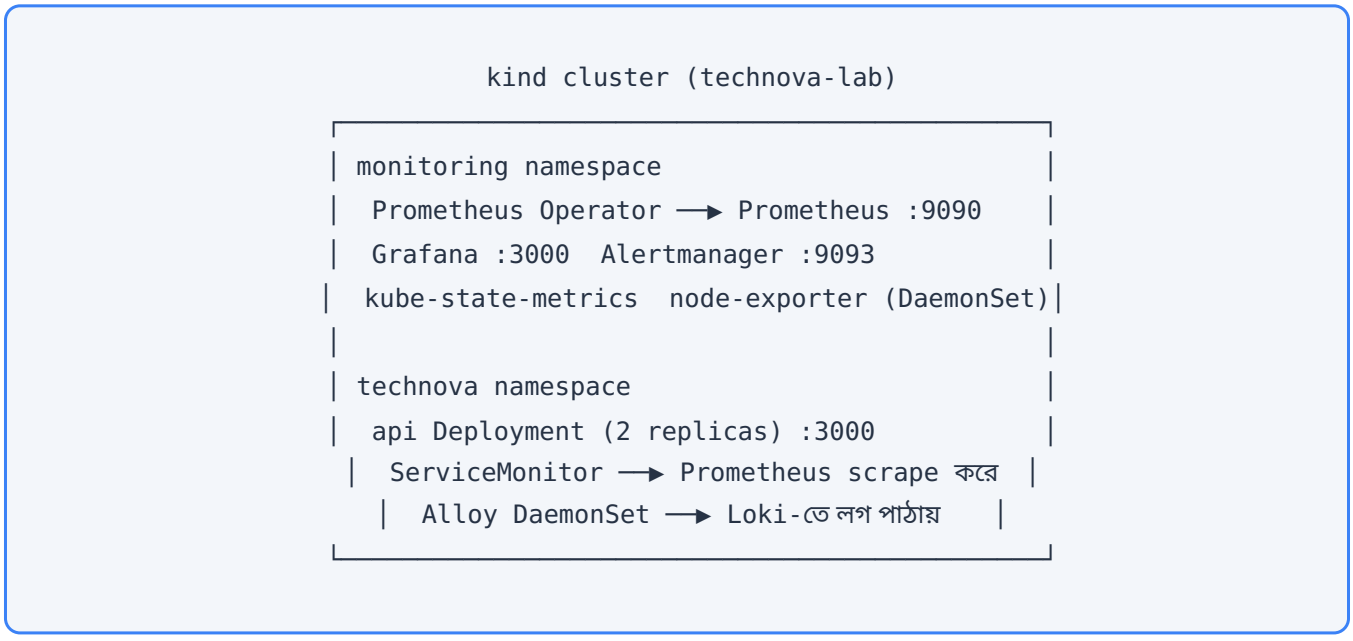
প্রয়োজনীয় টুল

- kind বা Minikube, kubectl, Helm 3.x
- 8 GB RAM, 4 CPU (Minikube/kind-এর জন্য)
- Docker (kind-এর জন্য)

পূর্বশর্ত

Lab 1–14 সম্পূর্ণ — Prometheus/Grafana/Alertmanager-এর ধারণা আছে। kubectl-এর প্রাথমিক ব্যবহার জানেন। এই ল্যাব স্থানীয় — কোনো ক্লাউড খরচ নেই।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — kind ক্লাস্টার তৈরি করুন

```
kind create cluster --name technova-lab --config kind-config.yml
kubectl cluster-info
kubectl get nodes
```

kind-config.yml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: technova-lab
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

ধাপ ২ — kube-prometheus-stack ইনস্টল করুন

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install kube-prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace \
  --set prometheus.prometheusSpec.retention=7d \
  --set
prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.resources.requests.storage=10Gi
\
  --set grafana.adminPassword=CHANGE-ME-grafana-admin
kubectl -n monitoring get pods -w
```

সব Pod `Running` হতে 2–5 মিনিট লাগতে পারে।

ধাপ ৩ — TechNova API deploy করুন

```
kubectl create namespace technova
kubectl -n technova apply -f k8s/api-deployment.yml
kubectl -n technova apply -f k8s/api-service.yml
kubectl -n technova apply -f k8s/api-servicemonitor.yml
kubectl -n technova get pods,svc
```

ধাপ ৪ — Grafana ও Prometheus UI খুলুন

```
kubectl -n monitoring port-forward svc/kube-prometheus-grafana 3000:80 &
kubectl -n monitoring port-forward svc/kube-prometheus-prometheus 9090:9090 &
# ব্রাউজার: http://localhost:3000 (admin / CHANGE-ME-grafana-admin)
# Prometheus: http://localhost:9090 → Status → Targets-এ technova-api দেখা যাবে
```

ধাপ ৫ — Pod লগ ও resource দেখুন

```
kubectl -n technova logs -l app=technova-api --tail=20
kubectl top pods -n technova
kubectl -n technova describe pod -l app=technova-api | head -40
```

সম্পূর্ণ কনফিগারেশন

k8s/api-deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-api
  namespace: technova
  labels: { app: technova-api }
spec:
  replicas: 2
  selector:
    matchLabels: { app: technova-api }
  template:
    metadata:
      labels: { app: technova-api }
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "3000"
        prometheus.io/path: "/metrics"
    spec:
      containers:
        - name: api
          image: technova-api:1.4.2
          ports:
            - containerPort: 3000
          env:
            - name: NODE_ENV
              value: "lab"
            - name: OTEL_EXPORTER_OTLP_ENDPOINT
              value: "http://otel-collector:4317"
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
            periodSeconds: 10
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
            periodSeconds: 5
```

k8s/api-service.yml

```
apiVersion: v1
kind: Service
metadata:
```

```
  name: technova-api
  namespace: technova
  labels: { app: technova-api }
spec:
  selector: { app: technova-api }
  ports:
    - name: http
      port: 3000
      targetPort: 3000
  type: ClusterIP
```

k8s/api-servicemonitor.yml

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: technova-api
  namespace: monitoring
  labels: { app: technova-api }
spec:
  selector:
    matchLabels: { app: technova-api }
  namespaceSelector:
    matchNames: [technova]
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

CONCEPT: ServiceMonitor হলো Prometheus Operator-এর CRD — Service-কে label দিয়ে চিহ্নিত করলে Operator স্বয়ংক্রিয়ভাবে scrape config তৈরি করে। `prometheus.io/scrape` annotation পুরনো পদ্ধতি; Operator-ভিত্তিক সেটআপে ServiceMonitor-ই মানদণ্ড।

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- `kubectl -n monitoring get pods` — সব Pod `Running`।
- Prometheus Targets-এ `technova-api` `UP`; `kube-state-metrics` ও `node-exporter` target দেখা যায়।
- Grafana-তে Kubernetes dashboard (ID 315, 6417) আমদানি করে Pod CPU/memory দেখা যায়।
- `kubectl top pods` — CPU/memory মান দেখায় (metrics-server থাকলে)।
- API Pod মুছলে (`kubectl delete pod`) Deployment নতুন Pod তৈরি করে এবং monitoring-এ গ্যাপ দেখা যায়।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Pod Pending	Resource কম বা image pull ব্যর্থ	`kubectl describe pod` দেখুন; kind-এ image লোড করুন: `kind load docker-image technova-api:1.4.2 --name technova-lab`

ServiceMonitor target আসছে না	Label বা namespaceSelector ভুল	Service-এর label ও ServiceMonitor-এর selector মিলিয়ে দেখুন; Prometheus-এ Service Discovery পেজ দেখুন
Grafana-তে ডেটা নেই	Prometheus data source ভুল	kube-prometheus-stack Grafana-তে data source স্বয়ংক্রিয়ভাবে যোগ করে — Connections-এ যাচাই করুন
kind ক্লাস্টার ভারী	RAM কম	Worker 1-এ কমান; অথবা Minikube-তে `--memory=4096 --cpus=2` ব্যবহার করুন

পরিষ্কারকরণ (Cleanup)

```
helm -n monitoring uninstall kube-prometheus
kubectl delete namespace technova monitoring
kind delete cluster --name technova-lab
# Minikube হলে: minikube delete
```

WARNING: `kind delete cluster` ক্লাস্টারের সব ডেটা মুছে দেয়। Lab 24-এর আগে ক্লাস্টার মুছবেন না — incident simulation এই ক্লাস্টারেই করা যাবে।

পর্যালোচনা প্রশ্ন

1. Prometheus Operator কী সুবিধা দেয়? সরাসরি Prometheus চালানোর সাথে পার্থক্য কী?
2. ServiceMonitor কী? এটি ছাড়া কীভাবে scrape করা যায়?
3. kube-state-metrics কী দেয় যা Node Exporter দেয় না?
4. Pod-এর resource request বনাম limit — পার্থক্য কী?
5. Kubernetes-এ লগ সংগ্রহের জন্য DaemonSet কেন ব্যবহার করা হয়?

ল্যাব 24 • Observability Lab Workbook

Lab 24 — Simulate and investigate a production incident

উদ্দেশ্য

একটি production incident-এর সম্পূর্ণ অনুশীলন করবেন — সমস্যা তৈরি করা (chaos), detect করা (alert), investigate করা (metrics → traces → logs), সমাধান করা (remediate) এবং post-incident report লেখা। এই ল্যাব Capstone-এর মহড়া।

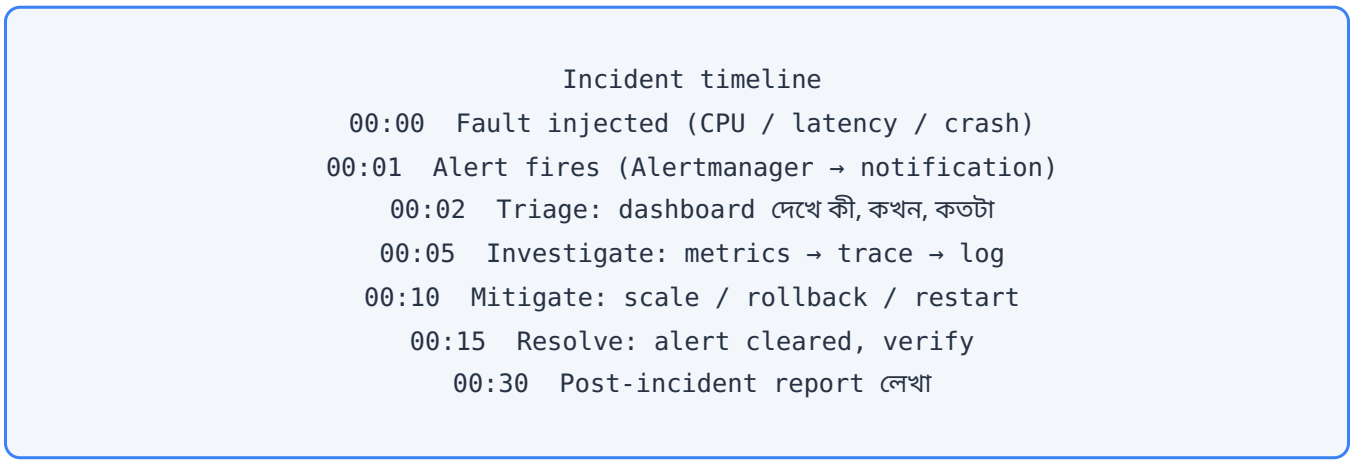
প্রয়োজনীয় টুল

- Lab 1–23-এর সম্পূর্ণ পরিবেশ (Prometheus, Grafana, Loki, Tempo, Alertmanager)
- TechNova API (Lab 7–8, 15, 19), `stress-ng`, `kubectl` (ঐচ্ছিক)

পূর্বশর্ত

সব signal চালু আছে। Alert rule-গুলো (Lab 10) ও dashboard (Lab 13) তৈরি আছে। Incident-এর ধাপগুলো (detect → investigate → correlate → resolve) জানেন।

আর্কিটেকচার



ধাপে ধাপে নির্দেশনা

ধাপ ১ — Incident পরিকল্পনা করুন

একটি দৃশ্য (scenario) বেছে নিন — উদাহরণ: **API latency বেড়েছে**। নিচের টেবিলে প্রতিটি দৃশ্যের injection পদ্ধতি দেওয়া আছে।

দৃশ্য	Injection পদ্ধতি	প্রত্যাশিত Alert
CPU saturation	`stress-ng --cpu 2 --timeout 300s`	HighCPUUsage

Memory pressure	`stress-ng --vm 1 --vm-bytes 80% --timeout 180s`	HighMemoryUsage
API latency	`curl "localhost:3000/api/slow?ms=3000"` বারবার	HighLatency
Application crash	`pkill -f "node server.js"`	InstanceDown
HTTP error rate	`/api/products/999` বারবার (404) বা failPayment পোস্ট	HighErrorRate / PaymentFailureRateHigh
Disk exhaustion	`dd if=/dev/zero of=/tmp/fill.img bs=1M count=5000` (সাবধানে!)	DiskSpaceLow

> **WARNING:** Disk exhaustion পরীক্ষা শুধু ল্যাব VM-এ করুন, কখনো production বা গুরুত্বপূর্ণ মেশিনে নয়। পরীক্ষার পর `rm /tmp/fill.img` দিয়ে ফাইল মুছে ফেলুন।

ধাপ ২ — Detect করুন

Fault চালু করুন এবং ঘড়ি দেখুন — কতক্ষণে Alertmanager-এ alert আসে? Grafana dashboard-এ কোন প্যানেল প্রথম বদলায়? সময় লিখে রাখুন।

ধাপ ৩ — Investigate করুন (Metrics → Trace → Logs)

```
# Metrics: কোন route? কখন থেকে?
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
# Trace: ধীর trace খুলুন (Tempo)
# Logs: সেই সময়ের error লগ (Loki)
{service="technova-api"} | json | status >= 500
```

ধাপ ৪ — Mitigate করুন

দৃশ্য অনুযায়ী: CPU হলে লোড বন্ধ করুন (`pkill stress-ng`); crash হলে API রিস্টার্ট করুন (`node server.js &`); latency হলে ধীর endpoint-এর কারণ খুঁজে timeout কমান; error হলে ভুল deployment rollback করুন।

ধাপ ৫ — Resolve যাচাই করুন

- Alert `resolved` হয়েছে কি?
- p95 latency স্বাভাবিক হয়েছে কি?
- Error rate 0-তে ফিরেছে কি?

ধাপ ৬ — Post-incident report লিখুন

নিচের ছকটি পূরণ করুন (Capstone-এর report-এর সংক্ষিপ্ত রূপ)।

```
Incident: API latency বেড়েছে (p95 0.05s → 3.2s)
Detected: 2026-11-09 06:10 +06, HighLatency alert
Impact: /api/slow route-এর সব request ধীর; অন্য route স্বাভাবিক
```

```
Root cause: /api/slow endpoint-এ 3s sleep (ইচ্ছাকৃত fault injection)
Timeline: 06:08 fault, 06:10 alert, 06:12 triage, 06:15 mitigate, 06:18 resolved
Action items: 1) /api/slow-এ timeout যোগ করা  2) p95 alert-এ route label যোগ করা
```

সম্পূর্ণ কনফিগারেশন

এই ল্যাবে নতুন কনফিগারেশন ফাইল নেই — Lab 10-এর alert rule ও Lab 13-এর dashboard-ই incident-এর হাতিয়ার। একটি incident runbook-এর উদাহরণ:

```
# Runbook: HighLatency
1. Grafana dashboard → কোন route-এর p95 বেড়েছে দেখুন
2. Tempo → সেই route-এর ধীর trace খুলুন, কোন span ধীর দেখুন
3. Loki → সেই সময়ের error লগ দেখুন
4. যদি DB span ধীর হয় → MongoDB slow query log দেখুন
5. যদি downstream span ধীর হয় → downstream সার্ভিসের health দেখুন
6. Mitigate: traffic সরান / scale করুন / rollback করুন
7. Resolve যাচাই করুন, তারপর post-incident report লিখুন
```

প্রত্যাশিত ফলাফল ও যাচাইকরণ

- প্রতিটি দৃশ্যে সঠিক alert নির্ধারিত সময়ে (`for` সময়সীমা + evaluation) firing হয়।
- Dashboard-এ সমস্যার শুরু ও শেষ স্পষ্ট দেখা যায়।
- Trace ও লগ দিয়ে মূল কারণ চিহ্নিত করা যায়।
- Mitigation-এর পর alert resolved হয় এবং metric স্বাভাবিক হয়।
- Post-incident report-এ timeline, impact, root cause ও action item আছে।

সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Alert আসছে না	Fault যথেষ্ট শক্তিশালী নয়	Threshold-এর চেয়ে বেশি লোড দিন; `for` সময় অপেক্ষা করুন
Trace পাওয়া যাচ্ছে না	Sampling-এ বাদ পড়েছে	Error/slow trace রাখা হয় — ধীর request-এর trace খুঁজুন
Mitigation-এর পরও alert firing	Alert-এর `for` বা resolve_timeout চলছে	5 মিনিট অপেক্ষা করুন; metric স্বাভাবিক হয়েছে কি না দেখুন
Disk পরীক্ষায় মেশিন আটকে গেছে	ডিস্ক পুরোপুরি ভরে গেছে	`rm /tmp/fill.img` করুন; প্রয়োজনে VM রিস্টার্ট করুন

পরিষ্কারকরণ (Cleanup)

```
pkill -f stress-ng || true
rm -f /tmp/fill.img
```

```
docker compose down
# kind ক্লাস্টার থাকলে: kind delete cluster --name technova-lab
```

> **WARNING:** `docker compose down -v` সব metrics, log ও trace ডেটা মুছে দেয়। Capstone-এর আগে ডেটা রাখতে চাইলে শুধু `down` (v ছাড়া) ব্যবহার করুন।

পর্যালোচনা প্রশ্ন

1. Incident-এর পাঁচটি ধাপ (detect, triage, investigate, mitigate, resolve) প্রতিটির উদ্দেশ্য কী?
2. Mitigation বনাম resolution — পার্থক্য কী?
3. Post-incident report-এ blame (দোষারোপ) কেন করা উচিত নয়?
4. একটি alert firing হলে প্রথম 5 মিনিটে কী কী করবেন?
5. Chaos experiment (fault injection) production-এ করার আগে কী কী সুরক্ষা দরকার?

অ্যাসাইনমেন্ট • Observability Lab Workbook

15 Practical Assignments

নিচের 15টি অ্যাসাইনমেন্ট একা বা দলে করে জমা দিন। প্রতিটি অ্যাসাইনমেন্টে কী করতে হবে, কী জমা দিতে হবে এবং কীভাবে মূল্যায়ন হবে তা দেওয়া আছে। সব অ্যাসাইনমেন্ট স্থানীয় পরিবেশে (Docker Compose / kind) করা যায় — কোনো ক্লাউড খরচ নেই।

Assignment 1 — Host monitoring সেটআপ

কাজ: Lab 1–2 অনুসরণ করে নিজের ল্যাপটপ/VM-এ Prometheus + Node Exporter + Grafana চালু করুন। CPU, মেমরি ও ডিস্কের তিনটি PromQL কুয়েরি লিখুন এবং ফলাফলের স্ক্রিনশটসহ একটি সংক্ষিপ্ত ব্যাখ্যা জমা দিন। **জমা:** docker-compose.yml, prometheus.yml, তিনটি কুয়েরি ও ফলাফল। **মূল্যায়ন:** সব target UP (40%), কুয়েরি সঠিক (40%), ব্যাখ্যা স্পষ্ট (20%)।

Assignment 2 — PromQL চ্যালেঞ্জ

কাজ: নিচের পাঁচটি প্রশ্নের PromQL লিখুন: (1) প্রতি instance-এর CPU %, (2) গত 1 ঘণ্টায় মোট HTTP request, (3) p99 latency, (4) ডিস্ক 80%-এর বেশি পূর্ণ এমন instance, (5) গত 24 ঘণ্টায় সবচেয়ে বেশি error-যুক্ত route। **জমা:** কুয়েরি ও প্রতিটির ফলাফল (সংখ্যা/গ্রাফ)। **মূল্যায়ন:** প্রতিটি সঠিক কুয়েরি 20%।

Assignment 3 — Custom business metrics

কাজ: TechNova API-তে (Lab 8) আরও দুটি business metric যোগ করুন — যেমন দৈনিক সক্রিয় ব্যবহারকারী (DAU-র নমুনা) বা গড় অর্ডার মূল্য। Metric-এর টাইপ বেছে নেওয়ার যুক্তি লিখুন। **জমা:** কোডের অংশ, /metrics আউটপুট, PromQL ও যুক্তি। **মূল্যায়ন:** টাইপ নির্বাচন সঠিক (40%), কোড কাজ করে (40%), যুক্তি (20%)।

Assignment 4 — Recording rules ডিজাইন

কাজ: আপনার dashboard-এর সবচেয়ে ধীর তিনটি কুয়েরি চিহ্নিত করুন এবং সেগুলোর জন্য recording rule লিখুন। Rule লোডের আগে ও পরে dashboard লোডের সময় তুলনা করুন (অনুমানসহ)। **জমা:** recording.yml, promtool যাচাইয়ের আউটপুট, তুলনা। **মূল্যায়ন:** Rule সঠিক (50%), যাচাই (25%), তুলনা (25%)।

Assignment 5 — Alert rule লেখা ও পরীক্ষা

কাজ: Lab 10-এর বাইরে আরও তিনটি alert লিখুন — একটি availability, একটি latency, একটি business metric-এর জন্য। প্রতিটি alert ইচ্ছাকৃতভাবে trigger করে (fault injection) firing ও resolved হওয়া দেখান। **জমা:** alerts.yml, প্রতিটি alert-এর firing/resolved-এর প্রমাণ। **মূল্যায়ন:** Alert সঠিক (50%), trigger প্রমাণ (30%), annotation-এ runbook (20%)।

Assignment 6 — Alertmanager routing

কাজ: Alertmanager-এ দুটি receiver (critical → webhook, warning → ভিন্ন webhook) ও একটি inhibition rule তৈরি করুন। একটি critical alert দিয়ে দেখান যে warning alert চাপা পড়ে। **জমা:** alertmanager.yml, দুটি notification-এর প্রমাণ। **মূল্যায়ন:** Routing সঠিক (50%), inhibition প্রমাণ (30%), ব্যাখ্যা (20%)।

Assignment 7 — Production dashboard

কাজ: Lab 13-এর dashboard-কে আরও উন্নত করুন — একটি variable, একটি table প্যানেল ও একটি alert-যুক্ত প্যানেল যোগ করুন। Dashboard JSON export করে জমা দিন। **জমা:** dashboard JSON, স্ক্রিনশট, প্রতিটি প্যানেলের উদ্দেশ্য। **মূল্যায়ন:** প্যানেলের মান (50%), variable কাজ করে (25%), ব্যাখ্যা (25%)।

Assignment 8 — Structured logging

কাজ: একটি ছোট Express অ্যাপে (বা TechNova API-তে) pino দিয়ে structured JSON logging চালু করুন। পাঁচটি ভিন্ন log level-এর উদাহরণ দেখান এবং একটি সংবেদনশীল field redact করুন। **জমা:** logger কোড, নমুনা লগ লাইন, redact প্রমাণ। **মূল্যায়ন:** JSON ফরম্যাট (40%), level সঠিক (30%), redact (30%)।

Assignment 9 — LogQL তদন্ত

কাজ: Loki-তে 100টির বেশি লগ লাইন তৈরি করুন (সফল, ব্যর্থ ও ধীর request মিশিয়ে)। LogQL দিয়ে উত্তর দিন: (1) error হার %, (2) সবচেয়ে ধীর 5টি request, (3) একটি নির্দিষ্ট requestId-র সম্পূর্ণ কাহিনি। **জমা:** কুয়েরি ও ফলাফল। **মূল্যায়ন:** প্রতিটি প্রশ্নের সঠিক উত্তর (প্রতিটি ~33%)।

Assignment 10 — Loki বনাম OpenSearch তুলনা

কাজ: Lab 16 ও 18-এর অভিজ্ঞতা থেকে একটি তুলনা টেবিল লিখুন — খরচ, খোঁজার ক্ষমতা, পরিচালনা, retention। আপনার দলের (5 জন, ছোট বাজেট) জন্য কোনটি সুপারিশ করবেন, যুক্তিসহ লিখুন। **জমা:** তুলনা টেবিল (কমপক্ষে 6টি মাত্রা) ও সুপারিশ। **মূল্যায়ন:** তুলনার গভীরতা (60%), সুপারিশের যুক্তি (40%)।

Assignment 11 — OpenTelemetry instrumentation

কাজ: TechNova API-তে (Lab 19) একটি নতুন endpoint-এ manual span যোগ করুন এবং একটি attribute ও একটি event যোগ করুন। Jaeger/Tempo-তে span-টি দেখান। **জমা:** কোড, trace-এর স্ক্রিনশট, span-এর attribute তালিকা। **মূল্যায়ন:** Span দেখা যায় (50%), attribute/event সঠিক (30%), ব্যাখ্যা (20%)।

Assignment 12 — Collector pipeline

কাজ: Lab 20-এর Collector কনফিগারেশনে একটি নতুন processor যোগ করুন — যেমন একটি attribute মুছে ফেলা (যেমন সংবেদনশীল header) বা একটি label যোগ করা। পরিবর্তনের আগে ও পরে span-এর attribute তুলনা করুন। **জমা:** কনফিগারেশন, আগে/পরে span-এর প্রমাণ। **মূল্যায়ন:** Processor সঠিক (50%), তুলনা (30%), ব্যাখ্যা (20%)।

Assignment 13 — Correlation তদন্ত প্রতিবেদন

কাজ: Lab 22 অনুসরণ করে একটি ধীর request-এর তদন্ত করুন এবং একটি এক পৃষ্ঠার প্রতিবেদন লিখুন — metric প্রমাণ, trace প্রমাণ, log প্রমাণ, মূল কারণ, সমাধান। **জমা:** প্রতিবেদন (timeline সহ)। **মূল্যায়ন:** তিনটি signal-এর প্রমাণ (60%), মূল কারণ সঠিক (25%), প্রতিবেদনের মান (15%)।

Assignment 14 — Kubernetes observability

কাজ: kind-এ kube-prometheus-stack চালু করে TechNova API deploy করুন (Lab 23)। ServiceMonitor দিয়ে scrape করুন এবং একটি Pod-এর CPU/memory গ্রাফসহ প্রমাণ দিন। **জমা:** YAML ফাইল, Targets পেজের প্রমাণ, গ্রাফ। **মূল্যায়ন:** Stack চালু (40%), ServiceMonitor কাজ করে (40%), গ্রাফ (20%)।

Assignment 15 — Incident simulation ও post-incident report

কাজ: Lab 24-এর একটি দৃশ্য (CPU, latency, crash, error rate — যেকোনো একটি) বেছে নিয়ে সম্পূর্ণ incident অনুশীলন করুন এবং post-incident report লিখুন (timeline, impact, root cause, action items)। **জমা:** report (Lab 24-এর ছক অনুযায়ী)। **মূল্যায়ন:** Timeline সম্পূর্ণ (30%), root cause প্রমাণসহ (40%), action items বাস্তবসম্মত (30%)।

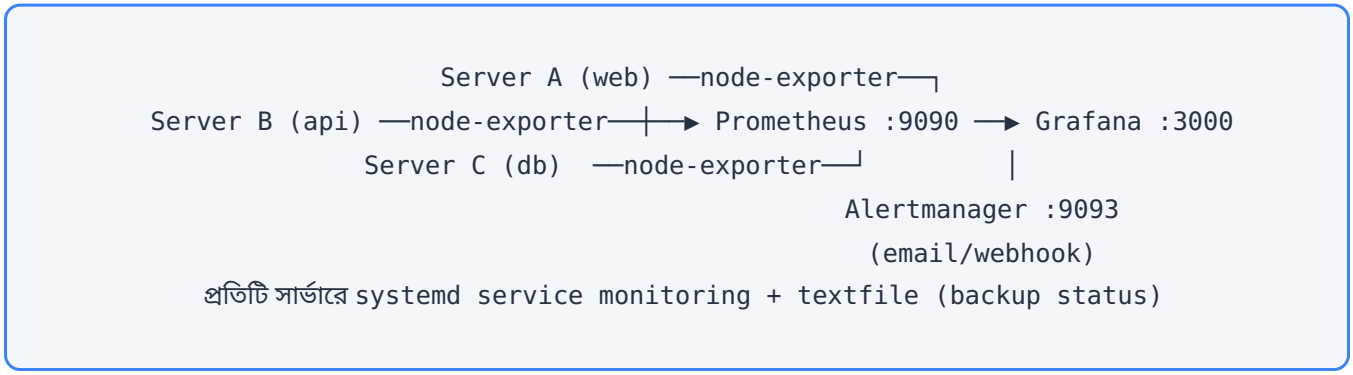
প্রজেক্ট 01 • Observability Lab Workbook

Project 01 — Linux Server Monitoring Platform

উদ্দেশ্য

একটি ছোট কোম্পানির (TechNova-র ল্যাব) তিনটি Linux সার্ভারের জন্য সম্পূর্ণ monitoring platform তৈরি করবেন — Prometheus, Node Exporter, Grafana দিয়ে host metrics, service availability, alert rules, dashboard ও operational documentation সহ। এটি Lab 1–14-এর সমন্বিত প্রয়োগ।

আর্কিটেকচার



সার্ভারগুলো Docker কনটেইনার বা kind-এর ভিন্ন Pod হিসেবে অনুকরণ করা যায় — প্রতিটি Node Exporter-এর আলাদা পোর্ট (9100, 9101, 9102) ব্যবহার করুন।

ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — পরিকল্পনা ও inventory

তিনটি সার্ভারের তালিকা লিখুন — হোস্টনেম, IP (203.0.113.11–13, উদাহরণ), ভূমিকা (web/api/db), গুরুত্বপূর্ণ সার্ভিস (nginx, node, mongod)।

ধাপ ২ — Node Exporter স্থাপন

প্রতিটি সার্ভারে Node Exporter ইনস্টল করুন (Lab 6) — systemd collector ও textfile collector (backup status) সহ।

ধাপ ৩ — Prometheus কনফিগারেশন

তিনটি সার্ভারের static target + file SD, recording rules (Lab 9) ও alert rules (Lab 10) — InstanceDown, HighCPU, HighMemory, DiskSpaceLow/Critical, ServiceDown (systemd)।

ধাপ ৪ — Grafana dashboard

Host overview dashboard (Lab 13) — প্রতি সার্ভারের CPU/memory/disk, service availability table, variable দিয়ে সার্ভার নির্বাচন।

ধাপ ৫ — Alertmanager ও documentation

Alertmanager routing (Lab 11) + operational documentation: প্রতিটি alert-এর runbook, on-call পদ্ধতি, backup ও restore পদ্ধতি।

কনফিগারেশন

```
# prometheus.yml — তিনটি সার্ভার (উদাহরণ)
scrape_configs:
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['203.0.113.11:9100', '203.0.113.12:9100', '203.0.113.13:9100']
        labels: { env: 'production', team: 'platform' }
# ল্যাবে: ['node-exporter:9100', 'node-exporter-2:9100', 'node-exporter-3:9100']
```

Alert rules, recording rules ও dashboard JSON — Lab 9, 10 ও 13-এর ফাইলগুলো এই প্রজেক্টে একত্রিত করুন।

যাচাইকরণ

- তিনটি সার্ভারের target-ই `UP`; একটি সার্ভার বন্ধ করলে `InstanceDown` alert আসে।
- Dashboard-এ তিনটি সার্ভারের metric আলাদাভাবে দেখা যায়।
- Disk 85% পূর্ণ হলে warning alert আসে (পরীক্ষার জন্য threshold সাময়িক কমিয়ে দেখান)।
- Documentation-এ প্রতিটি alert-এর জন্য করণীয় লেখা আছে।

ডেলিভারেবল

1. docker-compose.yml / ইনস্টলেশন স্ক্রিপ্ট
2. prometheus.yml, recording.yml, alerts.yml, alertmanager.yml
3. Grafana dashboard JSON + স্ক্রিনশট
4. Operational documentation (runbook, on-call, backup/restore) — Markdown, 2–4 পৃষ্ঠা
5. যাচাইকরণের প্রমাণ (Targets পেজ, alert firing-এর স্ক্রিনশট)

মূল্যায়ন

মাত্রা	ওজন	উত্তম মানের লক্ষণ
Monitoring সম্পূর্ণতা	30%	তিনটি সার্ভার, সব মূল metric, service monitoring
Alert-এর মান	25%	Actionable alert, সঠিক threshold, runbook annotation
Dashboard-এর মান	20%	স্পষ্ট, variable সহ, threshold সহ
Documentation	15%	Runbook, on-call, backup/restore স্পষ্ট
যাচাইকরণ	10%	Fault injection-এর প্রমাণ

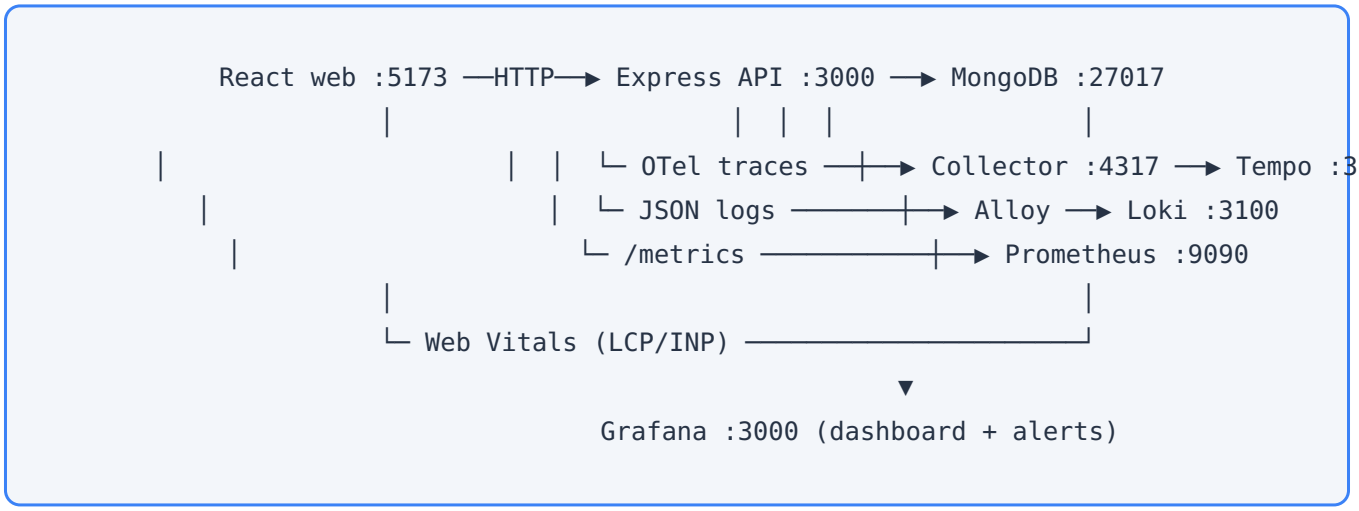
প্রজেক্ট 02 • Observability Lab Workbook

Project 02 — MERN Application Observability Platform

উদ্দেশ্য

TechNova MERN অ্যাপের (MongoDB, Express, React, Node.js) জন্য সম্পূর্ণ observability platform তৈরি করবেন — frontend performance, Node.js API metrics, structured JSON logs, OpenTelemetry traces, database visibility, request correlation, latency/error analysis, dashboard ও alerts। একটি API request কীভাবে একাধিক component অতিক্রম করে তা প্রদর্শন করবেন।

আর্কিটেকচার



ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — MERN অ্যাপ চালু করুন

MongoDB (Docker), Express API (Lab 7–8, 15, 19) ও React frontend (Vite, পোর্ট 5173) — Compose ফাইলে তিনটি সার্ভিস। Frontend-এ Web Vitals (LCP, INP, CLS) পরিমাপ করে API-তে পাঠানোর ব্যবস্থা করুন (নমুনা কোড নিচে)।

ধাপ ২ — API instrumentation

prom-client metrics (Lab 7–8), pino JSON logging (Lab 15), OTel tracing (Lab 19) — তিনটিই একসাথে। প্রতিটি request-এ একই trace ID metric exemplar, log ও trace-এ থাকবে।

ধাপ ৩ — MongoDB visibility

MongoDB Exporter যোগ করুন:

```
  mongodb-exporter:
    image: percona/mongodb_exporter:0.40.0
    container_name: mongodb-exporter
```

```
    command: ['--mongodb.uri=mongodb://mongo:27017', '--collect-all']
    ports: ["9216:9216"]
```

Prometheus-এ scrape করুন এবং `mongodb_connections`, `mongodb_opcounters_total` কুয়েরি করুন।

ধাপ ৪ — Collector, Loki, Tempo যুক্ত করুন

Lab 16, 20, 21-এর কনফিগারেশন একত্রিত করুন — Alloy দিয়ে লগ, Collector দিয়ে trace, Prometheus দিয়ে metrics।

ধাপ ৫ — একটি request-এর যাত্রা প্রদর্শন করুন

Frontend থেকে একটি অর্ডার তৈরি করুন → API-র লগে requestId → Tempo-তে trace (React → API → MongoDB span) → Prometheus-এ latency metric। এই যাত্রার স্ক্রিনশটসহ একটি এক পৃষ্ঠার বিবরণ লিখুন।

ধাপ ৬ — Dashboard ও alerts

RED dashboard (rate/error/duration), MongoDB প্যানেল, business প্যানেল (অর্ডার/পেমেন্ট) ও alert (HighErrorRate, HighLatency, PaymentFailureRateHigh, MongoDBDown)।

কনফিগারেশন

Frontend Web Vitals — নমুনা (React)

```
// web/src/vitals.js
import { onLCP, onINP, onCLS } from 'web-vitals';

function sendVital(metric) {
  fetch('/api/vitals', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name: metric.name, value: metric.value, id: metric.id }),
  }).catch(() => {});
}
onLCP(sendVital);
onINP(sendVital);
onCLS(sendVital);
```

docker-compose.yml — সম্পূর্ণ stack (সংক্ষিপ্ত)

```
services:
  mongo:
    image: mongo:7.0
    ports: ["27017:27017"]
    volumes: [mongo-data:/data/db]
  api:
    build: ./api
    ports: ["3000:3000"]
    environment:
      - MONGODB_URI=mongodb://mongo:27017/technova
      - OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
    depends_on: [mongo, otel-collector]
  web:
    build: ./web
```

```
    ports: ["5173:5173"]
  prometheus: { image: prom/prometheus:v3.0.0, ports: ["9090:9090"] }
  grafana: { image: grafana/grafana:11.4.0, ports: ["3000:3000"] }
  loki: { image: grafana/loki:3.2.0, ports: ["3100:3100"] }
  tempo: { image: grafana/tempo:2.6.0, ports: ["3200:3200"] }
  otel-collector: { image: otel/opentelemetry-collector-contrib:0.110.0, ports: ["4317:4317",
"4318:4318"] }
  alloy: { image: grafana/alloy:v1.3.0, ports: ["12345:12345"] }
```

WARNING: উপরের সংক্ষিপ্ত Compose-এ volume ও কনফিগারেশন মাউন্ট দেখানো হয়নি — Lab 1, 16, 20, 21-এর সম্পূর্ণ কনফিগারেশন ব্যবহার করুন। API ও Grafana দুটোই 3000 ব্যবহার করে — এই প্রজেক্টে Grafana-কে হোস্টে 3000 এবং API-কে কনটেইনারে 3000 রাখুন, হোস্টে API-র পোর্ট 3001-এ প্রকাশ করুন অথবা আলাদা Compose প্রজেক্ট ব্যবহার করুন।

যাচাইকরণ

- একটি অর্ডার request-এর trace Tempo-তে দেখা যায় — API span + MongoDB span।
- সেই trace ID দিয়ে Loki-তে লগ পাওয়া যায়।
- Prometheus-এ `http_requests_total`, `technova_orders_total` ও MongoDB metric দেখা যায়।
- Frontend Web Vitals API-তে পৌঁছায় (লগে দেখা যায়)।
- Latency ও error alert trigger করে দেখানো যায়।

ডেলিভারেবল

1. সম্পূর্ণ docker-compose.yml + সব কনফিগারেশন ফাইল
2. API কোড (metrics, logging, tracing)
3. Grafana dashboard JSON + স্ক্রিনশট
4. একটি request-এর যাত্রার বিবরণ (এক পৃষ্ঠা, স্ক্রিনশটসহ)
5. Alert rules ও যাচাইকরণের প্রমাণ
6. Latency/error বিশ্লেষণ — p50/p95/p99 ও error হারের ব্যাখ্যা

মূল্যায়ন

মাত্রা	ওজন	উত্তম মানের লক্ষণ
তিনটি signal-এর সমন্বয়	30%	Metrics + logs + traces একসাথে, correlation কাজ করে
Request যাত্রা প্রদর্শন	20%	একটি request frontend → API → MongoDB পর্যন্ত দেখানো যায়
Database visibility	15%	MongoDB metric ও slow query ধারণা
Dashboard ও alerts	20%	RED + business প্যানেল, actionable alert
বিশ্লেষণ	15%	Latency/error বিশ্লেষণ সংখ্যাসহ

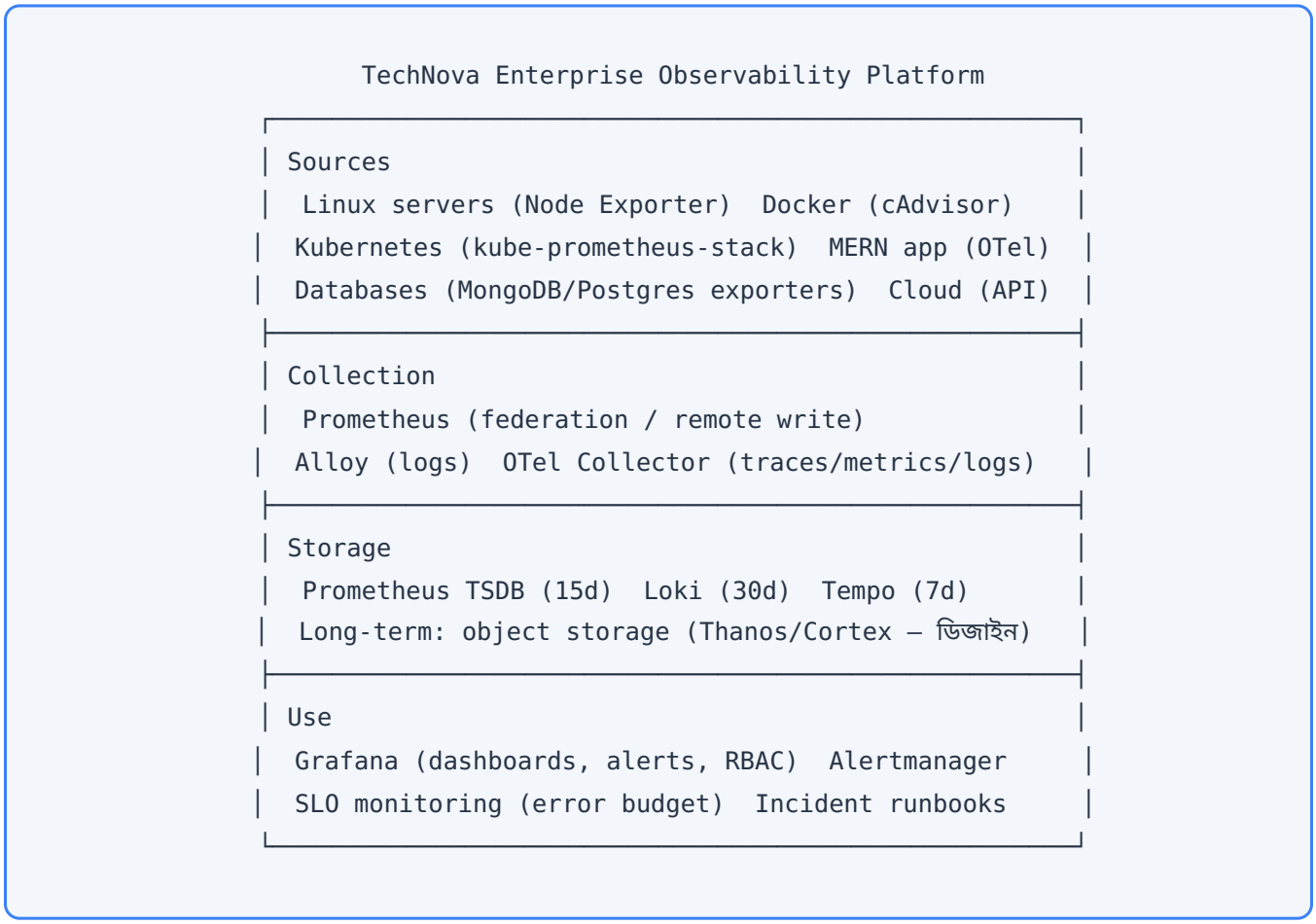
প্রজেক্ট 03 • Observability Lab Workbook

Project 03 — TechNova Enterprise Observability Platform

উদ্দেশ্য

TechNova-র enterprise-স্তরের observability platform ডিজাইন ও আংশিক বাস্তবায়ন করবেন — Linux সার্ভার, Docker, Kubernetes, distributed অ্যাপ, ডেটাবেস ও ক্লাউড অবকাঠামো; metrics, logs, traces, dashboard, alerts, SLO monitoring, retention, access controls, cost management ও incident investigation সহ। এটি বইয়ের সবচেয়ে বড় প্রজেক্ট — ডিজাইন ডকুমেন্ট ও বাস্তবায়নের প্রমাণ দুটোই জমা দিতে হবে।

আর্কিটেকচার



ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — প্রয়োজন ও SLO নির্ধারণ করুন

তিনটি সার্ভিসের (web, api, db) SLI/SLO লিখুন — যেমন API availability 99.9%, p95 latency < 500ms। Error budget হিসাব করুন (Lab: Capstone-এর সূত্র দেখুন)।

ধাপ ২ — Kubernetes observability (Lab 23)

kind-এ kube-prometheus-stack + TechNova API + ServiceMonitor + Alloy DaemonSet।

ধাপ ৩ — Docker ও হোস্ট monitoring (Lab 1–6)

cAdvisor যোগ করুন (Docker container metrics) ও Node Exporter-এর সাথে একত্রিত করুন।

ধাপ ৪ — Centralized logging ও tracing (Lab 16, 20, 21)

Loki + Tempo + Collector — সব namespace/service-এর লগ ও trace এক জায়গায়।

ধাপ ৫ — SLO monitoring ও alerting

Recording rules দিয়ে error budget, Grafana-তে SLO dashboard, burn-rate alert (দ্রুত ও ধীর burn)।

ধাপ ৬ — Access controls, retention ও cost

Grafana-তে role (Viewer/Editor/Admin) ও folder permission; retention নীতি (metrics 15d, logs 30d, traces 7d) ও খরচের অনুমান — স্থানীয় ল্যাবে খরচ শূন্য, কিন্তু production-এ storage ও cardinality-র খরচ হিসাব করে একটি টেবিল লিখুন।

ধাপ ৭ — Incident investigation অনুশীলন

Lab 24-এর দুটি দৃশ্য (যেমন downstream failure ও DB connectivity failure) এই platform-এ অনুশীলন করুন।

কনফিগারেশন

cAdvisor (Docker container metrics)

```
  cadvisor:
    image: gcr.io/cadvisor/cadvisor:v0.49.1
    container_name: cadvisor
    ports: ["8080:8080"]
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
      - /dev/disk/:/dev/disk:ro
    restart: unless-stopped
```

SLO recording rules — উদাহরণ (API availability 99.9%)

```
groups:
  - name: slo-rules
    interval: 1m
    rules:
      - record: job:slo_availability:ratio30d
        expr: |
          sum(rate(http_requests_total{status!~"5.."}[30d])) by (job)
          /
          sum(rate(http_requests_total[30d])) by (job)
      - record: job:slo_error_budget_remaining:ratio
```

```
        expr: |
          (job:slo_availability:ratio30d - 0.999) / (1 - 0.999)
```

Burn-rate alert-এর উদাহরণ Capstone-এ দেওয়া আছে।

যাচাইকরণ

- Kubernetes, Docker ও হোস্ট — তিনটি স্তরের metric এক Grafana-তে দেখা যায়।
- SLO dashboard-এ availability ও error budget দেখা যায়।
- একটি incident-এর তদন্ত metrics → traces → logs দিয়ে করা যায়।
- Retention নীতি ও access control ডকুমেন্টে লেখা আছে।
- খরচের অনুমান টেবিলে আছে (storage, cardinality, retention)।

ডেলিভারেবল

1. Architecture ডিজাইন ডকুমেন্ট (5–8 পৃষ্ঠা, diagram সহ)
2. বাস্তবায়ন: Compose + Kubernetes YAML, Prometheus/Alloy/Collector কনফিগারেশন
3. SLO সংজ্ঞা ও error budget হিসাব
4. Grafana dashboard ও alert rules
5. Retention, access control ও cost management নীতি
6. দুটি incident investigation report

মূল্যায়ন

মাত্রা	ওজন	উত্তম মানের লক্ষণ
Architecture ডিজাইন	25%	সব স্তর, স্পষ্ট diagram, যুক্তিসহ সিদ্ধান্ত
বাস্তবায়ন	25%	K8s + Docker + হোস্ট একসাথে কাজ করে
SLO ও error budget	20%	SLI/SLO সঠিক, হিসাব যাচাইযোগ্য
নীতি (retention/access/cost)	15%	বাস্তবসম্মত, সংখ্যাসহ
Incident investigation	15%	তিনটি signal-এর প্রমাণসহ report

ক্যাপস্টোন • Observability Lab Workbook

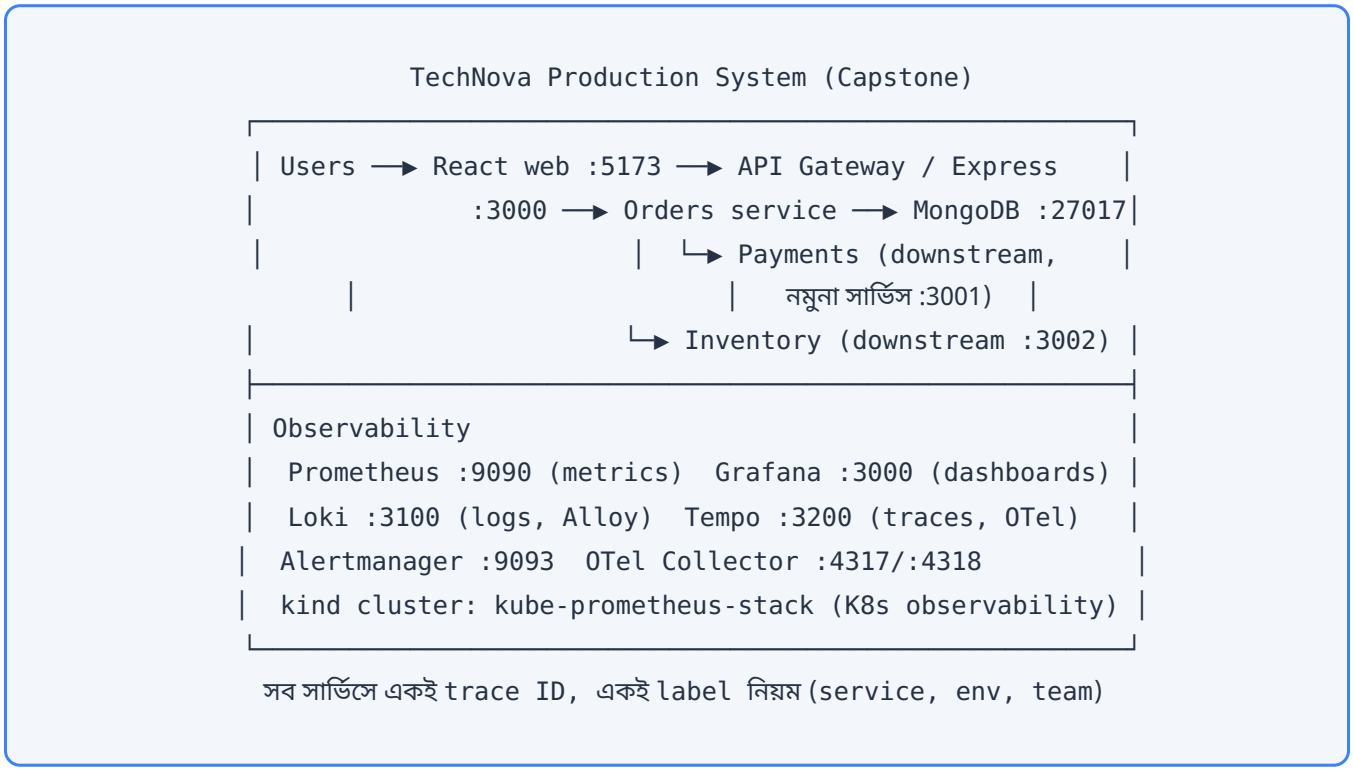
Capstone — TechNova Production Observability and Incident Response Platform

উদ্দেশ্য

এই ক্যাপস্টোনে আপনি TechNova-র জন্য একটি production-মানের distributed অ্যাপ ও তার observability system তৈরি করবেন — এবং আটটি ভিন্ন failure simulate করে প্রতিটি detect, investigate, correlate ও resolve করবেন। শেষে SLI/SLO, error budget, incident simulation, root cause analysis, remediation plan, post-incident report ও production runbook জমা দেবেন। এটি বইয়ের সব ল্যাবের সমন্বিত মূল্যায়ন।

সময়: 2–3 সপ্তাহ (দলে বা একা)। **পরিবেশ:** স্থানীয় — Docker Compose + kind/Minikube; কোনো ক্লাউড খরচ নেই। **দল:** একা বা 2–3 জন।

আর্কিটেকচার



ধাপে ধাপে বাস্তবায়ন

ধাপ ১ — Application architecture

Distributed অ্যাপ তৈরি করুন: React frontend, Express API, Orders service, Payments downstream (নমুনা), Inventory downstream (নমুনা), MongoDB। প্রতিটি সার্ভিসের জন্য README লিখুন — দায়িত্ব, API, নির্ভরতা (dependency)। Service map (কোন সার্ভিস কাকে ডাকে) diagram আকারে দিন।

ধাপ ২ — Instrumentation strategy

একটি এক পৃষ্ঠার strategy ডকুমেন্ট লিখুন: প্রতিটি সার্ভিসে কী metric (RED), কী log (structured JSON, কোন field), কী trace (auto + manual span, কোন attribute), sampling নীতি (head/tail), cardinality সীমা।

ধাপ ৩ — Prometheus metrics

Lab 7–9 অনুসরণ করে সব সার্ভিসে `/metrics`, recording rules ও ServiceMonitor (K8s-এ) তৈরি করুন।

ধাপ ৪ — Grafana dashboards

চারটি dashboard: (1) Executive overview (availability, error rate, p95), (2) Service RED, (3) Infrastructure (host/K8s), (4) Business (অর্ডার, পেমেন্ট, revenue-র নমুনা)।

ধাপ ৫ — Centralized logging

Alloy + Loki (Lab 16) — সব সার্ভিসের লগ এক জায়গায়, trace ID সহ। LogQL-এর জন্য চিট-শিট ডকুমেন্ট করুন।

ধাপ ৬ — OpenTelemetry tracing

Collector + Tempo (Lab 20–21) — প্রতিটি request-এর trace frontend থেকে MongoDB পর্যন্ত।

ধাপ ৭ — Alert rules

Lab 10–11-এর alert + burn-rate alert (নিচে)। প্রতিটি alert-এ runbook_url annotation।

ধাপ ৮ — SLI/SLO সংজ্ঞা ও error budget হিসাব

নিচের অংশে সূত্র ও উদাহরণ দেওয়া আছে — নিজের সার্ভিসের জন্য হিসাব করে জমা দিন।

ধাপ ৯ — Incident simulation (আটটি failure)

নিচের আটটি failure প্রতিটি আলাদাভাবে simulate করুন — প্রতিটির জন্য detect/investigate/correlate/resolve লিখুন।

ধাপ ১০ — Root cause analysis, remediation plan, post-incident report ও runbook

প্রতিটি incident-এর জন্য একটি post-incident report (নিচের ছক) ও একটি runbook লিখুন।

SLI/SLO সংজ্ঞা ও Error Budget হিসাব

SLI (Service Level Indicator): যা পরিমাপ করা হয় — যেমন সফল request-এর অনুপাত। **SLO (Service Level Objective):** লক্ষ্য — যেমন 99.9% availability, 30 দিনে। **Error budget:** অনুমোদিত ব্যর্থতার পরিমাণ — `(1 - SLO)` × মোট `request`।

সার্ভিস	SLI	SLO (30 দিন)	Error budget
API availability	সফল (non-5xx) request / মোট request	99.9%	0.1% — 1,000,000 request-এ 1,000টি ব্যর্থতা; সময়ে 43.2 মিনিট downtime
API latency		95%	5% request 500ms-এর বেশি হতে পারে

	p95 latency < 500ms এমন request-এর অনুপাত		
Checkout success	সফল checkout / মোট checkout চেষ্টা	99.5%	0.5% ব্যর্থতা অনুমোদিত

হিসাবের উদাহরণ: 30 দিন = 43,200 মিনিট। 99.9% SLO-তে অনুমোদিত downtime = 43,200 × 0.001 = 43.2 মিনিট। যদি ইতিমধ্যে 30 মিনিট downtime হয়ে থাকে, তবে error budget-এর অবশিষ্ট = 13.2 মিনিট — নতুন risky deployment স্থগিত করা উচিত।

Burn-rate alert (উদাহরণ):

```
# দ্রুত burn: 1 ঘণ্টায় error budget-এর 14.4 গুণ হারে খরচ হচ্ছে
- alert: ErrorBudgetBurnFast
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[1h])) by (job)
    /
    sum(rate(http_requests_total[1h])) by (job) > 0.0144
  for: 5m
  labels: { severity: critical }
  annotations:
    summary: "Fast error budget burn on {{ $labels.job }}"
    description: "Error budget 1 ঘণ্টায় 14.4x হারে খরচ হচ্ছে — এখনই ব্যবস্থা নিন।"
```

Incident Simulation — আটটি Failure

প্রতিটি failure-র জন্য চারটি ধাপ লিখুন: **Detect** (কোন alert/dashboard), **Investigate** (metrics → traces → logs), **Correlate** (তিনটি signal কীভাবে মিলল), **Resolve** (mitigation + যাচাই)।

Failure 1 — CPU saturation

Inject: `stress-ng --cpu 2 --timeout 300s` API হোস্টে। **Detect:** HighCPUUsage alert, CPU % dashboard 90%+। **Investigate:** `job:node_cpu_usage:ratio` বেড়েছে; trace-এ সব span ধীর (CPU-র কারণে); লগে timeout error। **Correlate:** CPU বৃদ্ধির সময়ের সাথে p95 বৃদ্ধির সময় মিলে যায়। **Resolve:** লোড বন্ধ করুন / scale করুন; CPU স্বাভাবিক হলে alert resolved।

Failure 2 — Memory pressure

Inject: `stress-ng --vm 1 --vm-bytes 80% --timeout 180s`। **Detect:** HighMemoryUsage alert। **Investigate:** মেমরি % 90%+, `node_memory_MemAvailable_bytes` কমেছে; API-র `technova_process_resident_memory_bytes` বেড়েছে; OOMKilled আছে কি না `kubectl get pods` দেখুন। **Correlate:** মেমরি বৃদ্ধি ও GC সময় (trace-এ) একসাথে বেড়েছে। **Resolve:** লোড বন্ধ করুন; প্রয়োজনে memory limit বাড়ান / leak খুঁজুন।

Failure 3 — Disk exhaustion

Inject: `dd if=/dev/zero of=/tmp/fill.img bs=1M count=5000` (ল্যাবে, সাবধানে)। **Detect:** DiskSpaceLow/ Critical alert। **Investigate:** `node_filesystem_avail_bytes` কমেছে; লগে `no space left on device`; Loki/

Prometheus লেখা বন্ধ হতে পারে। **Correlate:** ডিস্ক পূর্ণ হওয়ার সময়ের সাথে log ingestion বন্ধের সময় মেলে। **Resolve:** `rm /tmp/fill.img`; পুরনো লগ/চাঙ্ক মুছুন; retention কমান।

Failure 4 — API latency

Inject: `/api/slow?ms=3000` বারবার কল করুন। **Detect:** HighLatency alert (p95 > 1s)। **Investigate:** p95 কুয়েরিতে `/api/slow` route চিহ্নিত করুন; Tempo-তে ধীর trace-এ কোন span ধীর দেখুন; লগে ধীর request-এর durationMs দেখুন। **Correlate:** একই trace ID metric, trace ও লগে মেলে। **Resolve:** ধীর endpoint-এ timeout/cache যোগ করুন; লোড বন্ধ করুন।

Failure 5 — Application crash

Inject: `pkill -f "node server.js"` (বা K8s-এ Pod মুছুন)। **Detect:** InstanceDown alert, uptime প্যানেলে গ্যাপ। **Investigate:** `up == 0`; লগে শেষ লাইন (crash-এর আগে কী error); K8s-এ `kubectl get pods`-এ CrashLoopBackOff। **Correlate:** Crash-এর সময়ের সাথে error লগের শেষ লাইন মেলে। **Resolve:** API রিস্টার্ট করুন; crash-এর কারণ (যেমন ভুল env var) ঠিক করুন।

Failure 6 — Database connectivity failure

Inject: MongoDB কনটেইনার বন্ধ করুন (`docker compose stop mongo`)। **Detect:** MongoDBDown / HighErrorRate alert। **Investigate:** API লগে `MongoNetworkError` / connection timeout; trace-এ MongoDB span error; Prometheus-এ `mongodb_connections` গ্যাপ। **Correlate:** DB বন্ধের সময়ের সাথে API error বৃদ্ধির সময় মেলে। **Resolve:** MongoDB চালু করুন; connection retry ও timeout যাচাই করুন; প্রয়োজনে circuit breaker যোগ করুন।

Failure 7 — Increased HTTP error rate

Inject: ভুল deployment (যেমন 500 ফেরত দেওয়া সংস্করণ) বা `/api/products/999` বারবার কল করুন। **Detect:** HighErrorRate alert (5xx > 5%)। **Investigate:** Error rate কুয়েরিতে কোন route/status বেড়েছে; Tempo-তে error trace-এ exception দেখুন; Loki-তে `status >= 500` লগ দেখুন। **Correlate:** Deployment-এর সময়ের সাথে error বৃদ্ধির সময় মেলে। **Resolve:** আগের সংস্করণে rollback করুন; ভুল কোড ঠিক করুন।

Failure 8 — Downstream service failure

Inject: Payments downstream সার্ভিস (:3001) বন্ধ করুন। **Detect:** PaymentFailureRateHigh alert, checkout error বেড়েছে। **Investigate:** Trace-এ payments span error/timeout; API লগে `payment gateway timeout`; downstream-এর নিজের health endpoint DOWN। **Correlate:** Checkout ব্যর্থতা শুধু payment ধাপে — অন্য ধাপ স্বাভাবিক। **Resolve:** Downstream চালু করুন; timeout ও retry নীতি যাচাই করুন; প্রয়োজনে degraded mode (পরে পেমেন্ট) চালু করুন।

Post-Incident Report ছক (প্রতিটি incident-এর জন্য)

```
Incident ID: INC-2026-11-09-01
Title:
Detected (সময়, কীভাবে):
Impact (ব্যবহারকারী/সার্ভিস, সময়কাল):
Timeline (ঘটনার ক্রম, সময়সহ):
Root cause:
```

```
Mitigation (তাৎক্ষণিক সমাধান):
Resolution (স্থায়ী সমাধান):
Action items (দায়িত্ব + সময়সীমাসহ):
Lessons learned:
```

Production Runbook (উদাহরণ — প্রতিটি alert-এর জন্য একটি)

প্রতিটি runbook-এ থাকবে: লক্ষণ, নির্ণয়ের ধাপ (কুয়েরি সহ), mitigation, escalation (কখন কাকে ডাকবেন), যাচাই। Lab 24-এর runbook উদাহরণ দিয়ে শুরু করুন এবং আটটি failure-র জন্য আটটি runbook লিখুন।

Acceptance Criteria

- Distributed অ্যাপের সব সার্ভিস চালু এবং একে অপরের সাথে যুক্ত।
- Metrics, logs ও traces — তিনটি signal-ই Grafana-তে একসাথে দেখা যায় এবং correlation (trace ID) কাজ করে।
- চারটি dashboard তৈরি ও কার্যকর।
- SLI/SLO সংজ্ঞায়িত ও error budget হিসাব করা হয়েছে (সংখ্যাসহ)।
- আটটি failure-র প্রতিটি simulate করা হয়েছে এবং প্রতিটির detect/investigate/correlate/resolve লেখা হয়েছে।
- প্রতিটি incident-এর post-incident report ও runbook আছে।
- কোনো আসল secret উদাহরণে নেই; সব কনফিগারেশন `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করে।
- সব ফাইল Git repository-তে সংরক্ষিত (commit ইতিহাসসহ)।

Grading Rubric

মাত্রা	ওজন	Excellent (90–100%)	Good (70–89%)	Needs work (<70%)
Application architecture	10%	সব সার্ভিস, স্পষ্ট diagram, dependency নথিভুক্ত	বেশিরভাগ সার্ভিস, diagram আংশিক	অ্যাপ অসম্পূর্ণ বা diagram নেই
Instrumentation (metrics/logs/traces)	20%	তিনটি signal, correlation কাজ করে	দুটি signal ভালো, একটি আংশিক	একটি signal-ও ঠিকমতো কাজ করে না
Dashboards ও alerts	15%	চারটি dashboard, actionable alert, runbook	দুটি–তিনটি dashboard, alert আংশিক	Dashboard/alert অসম্পূর্ণ
SLI/SLO ও error budget	15%	সঠিক সংজ্ঞা, যাচাইযোগ্য হিসাব	সংজ্ঞা আছে, হিসাব আংশিক	SLO নেই বা হিসাব ভুল
Incident simulation (8টি)	25%	আটটিই প্রমাণসহ, চার ধাপ স্পষ্ট	5–7টি, প্রমাণ আংশিক	4টির কম
Report ও runbook	15%	সব report সম্পূর্ণ, runbook কার্যকর	বেশিরভাগ report আছে	Report/runbook অনুপস্থিত

পাস মার্ক: মোট 60% এবং incident simulation-এ কমপক্ষে 50%।

ক্যাপস্টোন সমাধান নির্দেশিকা • Observability Lab Workbook

Capstone Solution Guide — Instructor Reference

এই নির্দেশিকা শিক্ষকের জন্য — শিক্ষার্থীর ক্যাপস্টোন মূল্যায়নের রেফারেন্স। এটি সম্পূর্ণ সমাধান নয়, বরং প্রত্যাশিত কাঠামো, মূল কনফিগারেশন ও প্রত্যাশিত ফলাফলের রূপরেখা। শিক্ষার্থীকে সরাসরি এই ফাইল দিয়ে দেবেন না — মূল্যায়নের সময় রেফারেন্স হিসেবে ব্যবহার করুন।

উদ্দেশ্য

শিক্ষক যাতে দ্রুত যাচাই করতে পারেন: শিক্ষার্থীর architecture সম্পূর্ণ কি না, তিনটি signal যুক্ত কি না, আটটি failure-র প্রতিটি সঠিকভাবে তদন্ত করা হয়েছে কি না, এবং SLO/error budget হিসাব সঠিক কি না।

আর্কিটেকচার (প্রত্যাশিত)

React frontend → Express API → Orders service → MongoDB; Payments ও Inventory downstream (নমুনা সার্ভিস)। প্রতিটি সার্ভিসে prom-client metrics, pino JSON logging, OTel tracing। Collector → Tempo/Prometheus/Loki। Grafana-তে চারটি dashboard। kind-এ kube-prometheus-stack (ঐচ্ছিক কিন্তু উৎসাহিত)।

ধাপে ধাপে বাস্তবায়ন (প্রত্যাশিত ক্রম)

1. Compose stack চালু (mongo, api, web, prometheus, grafana, loki, tempo, collector, alloy)।
2. API-তে metrics/logging/tracing যাচাই (`/metrics`, Loki-তে লগ, Tempo-তে trace)।
3. Dashboard তৈরি (overview, RED, infra, business)।
4. Alert rules + Alertmanager যাচাই (fault injection দিয়ে)।
5. SLO সংজ্ঞা ও error budget হিসাব (99.9% → 43.2 মিনিট/মাস)।
6. আটটি failure একে একে simulate ও নথিভুক্ত করা।
7. Post-incident report ও runbook লেখা।

কনফিগারেশন (মূল অংশ)

Prometheus — মূল scrape jobs: `prometheus`, `node-exporter`, `technova-api`, `mongodb-exporter`, `cadvisor`। **Recording rules:** `job:node_cpu_usage:ratio`, `job:http_errors:ratio5m`, `job:http_latency_p95:5m`, `job:slo_availability:ratio30d`। **Alert rules:** InstanceDown, HighCPUUsage, HighMemoryUsage, DiskSpaceLow/Critical, HighErrorRate, HighLatency, PaymentFailureRateHigh, ErrorBudgetBurnFast। **Collector:** OTLP receiver, batch + tail_sampling processor, Tempo exporter। **Alloy:** Docker logs → JSON parse → Loki। সম্পূর্ণ ফাইল Lab 9–11, 16, 20-এর অনুরূপ হওয়া উচিত।

```
# প্রত্যাশিত যাচাই কমান্ড (শিক্ষকের জন্য)
curl -s http://localhost:9090/api/v1/targets | grep -c '"health":"up"'
curl -s 'http://localhost:9090/api/v1/query?query=up' | grep -c '"value":\[.*"1"'
curl -s http://localhost:3100/ready
```

```
curl -s http://localhost:3200/ready
curl -s http://localhost:9093/-/healthy
```

যাচাইকরণ (প্রত্যাশিত ফলাফল)

- সব target `UP`; Grafana-তে চারটি dashboard-এ বাস্তব ডেটা।
- একটি অর্ডার request-এর trace Tempo-তে দেখা যায় — API + MongoDB span।
- সেই trace ID দিয়ে Loki-তে লগ পাওয়া যায়।
- আটটি failure-র প্রতিটির জন্য alert firing-এর প্রমাণ (স্ক্রিনশট বা API আউটপুট) ও resolved-এর প্রমাণ আছে।
- Error budget হিসাব: 99.9% SLO-তে 43.2 মিনিট/মাস — শিক্ষার্থীর হিসাবে এই সংখ্যা থাকা উচিত।
- Post-incident report-এ timeline, impact, root cause ও action item আছে; কোনো report-এ দোষারোপ নেই।

ডেলিভারেবল (প্রত্যাশিত তালিকা)

1. Architecture ডকুমেন্ট (diagram সহ)
2. সব কনফিগারেশন ফাইল (Git repository)
3. চারটি dashboard JSON
4. SLO ডকুমেন্ট ও error budget হিসাব
5. আটটি post-incident report
6. আটটি runbook
7. একটি request-এর যাত্রার বিবরণ

মূল্যায়ন (শিক্ষকের জন্য নির্দেশনা)

Capstone-এর rubric (উপরের অংশ) ব্যবহার করুন। Incident simulation-এ প্রতিটি failure-র জন্য চারটি ধাপ (detect/investigate/correlate/resolve) আলাদাভাবে নম্বর দিন — শুধু "সমস্যা ঠিক হয়েছে" বললেই পূর্ণ নম্বর নয়, প্রমাণ (metric সংখ্যা, trace ID, লগ লাইন) থাকতে হবে। Error budget হিসাবে 43.2 মিনিটের সংখ্যা যাচাই করুন। Secret (আসল পাসওয়ার্ড/API key) থাকলে নম্বর কাটুন এবং শিক্ষার্থীকে `CHANGE-ME-...` দিয়ে বদলাতে বলুন। সাধারণ ভুল: (1) Grafana ও API-র পোর্ট সংঘাত, (2) Tempo/Collector-এর 4317 সংঘাত, (3) high-cardinality label (userId/orderId), (4) `/metrics` public-এ খোলা, (5) alert-এ `for` না দেওয়া।